STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

STAR Development Team
Locked Inc.

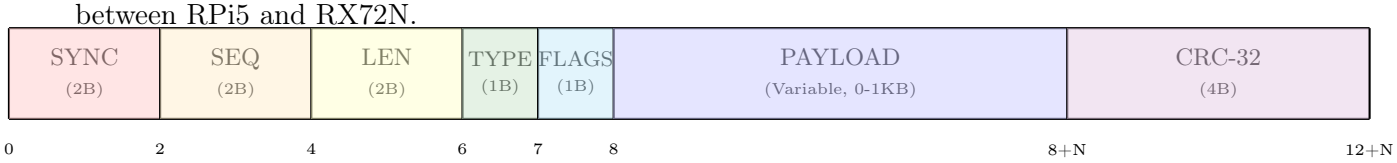
Contents

== == == == == == == == == == == == = =

1 Nanopb (Protobuf) Protocol with HARQ and CRC for SPI Communication

1.1 Executive Summary

Overview : Protocol Buffers(nanopb) encapsulated in frames with CRC - 32 and Hybrid Automatic Repeat Request(HARQ) with Forward Error Correction (FEC) for reliable SPI/USB communication between RPi5 and RX72N.



System Constraints:

- Renesas RX72N : 4MB flash, 1MB SRAM
- 100Hz SPI communication(10ms period)
- 250Hz motor control loop(4ms period)
- Static allocation only(no malloc)

1.1.1 Control Loop Frequency Rationale

Why different loop rates ?

The nested loop architecture follows classic control systems design .The outer loop(SPI at 100Hz) handles command distribution and telemetry collection, while the inner loop(PID at 250Hz) responds quickly to motor dynamics .Running PID at 2.5 times the communication rate ensures smooth control between commands without wasting CPU on excessive computation.

Why 250Hz for motor control?

This is a Nyquist sampling consideration.With DC gearmotors at 210 RPM(3.5 Hz mechanical speed) and a first - order time constant $\tau = 75$ ms(cutoff frequency $f_c = 1/(2\pi\tau) \approx 2.1$ Hz), the 250Hz control rate provides approximately 60 times oversampling of the mechanical dynamics - well above the 2 times Nyquist minimum.

Running at 250Hz balances :

- Adequate sampling of motor dynamics(60 times oversampling)
- Efficient CPU utilization(4 motors $\times$ PID + encoder reads)
- Low latency between velocity commands and motor response
- Power efficiency(fewer ADC conversions, GPIO reads, and computations)

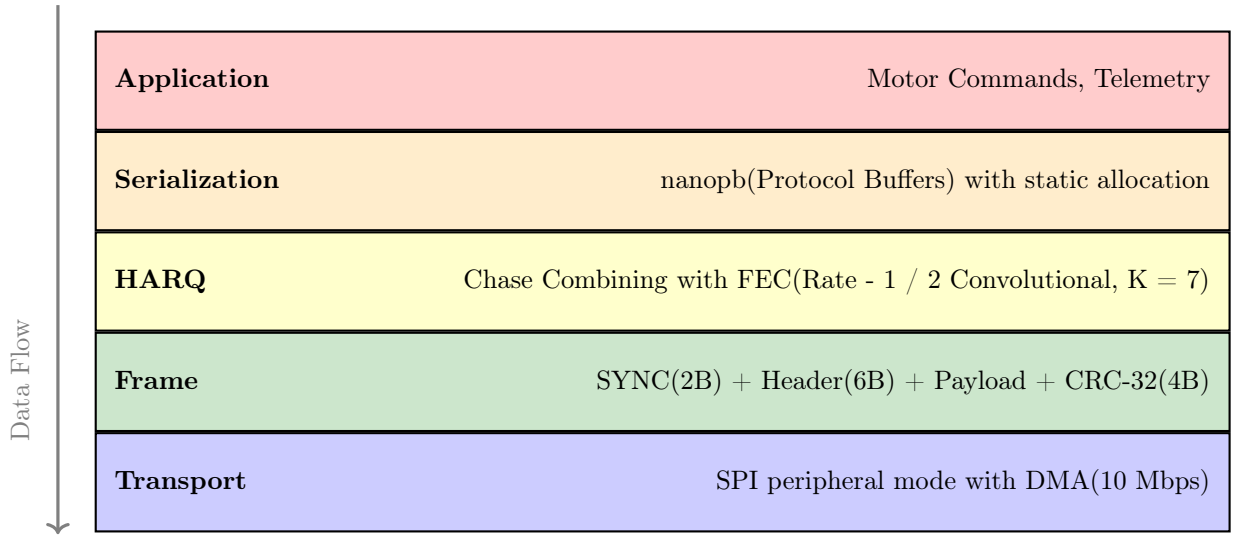
For context, control loop frequencies should match system dynamics :

- DC gearmotors : 50 - 500 Hz typical
- High - speed servos : 1 - 10 kHz

- Motor current control : 10 - 50 kHz
- Temperature control : 0.1 - 10 Hz

1.2 Key Design Decisions

1.2.1 Protocol Stack



1.2.2 Byte Order and Standards Compliance

The protocol uses little-endian byte ordering for all multi-byte fields, matching the native byte order of both the RX72N (Renesas RXv3 core, 32-bit CISC, little-endian) and Raspberry Pi 5 (ARM little-endian). This eliminates byte-swapping overhead on both platforms.

Table 1: Byte Order Standards by Protocol Field

Field	Byte Order	Standard	Rationale
SYNC	Little - endian	Project convention	Matches native byte order on both platforms
SEQ	Little - endian	Project convention	No byte - swap needed on RX72N or RPi5
LEN	Little - endian	Project convention	No byte - swap needed on RX72N or RPi5
TYPE	N / A(1 byte)	–	Single byte, no endianness
FLAGS	N / A(1 byte)	–	Single byte, no endianness
CRC - 32	Little - endian	IEEE 802.3	Standard CRC - 32 polynomial

SYNC Magic Word (0x55AA): The frame synchronization marker uses the distinctive bit pattern 0x55AA (binary: 0101010101010101). This alternating pattern is easily recognizable and unlikely to appear in random data, making it ideal for frame alignment. The value 0x55AA is transmitted in little-endian format as bytes `[0xAA, 0x55]` on the wire.

Little-Endian Convention: All multi-byte header fields (SYNC, SEQ, LEN) use little-endian format, matching the native byte order of both communicating platforms (ARM RPi5 and Renesas RX72N). This eliminates the need for byte-swapping, reducing latency and code complexity in the real-time motor control path.

IEEE 802.3 (CRC-32): The frame checksum uses IEEE 802.3 polynomial (0x04C11DB7) in little-endian format, compatible with standard Ethernet CRC libraries. This allows zero-copy CRC validation using hardware accelerators where available.

Key Insight: Uniform little-endian byte order across all fields eliminates byte-swapping overhead on both RX72N and RPi5, which are natively little-endian. CRC-32 follows IEEE 802.3 for hardware acceleration and library compatibility.

1.2.3 CRC - 32 Implementation

The CRC - 32 module supports both hardware and software implementations with compile - time selection.

Table 2: CRC - 32 Hardware / Software Selection

Build Target	Default Implementation	Override
RX72N(`__RX__` defined)	Hardware CRC Calculator	`-DRX_CRC32_USE_SOFTWARE`
Host(testing)	Software lookup table	N/A(automatic)

Hardware CRC(RX72N) :

- Peripheral : CRC Calculator at `0x00088280`
- Module stop control : `MSTPCRB` bit 23
- Polynomial : IEEE 802.3(0x04C11DB7), LSB - first(reflected)
- Configuration : `CRCCR = 0x43`(CRC - 32 + LSB - first)

Software CRC(Host / Fallback) :

- Algorithm : 256 - entry lookup table(1 KB)
- Polynomial : Reflected form `0xEDB88320`
- Always available for incremental CRC operations

API (identical for both implementations):

```
uint32_t rx_crc32_ieee(const uint8_t *data, size_t len);
uint32_t rx_crc32_update(uint32_t crc, const uint8_t *data, size_t len);
```

Go Compatibility : Both implementations produce bit - exact results matching Go 's `crc32.ChecksumIEEE()`, verified by unit tests including the canonical test vector "123456789" = `0xCBF43926`.

References:

- RX72N Group User's Manual: Hardware (CRC Calculator section)
- https://renesas.github.io/fsp/group___c_r_c.html

1.2.4 Memory Budget

Table 3: SRAM Usage(512 KB total)

Component	Size(KB)	Usage(%)
DMA double - buffer	8.0	1.56
RX / TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
HARQ state	4.0	0.78
HARQ soft buffers(3x)	48.0	9.38
Viterbi decoder	3.0	0.59
Total	**66.5**	**12.99**

Table 4: Flash Usage(16 MB total)

Component	Size(KB)	Usage(%)
Generated protobuf	80	0.49
Nanopb runtime	12	0.07
Frame + HARQ layers	12	0.07
FEC codec(Conv + Viterbi)	8	0.05
Total	**112**	**0.68**

Memory Budget Calculations:

SRAM(512 KB total) :

- **DMA double - buffer :** $2 \times 4092\text{bytes} = 8184\text{bytes} = 8.0\text{KB}$
  Calculation : Two ping - pong DMA buffers at `STAR_SPI_DMA_MAX_SIZE`(4092 bytes each)
  Usage : $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX / TX buffers :** $1024 + 1024 = 2048\text{bytes} = 2.0\text{KB}$
  Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
  Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs :** $1536\text{bytes} = 1.5\text{KB}$
  Calculation : Largest message structures(SetVelocityResponse ≈ 600 bytes, TelemetryData ≈ 400 bytes)
  Usage : $\frac{1.5}{512} \times 100 = 0.29\%$
- **HARQ state :** $4 + 4 + 8 + 4 + 4092 = 4112$ bytes $\approx 4.0\text{KB}$
  Calculation : Sequence number(4B) + retry counter(4B) + timestamp(8B) + state(4B) + retry buffer(4092B)
  Usage : $\frac{4.0}{512} \times 100 = 0.78\%$
- **HARQ soft buffers(3x) :** $3 \times 1024 \times 16 = 49152$ bytes $\approx 48.0\text{KB}$
  Calculation: 3 combining buffers × 1024B max payload ×(2 output bits per input bit × 8 bits / byte = 16 encoded bits / byte)
  Usage : $\frac{48.0}{512} \times 100 = 9.38\%$
- **Viterbi decoder :** ≈ 3.0KB
  Core state: 64-state path metrics (128B) + traceback survivors (280B) = 408B
  Working buffers: New path metrics (128B) + decoded bits (∼1KB for max payload) + temp buffers ≈ 2.5 KB
  Usage: $\frac{3.0}{512} \times 100 = 0.59\%$
- **Total SRAM :** $8.0 + 2.0 + 1.5 + 4.0 + 48.0 + 3.0 = 66.5\text{KB}$
  Total usage : $\frac{66.5}{512} \times 100 = 12.99\%$

Flash(16 MB = 16384 KB total) :

- **Generated protobuf :** 80KB
  Calculation: 6 proto files(common, motor_control, telemetry, battery_management, configuration, firmware_update) × 13 KB avg compiled size
  Usage : $\frac{80}{16384} \times 100 = 0.49\%$

- **Nanopb runtime :** 12KB
  Calculation : Compiled size of pb_encode.c, pb_decode.c, pb_common.c
  Usage : $\frac{12}{16384} \times 100 = 0.07\%$
- **Frame + HARQ layers :** 12KB
  Calculation : Framing logic + CRC - 32 + HARQ state machine + Chase combining logic
  Usage : $\frac{12}{16384} \times 100 = 0.07\%$
- **FEC codec(Conv + Viterbi) :** 8KB
  Calculation : Convolutional encoder + Viterbi decoder(K = 7, rate 1 / 2)
  Usage : $\frac{8}{16384} \times 100 = 0.05\%$
- **Total Flash :** $80 + 12 + 12 + 8 = 112$KB
  Total usage : $\frac{112}{16384} \times 100 = 0.68\%$

1.3 Implementation Requirements

1.3.1 Protocol Stack Architecture

The protocol is implemented as a layered stack on both the RPi5(SPI controller) and RX72N(SPI peripheral) .Each layer has specific responsibilities :

Layer	Responsibility	Implementation
5. Application	Motor commands, telemetry, control logic	Different per platform
4. Protobuf	Message encode / decode(nanopb)	Both sides
3. HARQ	Chase Combining, Convolutional FEC, soft decode	Both sides(symmetric)
2. Framing	Header / footer, CRC - 32 verification	Both sides(identical)
1. SPI + DMA	Physical transport with double - buffering	Both sides(different roles)

Table 5: Protocol stack layers and implementation requirements

Key Insight : Layers 2 - 4 are nearly identical on both platforms.Only the application layer(Layer 5) and SPI role(Layer 1 : controller vs peripheral) differ.

1.3.2 RX72N(SPI Peripheral) Implementation

Layer 1 : SPI Peripheral with DMA

- Configure RSPI in peripheral mode
- Set up DMA with double-buffering: 2×4092 bytes
- Register DMA interrupt callbacks for buffer swapping
- Wait for chip select from RPi5, respond to clock

Layer 2 : Frame Layer(Custom Code)

RX Path:

- Parse frame header(magic bytes, length, sequence number)
- Verify CRC - 32 over header + payload
- If CRC fails → discard packet, send NACK
- If CRC valid → extract payload, pass to ARQ layer

TX Path:

- Build frame : `[Header | Payload | CRC32 | Footer]`
- Calculate CRC-32 over header + payload
- Queue complete frame to DMA buffer for transmission

Layer 3 : HARQ Layer with FEC(Custom Code)

RX Path(with Chase Combining) :

- Extract soft bits from received signal
- If first attempt : Viterbi decode, verify CRC
- If CRC fails : Store soft bits in combining buffer, send NACK
- On retransmission : Combine soft bits with buffer, Viterbi decode
- If decode success : send ACK, pass payload to application

TX Path(with FEC Encoding) :

- Encode payload with rate - 1 / 2 convolutional code(K = 7)
- Assign `tx_sequence_number`, store in retry buffer
- Start timeout timer(10ms), retransmit same frame on NACK
- Chase Combining : receiver accumulates soft bits across retries
- On ACK : clear retry buffer, increment `tx_seq`

Layer 4 : Protobuf(nanopb library)

- Decode incoming bytes → `SetVelocityRequest`, `EmergencyStopRequest`
- Encode outgoing structs → `SetVelocityResponse`, `TelemetryData`
- Uses `star.v1` package following Boston Dynamics style guide
- Static allocation(no malloc) with `.options` file constraints

Layer 5 : Application

- Handle motor velocity commands
- Read encoder data via multiplexer
- Execute PID control loop(250 Hz per motor)
- Send telemetry responses(encoder feedback, current, temperature)

1.3.3 RPi5(SPI Controller) Implementation

Layer 1 : SPI Controller with DMA

- Use Linux `/dev/spidev` interface
- Configure SPI3 : 10 Mbps, mode 0, 8 - bit words
- Use `ioctl(SPI_IOC_MESSAGE)` for DMA transfers
- Implement userspace double-buffering for continuous operation

Layer 2 : Frame Layer *(Custom Code - SAME AS RX72N)*

RX Path : Parse frame header, verify CRC - 32, extract payload or discard on CRC failure

TX Path : Build frame `[Header | Payload | CRC32 | Footer]`, calculate CRC - 32, send via SPI

Layer 3 : HARQ Layer with FEC *(Custom Code - SAME AS RX72N)*

RX Path : Extract soft bits, Viterbi decode, Chase Combining on retry, verify CRC, send ACK / NACK

TX Path : FEC encode(rate - 1 / 2, K = 7), assign sequence, store in retry buffer, retransmit identical frame on NACK

Layer 4 : Protobuf(Go with protoc - gen - go)

- Encode : `SetVelocityRequest`, `EmergencyStopRequest`
- Decode : `SetVelocityResponse`, `TelemetryData`, `EncoderData`
- Uses `star.v1` package following Boston Dynamics style guide

Layer 5 : Application(star - gateway)

- Send motor velocity commands from Nav2 (autonomous) or UI (manual)
- All control flows through Nav2 stack for consistent behavior
- Receive encoder feedback for odometry calculations
- Bridge ROS2 topics to/from RX72N via SPI

1.3.4 HARQ Symmetry and State Management

The HARQ layer uses **Chase Combining** with Stop-and-Wait protocol. Each side sends one FEC-encoded frame at a time, waits for acknowledgment (ACK) with a timeout, and retries up to 3 times. On the receiver side, soft bits from failed transmissions are stored and combined with subsequent retries, improving decode probability with each attempt. This approach provides:

- Simple implementation suitable for embedded systems
- Predictable worst-case latency for real-time control
- 4-5 dB coding gain from FEC reduces retry frequency
- Soft combining improves success probability with each retry
- Sufficient throughput for 10 Mbps SPI with <1ms frame time

The HARQ protocol is **symmetric** – both sides implement identical Chase Combining logic with FEC encode/decode but maintain separate state for their own transmissions and receptions.

Each Side Maintains:

- **TX sequence number** (`tx_seq`): For packets it sends
- **RX last acknowledged** (`rx_last_ack`): For packets it receives
- **Retry buffer**: Stores FEC-encoded outgoing packets for retransmission
- **Soft combining buffer**: Stores soft bits from failed RX attempts (3 slots)
- **Timeout timer**: Triggers retransmission if no ACK received

Responsibility	**RX72N(Peripheral)**	**RPi5(Controller)**
Sends	Telemetry, responses	Commands, requests
TX sequence numbering	Own `tx_seq`	Own `tx_seq`
ACK/NACK generation	Sends ACK for RPi5	Sends ACK for RX72N
Retry logic	Retries if no ACK	Retries if no ACK
Duplicate detection	Checks RPi5's `seq`	Checks RX72N's `seq`

Table 6: HARQ role distribution between controller and peripheral

Important : The SPI controller / peripheral roles(Layer 1) are **hardware - level only**.The HARQ layer(Layer 3) treats both sides as equals – each can send, FEC encode / decode, combine soft bits, and retry independently.

1.3.5 Example Transaction : RPi5 Sends Velocity Command

1. **RPi5 Application :** Create `SetVelocityRequest`(left=1.0 m / s, right=1.0 m / s)
2. **RPi5 Protobuf :** Encode to binary(nanopb)
3. **RPi5 HARQ :** FEC encode(rate - 1 / 2, K = 7), assign `seq = 42`, save to retry buffer, start 10ms timeout
4. **RPi5 Frame :** Build packet with header, CRC - 32, footer
5. **RPi5 SPI :** DMA transfer to RX72N(controller initiates)
6. **RX72N SPI :** DMA receives packet in double - buffer
7. **RX72N Frame :** Extract soft bits from received signal
8. **RX72N HARQ :** Viterbi decode, verify CRC - 32 → valid, check `seq = 42` = `rx_last_ack + 1` → accept
9. **RX72N Protobuf :** Decode to `SetVelocityRequest` struct
10. **RX72N Application :** Update motor setpoints, execute PID control
11. **RX72N HARQ :** Build ACK packet with `ack_seq = 42`
12. **RX72N Frame :** Wrap ACK in frame with CRC - 32

13. **RX72N SPI :** DMA transmits ACK(peripheral responds)
14. **RPi5 SPI :** DMA receives ACK packet
15. **RPi5 Frame :** Verify CRC - 32 → valid
16. **RPi5 HARQ :** Got `ACK(42)` → clear retry buffer, cancel timeout

Outcome : Command successfully delivered with guaranteed delivery via HARQ.If CRC failed, RX72N would store soft bits and NACK; on retry, it combines soft bits from both attempts before Viterbi decoding, improving decode probability.

1.4 HARQ Communication Flow

1.4.1 Normal Operation(No Retransmission)

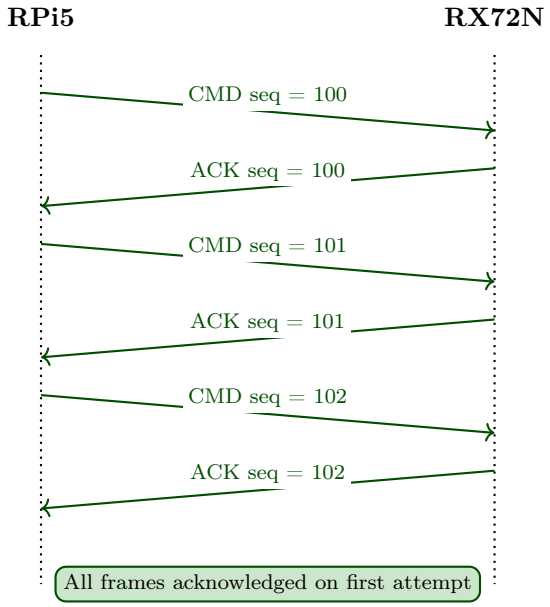


Figure 1: Normal HARQ Flow - Three Successful Transactions

1.4.2 Error Recovery Scenarios

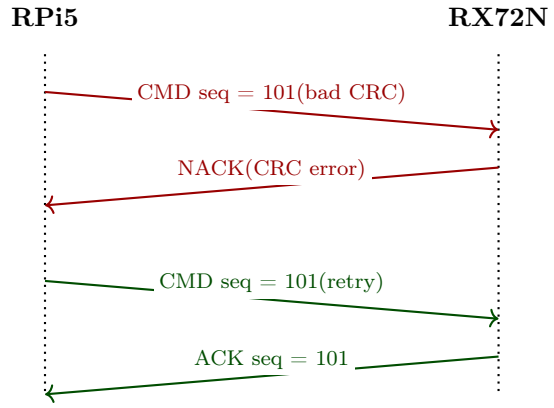


Figure 2: CRC Error - Retry with Same Sequence Number

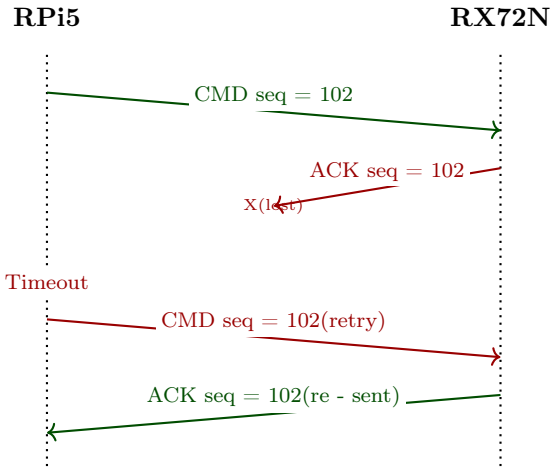


Figure 3: Duplicate Packet Scenario(Lost ACK)

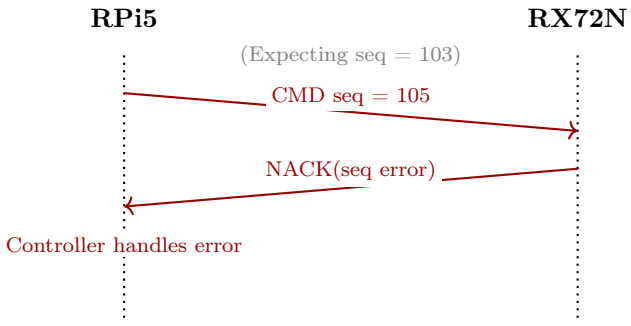


Figure 4: Out - of - Order Packet Scenario

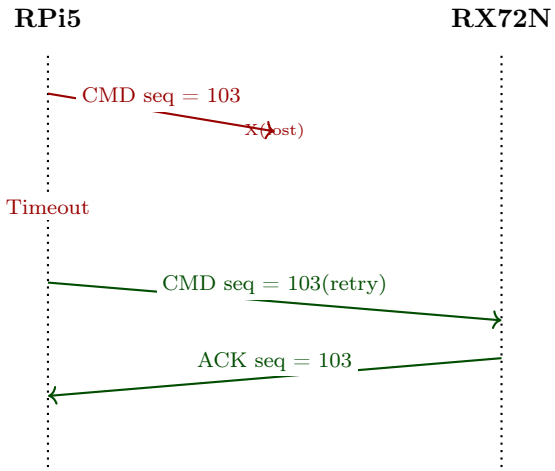


Figure 5: Lost Packet Scenario(Timeout)

1.5 Error Handling Strategies

CRC Mismatch
Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment crc_errors

Sequence Error
Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout
RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

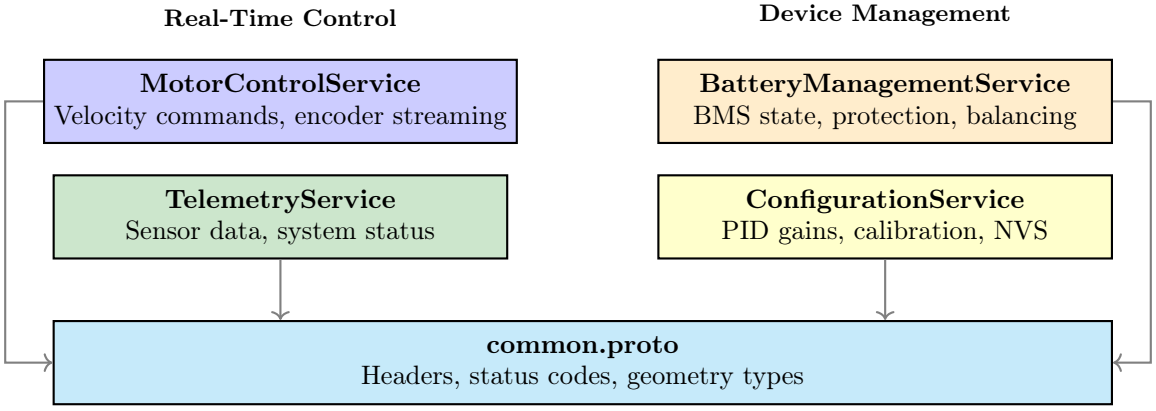
Invalid Protobuf
Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment proto_errors

2 Protocol Buffer Schema Architecture

This section documents the Protocol Buffer schema architecture for RPi5↔RX72N communication. All schemas follow the **Boston Dynamics Protobuf Style Guide** for consistency, maintainability, and industry best practices.

2.1 Architecture Overview

Overview: Five gRPC services define the complete API surface for robot control, organized by functional domain. Services communicate via the nanopb-serialized frames described in Section 1.



Key Design Principles

- **Versioned Package :** All services use `star.v1` namespace for API evolution
- **Request/Response Headers:** Every RPC includes tracing, timestamps, and status
- **Streaming Support:** Continuous data uses server streaming (up to 100Hz)
- **Static Allocation:** All messages sized for nanopb constraints (no dynamic memory)

Note: Naming conventions, unit suffixes, and coding standards are consolidated in Section 4 (Style Guide and Conventions).

2.2 Service Specifications

2.2.1 MotorControlService

Purpose: Low-latency differential drive control with real-time encoder feedback.

Target Latency: <10ms round-trip for velocity commands.

Table 7: MotorControlService RPCs

RPC	Type	Description
SetVelocity	Unary	Set left / right wheel velocities(m / s)
EmergencyStop	Unary	Immediate stop, priority over all commands
SetMotorPower	Unary	Direct PWM control(bypass PID, testing only)
StreamEncoders	Server stream	Encoder ticks and velocity at 1 - 100Hz
ControlStream	Bidirectional	Velocity in, encoder feedback out

Key Messages:

- `VelocityCommand` –Left / right velocity(m / s), sequence number, timestamp
- `EncoderData` –Motor ID, tick count(341.2 PPR), velocity, timestamp
- `MotorStatus` –Duty cycle, velocity, temperature, current, fault flags
- `PidConfig` –Kp, Ki, Kd gains with output saturation limits

2.2.2 TelemetryService

Purpose : Real - time sensor data aggregation and system health monitoring.

Table 8: TelemetryService RPCs

RPC	Type	Description
GetTelemetry	Unary	Snapshot of all sensor data
StreamTelemetry	Server stream	Continuous sensor updates at 1 - 100Hz
GetSystemStatus	Unary	Firmware version, uptime, connection states

Key Messages:

- `TelemetryData` –IMU, GPS, battery %, WiFi signal, CPU usage, temperature
- `ImuData` –Pitch / roll / yaw(rad), acceleration(m / s^2), angular velocity(rad / s)
- `GpsData` –Lat / lon(degrees), altitude(m), accuracy, satellite count, fix type
- `SystemStatus` –Connection states, operating mode, free heap, uptime

Enumerations:

- `RobotMode` –IDLE, MANUAL, AUTONOMOUS, MAPPING, EMERGENCY_STOP
- `ConnectionStatus` –CONNECTED, DISCONNECTED, CONNECTING, ERROR
- `GpsFix` –NONE, 2D, 3D, DGPS, RTK_FLOAT, RTK_FIXED

2.2.3 BatteryManagementService

Purpose: BQ7850 BMS monitoring for lithium-ion battery pack safety and management.
Hardware: TI BQ7850 16-cell BMS with I2C interface.

Table 9: BatteryManagementService RPCs

RPC	Type	Description
GetBatteryState	Unary	Complete battery snapshot(cells, temps, SOC)
StreamBatteryState	Server stream	Continuous battery monitoring
Get / SetProtectionThresholds	Unary	OV / UV / OC / OT protection limits
Enable / DisableCellBalancing	Unary	Per - cell passive balancing control
ControlFets	Unary	Charge / discharge FET enable / disable
GetDeviceInfo	Unary	Manufacturer, serial, chemistry, versions

Key Messages:

- `BatteryState` –Composite of all battery data
- `CellData` –Individual cell voltages(mV), pack voltage, valid cell count
- `BatteryTemperatureData` –Thermistor readings(0.1°C), average
- `StateOfChargeData` –Relative / absolute SOC(%), capacity(mAh), cycle count
- `ProtectionThresholds` –OV / UV voltage limits, OC current limits, OT temperatures

2.2.4 ConfigurationService

Purpose : Runtime parameter management with NVS persistence across power cycles.

Table 10: ConfigurationService RPCs

RPC	Type	Description
GetConfiguration	Unary	Read current system configuration
SetConfiguration	Unary	Apply configuration(optional NVS persist)
ResetToDefaults	Unary	Restore factory defaults
ValidateConfiguration	Unary	Validate without applying
SaveConfiguration	Unary	Persist in - memory config to NVS
Get / SetMotorPidConfig	Unary	Per - motor PID tuning

Key Messages:

- `SystemConfiguration` –Complete config with version and CRC32
- `MotorPidConfig` –Per - motor PID gains(4 motors supported)
- `CurrentSensorCalibration` – Offset and scale for current sensing

- `SafetyThresholds` –Overcurrent, thermal shutdown, cell imbalance limits
- `TimingConfig` –Motor period, telemetry period, communication timeout

2.3 common.proto– Shared Types

All services import `common.proto` for standardized request / response handling.

Table 11: Standard Headers

Message	Fields
RequestHeader	request_id(UUID), client_timestamp, client_version, timeout
ResponseHeader	request_id(echo), server_timestamp, status, error_message, latency_us

Status Codes : UNKNOWN, OK, INVALID_REQUEST, INTERNAL_ERROR, NOT_FOUND, TIMEOUT, INVALID_STATE, ESTOP_ACTIVE

Geometric Types:

- `Vector3` –Generic 3D vector(x, y, z)
- `Quaternion` –Orientation(w, x, y, z)
- `SE2Pose` –2D pose(x_m, y_m, angle_rad)
- `SE2Velocity` –2D velocity(vx_mps, vy_mps, omega_rad_per_s)

2.4 Code Generation and Integration

Table 12: Code Generation Targets

Target	Generator	Output	Usage
Go	protoc-gen-go + grpc	`gen/go/`	star-gateway(RPi5)
nanopb	nanopb_generator	`gen/nanopb/`	RX72N firmware

nanopb Constraints: RX72N requires static allocation. Field sizes are configured in `.options` files:

- String fields: `max_size:64` (request_id), `max_size:128` (error_message)
- Repeated fields: `max_count:16` (cell voltages), `max_count:4` (motors)
- Bytes fields: `max_size:1024` (firmware chunks)

Build Integration:

- **Linting:** `buf lint` enforces style guide compliance
- **Breaking Changes:** `buf breaking` detects API incompatibilities
- **CI Pipeline :** Automatic generation and testing on PR

3 Hardware Pin Connections

3.1 RX72N Microcontroller Pinout– - 144 - Pin LFQFP(R5F572NNHxFB)

Color Legend:

Function Category	Color
PWM Motor Control(GPTW)	Blue
Encoder Inputs(MTU / TPU)	Green
Host SPI / IRQ Communication(RSPI2)	Orange
I2C Communication(RIIC0 / RIIC1)	Violet
ADC Current Sensing	Red
Debug / JTAG / Boot / USB	Yellow
Power / Ground	Gray
GTETRG Emergency Stop / Temp / Alert	Lime
Motor Driver SPI(SCI7)	Purple
Ultrasonic Sonar(HC - SR04)	Cyan
Status LEDs	Teal
Unused / Available	White

Pin	Pin Name	Connected To	Note / Comment
1	AVSS0	Ground	Analog ground
2	P05 / IRQ13	BMS_ALERT	BMS alert interrupt(active low)
3	AVCC1	Boards 3v3	Analog power supply(3.3V, connect to VCC via branch with 0.1µF bypass to AVSS1)
4	P03/IRQ11	Sonar 0 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
5	AVSS1	Ground	Analog ground
6	P02/IRQ10	Sonar 1 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
7	P01/IRQ9	Sonar 2 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
8	P00/IRQ8	Sonar 3 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
9	PF5	Sonar 0 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
10	EMLE	Emulator Configuration jumper	See Table **??**
11	PJ5	Sonar 1 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
12	VSS	Ground	Power supply
13	PJ3	Sonar 2 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
14	VCL	220nF cap to ground	Voltage regulator
15	VBATT	Boards 3v3	Battery backup

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
16	MD / FINED	E1E2 - Emulator - Interface, DIP switch	See Table **??**.Default : OFF, OFF
17	XCIN	10k resistor to ground	RTC crystal input(not used)
18	XCOUT	Floating	RTC crystal output(not used)
19	RES#	E1E2 - Emulator - Interface, Reset button	MCU reset
20	P37 / XTAL	24MHz crystal(to pin 22)	System clock crystal
21	VSS	Ground	Power supply
22	P36 / EXTAL	24MHz crystal(to pin 20)	System clock crystal
23	VCC	Boards 3v3	Power supply
24	P35/NMI/UPSEL	UPSEL configuration jumper	Can be used for NMI (not used). See Table **??**
25	P34 / TRST#	E1E2 - Emulator - Interface	JTAG(TRST#)
26	P33	Sonar 3 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
27	P32	Unused	Available GPIO
28	P31 / TMS	E1E2 - Emulator - Interface	JTAG(TMS)
29	P30 / TDI	E1E2 - Emulator - Interface	JTAG(TDI)
30	P27 / TCK	E1E2 - Emulator - Interface	JTAG(TCK)
31	P26 / TDO	E1E2 - Emulator - Interface	JTAG(TDO)
32	P25 / MTCLKB	Encoder 0 Phase B	MTU1 encoder(quadrature counting)
33	P24 / MTCLKA	Encoder 0 Phase A	MTU1 encoder(quadrature counting)
34	P23 / GTIOC0A	Motor 0 PH(Phase)	GPTW PWM channel 0A(32 - bit)
35	P22 / GTIOC1A	Motor 1 PH(Phase)	GPTW PWM channel 1A(32 - bit)
36	P21/SCL1	BMS I2C Clock	RIIC1 for BMS (dedicated bus)
37	P20/SDA1	BMS I2C Data	RIIC1 for BMS (dedicated bus)
38	P17 / GTIOC0B	Motor 0 EN(Enable)	GPTW PWM channel 0B(32 - bit)
39	P87	Unused	Available GPIO
40	P16 / USB0_VBUS	USB0 VBUS detection	USB CDC debug interface
41	P86 / GTIOC2B	Motor 2 EN(Enable)	GPTW PWM channel 2B(32 - bit)
42	P15 / GTETRGA	Motor 0 nFAULT	GPTW hardware emergency stop(auto - disables PWM)
43	P14 / GTETRGD	Motor 3 nFAULT	GPTW hardware emergency stop(auto - disables PWM)
44	P13/SDA0	Host I2C Data	RIIC0 for RPi5 communication (FM+ capable)
45	P12/SCL0	Host I2C Clock	RIIC0 for RPi5 communication (FM+ capable)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
46	VCC_USB	Boards 3v3	USB power supply
47	USB0_DM	USB0 DN(27R resistor)	USB data -
48	USB0_DP	USB0 DP(27R resistor)	USB data +
49	VSS_USB	Ground	USB ground
50	P56	Unused	Available GPIO
51	P55	Unused	Available GPIO
52	P54	Unused	Available GPIO
53	P53	Unused	Not available as GPIO when external bus enabled(BCLK)
54	P52	Unused	Available GPIO
55	P51	DS18B20 DQ	1 - Wire temperature sensor data pin
56	P50	Unused	Available GPIO(WR0# / WR#, SCI2 TX)
57	VSS	Ground	Power supply
58	P83	Unused	Available GPIO(TRCLK, GTIOC0A, SCK10, Ethernet)
59	VCC	Boards 3v3	Power supply
60	PC7 / UB	E1E2 - Emulator - Interface, DIP switch	Operation mode configuration.See Table **??**
61	PC6 / GTIOC3B	Motor 3 EN(Enable)	GPTW PWM channel 3B(32 - bit)
62	PC5 / MTCLKD	Encoder 1 Phase B	MTU2 encoder(quadrature counting)
63	P82	Unused	Available GPIO(GTIOC2A, SCI10, Ethernet)
64	P81	Unused	Available GPIO(GTIOC0B, SCI10, Ethernet, QSPI)
65	P80	Unused	Available GPIO(SCK10, Ethernet, QSPI)
66	PC4 / GTETRGC	Motor 2 nFAULT	GPTW hardware emergency stop(auto - disables PWM)
67	PC3 / GTIOC1B	Motor 1 EN(Enable)	GPTW PWM channel 1B(32 - bit)
68	P77	Unused	Available GPIO(SCI11, Ethernet, QSPI, SDHI)
69	P76	Unused	Available GPIO(SCI11, Ethernet, QSPI, SDHI)
70	PC2 / TCLKA	Encoder 2 Phase A	TPU1 encoder(quadrature counting, 16 - bit)
71	P75	Unused	Available GPIO(SCI11, Ethernet, SDHI)
72	P74/CS4#	Motor Driver SPI nCS0	SCI7 chip select for Motor 0 (DRV8243S)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
73	PC1	Motor Driver SPI nCS1	SCI7 chip select for Motor 1 (DRV8243S)
74	VCC	Boards 3v3	Power supply
75	PC0 / TCLKC	Encoder 3 Phase A	TPU2 encoder(quadrature counting, 16 - bit)
76	VSS	Ground	Power supply
77	P73	Unused	Available GPIO(Ethernet)
78	PB7 / TXD9	Debug SCI USB - C RX	UART crossed.See Table **??**
79	PB6 / RXD9	Debug SCI USB - C TX	UART crossed.See Table **??**
80	PB5	Motor Driver SPI nCS2	SCI7 chip select for Motor 2 (DRV8243S)
81	PB4	Motor Driver SPI nCS3	SCI7 chip select for Motor 3 (DRV8243S)
82	PB3 / TCLKD	Encoder 3 Phase B	TPU2 encoder(quadrature counting, 16 - bit)
83	PB2	LED5	Status LED(GPIO output)
84	PB1	LED4	Status LED(GPIO output)
85	P72	LED3	Status LED(GPIO output)
86	P71	LED2	Status LED(GPIO output)
87	PB0	LED1	Status LED(GPIO output)
88	PA7	LED0	Status LED(GPIO output)
89	PA6 / GTETRGB	Motor 1 nFAULT	GPTW hardware emergency stop(auto - disables PWM)
90	PA5	Unused	Available GPIO(GTIOC0A, RSPI_A SCLK, Ethernet)
91	VCC	Boards 3v3	Power supply
92	PA4	Unused	Available GPIO(RSPI_A CS, Ethernet, IRQ5)
93	VSS	Ground	Power supply
94	PA3 / TCLKB	Encoder 2 Phase B	TPU1 encoder(quadrature counting, 16 - bit)
95	PA2	Unused	Available GPIO(GTIOC1A, SCI5 RX)
96	PA1 / MTCLKC	Encoder 1 Phase A	MTU2 encoder(quadrature counting)
97	PA0	Unused	Available GPIO(GTIOC0B, Ethernet)
98	P67 / IRQ15	HOST_IRQ	Host interrupt output to RPi5(active - low, data ready)
99	P66	Unused	Available GPIO(GTIOC2B, CAN2 TX)
100	P65	Unused	Available GPIO(external bus CKE)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
101	PE7 / GTIOC3A	Motor 3 PH(Phase)	GPTW PWM channel 3A(32 - bit)
102	PE6 / GTIOC3B	Motor 3 EN(Enable)	GPTW PWM channel 3B(32 - bit)
103	VCC	Boards 3v3	Power supply
104	P70	Unused	Available GPIO(SDCLK)
105	VSS	Ground	Power supply
106	PE5 / GTIOC0A	Motor 0 PH(Phase)	GPTW PWM channel 0A(32 - bit)
107	PE4 / GTIOC1A	Motor 1 PH(Phase)	GPTW PWM channel 1A(32 - bit)
108	PE3 / GTIOC2A	Motor 2 PH(Phase)	GPTW PWM channel 2A(32 - bit)
109	PE2 / GTIOC0B	Motor 0 EN(Enable)	GPTW PWM channel 0B(32 - bit)
110	PE1 / GTIOC1B	Motor 1 EN(Enable)	GPTW PWM channel 1B(32 - bit)
111	PE0 / GTIOC2B	Motor 2 EN(Enable)	GPTW PWM channel 2B(32 - bit)
112	P64	Unused	Available GPIO(external bus, Ethernet)
113	P63	Unused	Available GPIO(external bus, Ethernet)
114	P62	Unused	Available GPIO(external bus, Ethernet)
115	P61	Unused	Available GPIO(external bus, Ethernet)
116	VSS	Ground	Power supply
117	P60	Unused	Available GPIO(external bus, Ethernet)
118	VCC	Boards 3v3	Power supply
119	PD7	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ7, AN107)
120	PD6	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ6, AN106)
121	PD5	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ5, AN113)
122	PD4 / SSLC0	RPi5 RSPI2 CS	RSPI2 - A chip select
123	PD3 / RSPCKC	RPi5 RSPI2 SCLK	RSPI2 - A clock
124	PD2 / MISOC	RPi5 RSPI2 CIPO	RSPI2 - A Controller In Peripheral Out
125	PD1 / MOSIC	RPi5 RSPI2 COPI	RSPI2 - A Controller Out Peripheral In
126	PD0	Unused	Available GPIO(GTIOC1B, IRQ0, AN108)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
127	P93	Unused	Available GPIO(SCI7, Ethernet, AN117)
128	P92 / SMISO7	Motor Driver SPI CIPO	SCI7 controller in from DRV8243S
129	P91 / SCK7	Motor Driver SPI SCLK	SCI7 clock to all 4 DRV8243S drivers
130	VSS	Ground	Power supply
131	P90 / SMOSI7	Motor Driver SPI COPI	SCI7 controller out to DRV8243S
132	VCC	Boards 3v3	Power supply
133	P47 / AN007	Motor 0 Current Sense	ADC Unit 0, 12 - bit, 2.0A max(IPROPI from DRV8243S)
134	P46 / AN006	Motor 1 Current Sense	ADC Unit 0, 12 - bit, 2.0A max(IPROPI from DRV8243S)
135	P45 / AN005	Motor 2 Current Sense	ADC Unit 0, 12 - bit, 2.0A max(IPROPI from DRV8243S)
136	P44 / AN004	Motor 3 Current Sense	ADC Unit 0, 12 - bit, 2.0A max(IPROPI from DRV8243S)
137	P43	Unused	Available GPIO(IRQ11, AN003)
138	P42	Unused	Available GPIO(IRQ10, AN002)
139	P41	Unused	Available GPIO(IRQ9, AN001)
140	VREFL0	Ground	ADC reference low
141	P40	Unused	Available GPIO(IRQ8, AN000)
142	VREFH0	3.3V	ADC reference high
143	AVCC0	Boards 3v3	ADC power supply
144	P07	DBG_RESETn	Debug reset(active low)

3.2 Motor Control Architecture

3.2.1 GPTW vs MTU Selection for PWM Generation

The RX72N provides two timer peripherals suitable for motor control PWM generation: the General PWM Timer (GPTW) and the Multi-Function Timer Unit (MTU). After detailed analysis, **GPTW was selected** for the following technical reasons :

Resolution and Performance:

- **GPTW:** 32 - bit timer resolution enabling precise duty cycle control
- **MTU:** 16 - bit timer resolution(65, 536 maximum count)

- At 20 kHz PWM frequency with 120 MHz peripheral clock :
  - GPTW provides 6, 000 discrete duty cycle steps(120 MHz / 20 kHz)
  - MTU limited to maximum 3, 276 steps before counter overflow

Channel Architecture:

- **GPTW:** 4 independent channels, each with 2 complementary outputs(GTIOC0A / B through GTIOC3A / B)
- **Perfect match** for 4 motors in Phase / Enable(PH / EN) control mode
- Each motor driver(DRV8243S) requires exactly 2 PWM signals

Peripheral Separation:

- Using GPTW for PWM provides dedicated high-resolution motor control
- Encoders use MTU (front wheels) and TPU (rear wheels) for phase counting
- No resource conflicts between PWM generation and encoder reading

Encoder Peripheral Selection:

- **Front encoders (0, 1)**: MTU1/MTU2 - 32-bit counters with phase counting support
- **Rear encoders (2, 3)**: TPU1/5, TPU2/4 - 16-bit counters with phase counting support
- **IMPORTANT**: MTU3 and MTU4 do NOT support phase counting mode per RX72N Manual Ch24 §24.1.2
- Only MTU1 and MTU2 have phase counting capability, hence TPU required for rear encoders
- TPU provides adequate performance for 210 RPM max speed (13.7s overflow period with 16-bit counter)
- Software overflow detection at 100 Hz control loop provides 137× safety margin

Pin Flexibility:

- GPTW signals available on multiple pin locations via MPC(Multi - Function Pin Controller)
- Example : GTIOC0A available on pins 34, 90, 106, 123
- Enabled conflict - free allocation by selecting alternate pin groups

3.2.2 Pin Allocation Strategy

Design Philosophy:

1. **Peripheral Grouping :** Place related functions on contiguous port pins where possible
   - GPTW PWM : All motors on PORT E(PE0 - PE7, pins 101 - 102, 106 - 111)
   - Host SPI to RPi5 : PD1 - PD4(RSPI2_A, pins 122 - 125)
   - Motor Driver SPI : P90 - P92(SCI7, pins 131 / 129 / 128) + 4 CS pins

- MTU Encoders : P24 / P25(MTU1), PA1 / PC5(MTU2)
- TPU Encoders : PC2 / PA3(TPU1), PC0 / PB3(TPU2)
- I2C : P12 - P13(RIIC0 Host), P20 - P21(RIIC1 BMS)

2. **Conflict Resolution via Alternate Functions:**
   - 144-pin package provides more pins, reducing conflicts vs. 100-pin
   - Table 1.9 analysis revealed all peripherals have 2-4 alternate pin locations
   - SCI7 used for motor driver SPI (moved from SCI12 to resolve PE0 / PE1 / PE2 conflict)
   - GTETRG hardware triggers used for motor nFAULT (replaces software IRQ)
   - Dedicated RIIC1 for BMS (previously shared RIIC0 with host in 100-pin)
3. **PCB Routing Optimization:**
   - Host SPI signals on PD1-PD4 (pins 122-125) for compact layout
   - Motor driver SPI data on P90 / P91 / P92(SCI7, pins 131 / 129 / 128)
   - Motor PWM signals grouped on PORT E(PE0 - PE7, pins 101 - 102, 106 - 111)
   - Sonar sensors grouped on pins 4-13 (top of package)
   - ADC current sense on P44-P47 (pins 133-136) for short analog traces
4. **Future Expandability:**
   - 43 GPIO pins available after critical allocations (of 144 total)
   - Unused Port D pins (PD5-PD7) available for additional QSPI/SDHI
   - All Port E GPTW pins (PE0-PE7) allocated to 4-motor PWM control
   - Ethernet pins (Port B/P7x) available for future network connectivity
   - Additional ADC channels available (AN000-AN003, AN100-AN117)

3.2.3 Communication Bandwidth Analysis

SPI Configuration: 10 Mbps is Optimal

Peak bandwidth calculation for RPi5 $\leftrightarrow$ RX72N communication:

- **Velocity Commands(100 Hz) :**
  - Message size : 34 bytes(Nanopb encoded MotorVelocityCommand)
  - Bandwidth : 34 bytes $\times$ 100 Hz $\times$ 8 = 27.2Kbps
- **Encoder Feedback(100 Hz) :**
  - Message size : 18 bytes(4 encoders $\times$ 4 bytes + header)
  - Bandwidth : 18 bytes $\times$ 100 Hz $\times$ 8 = 14.4Kbps
- **Telemetry(50 Hz) :**
  - Message size : 64 bytes(battery, motor current, temperature, status)
  - Bandwidth : 64 bytes $\times$ 50 Hz $\times$ 8 = 25.6Kbps

- **ARQ Protocol Overhead(estimated 40%) :**
  - Headers : 8 bytes per frame(sequence, CRC - 32, flags)
  - Retransmissions : Average 5% packet loss × 3 retry attempts
  - Total overhead : $(27.2 + 14.4 + 25.6) \times 1.4 = 94.1$Kbps

Total Peak Bandwidth :

$$\text{Peak} = 27.2 + 14.4 + 25.6 + 94.1 = \mathbf{161.3\ Kbps}$$

SPI Utilization :

$$\frac{161.3\text{Kbps}}{10\text{Mbps}} = \mathbf{1.6\%\ utilization}$$

Margin Analysis:

- Available headroom: $10,000/161.3 = 62\times$ current requirements
- No need for Ethernet (would add PHY chip, 9+ pins, PCB complexity)
- No need for QSPI (RPi5 doesn't support quad-lane SPI natively)
- 10 Mbps provides excellent margin for future features (camera data, additional sensors)

3.2.4 DS18B20+ Temperature Sensor

The DS18B20+ is a digital 1-Wire temperature sensor connected to P51 (pin 55).

Specifications:

- **Temperature Range:** -55°C to +125°C
- **Accuracy:** ±0.5°C from -10°C to +85°C
- **Resolution:** 9-bit to 12-bit configurable (0.5°C to 0.0625°C)
- **Interface:** 1-Wire protocol on DQ pin (P51)
- **Power:** Parasite power or separate VDD (3.0V to 5.5V)

Pin Connections:

- **DQ (Data):** P51 (pin 55) - 1-Wire data line
- **VDD:** 3.3V power supply
- **GND:** Ground

Required External Components:

- 4.7kΩ pull-up resistor on DQ line (P51 to 3.3V)
- 0.1μF decoupling capacitor near VDD pin
- Optional: 100nF ceramic capacitor if using parasite power mode

Firmware Implementation:

- Use 1-Wire protocol library (timing-critical, requires precise delays)
- Typical read cycle: 750ms for 12-bit conversion
- Multiple sensors can share same 1-Wire bus (each has unique 64-bit ROM code)

Use Cases:

- Ambient temperature monitoring
- Motor driver thermal management (supplement DRV8243S internal temp)
- Battery temperature monitoring
- System enclosure temperature

3.3 Motor Driver Dual-Interface Architecture

The DRV8243S motor drivers use two separate interfaces:

Real - Time Control(PWM) :

- **Signals:** PH (Phase) and EN (Enable) distributed across GPTW channels 0-3
- **Purpose:** Motor speed/direction control at 20 kHz PWM frequency
- **Update rate:** 100 Hz velocity commands from RPi5

Configuration(SPI) :

- **Peripheral:** SCI7 hardware SPI using dedicated motor driver SPI pins
- **Purpose:** Configure current limits, fault thresholds, slew rates, dead-time insertion
- **Update rate:** Startup only, or when changing operating parameters
- **Speed:** 1 MHz SPI clock (configuration is not time-critical)
- **Signals:** SCLK (P91), COPI (P90), CIPO (P92), nCS0 (P74), nCS1 (PC1), nCS2 (PB5), nCS3 (PB4)

Design Rationale:

- PWM provides low-latency real-time control for motor speed/direction
- SPI allows runtime configuration of driver protection features
- Separation of concerns: high-frequency control vs. low-frequency configuration
- Uses SCI7 to keep RSPI2 available for RPi5 communication

3.4 RX72N Configuration Tables

Table 14: Emulator Configuration Jumper Settings

Jumper Position	**Configuration**
Shorted Pin 1-2 (normal Operation)	E1/E2 Lite normal debugging (use this mode for normal operation). Microcontroller single operation (without emulator)
Shorted Pin 2-3 (debug with E1/E2)	E1/E2 Lite debugging with Hot plug-in
All open	DO NOT SET

Table 15: Operation Configuration Mode DIP Switch Settings

Note: There are 2 USB-C ports. To flash we can use the SCI boot mode if we want to flash with the SCI USB-C. To flash we can use the USB0 boot mode if we want to flash with the USB0 USB-C.

Pin1	**Pin2**	**UPSEL_CFG**	**OP Mode**
OFF	X	X	Single Chip Mode
ON	OFF	X	SCI Boot Mode (SCI1)
ON	ON	1/2	USB (Bus Powered) (USB0)
ON	ON	2/3	USB (Self Powered) (USB0)

Table 16: UPSEL Power Configuration for USB Boot Mode

Jumper Position	**Power Configuration for USB Boot Mode**
Shorted Pin 1-2 (No battery pack)	Bus Powered
Shorted Pin 2-3 (Battery Pack)	Self Powered

Table 17: Required Pin States Upon Reset Release

To enter a mode capable of flashing firmware, the following pin configurations must be held when the **RES#pin** goes from Low to High:

Desired Flashing Mode	**MD Pin Level**	**UB Pin Level**	**Interface Used**
Boot Mode (SCI)	Low	Low	Serial (SCI1)
Boot Mode (USB)	Low	High	USB
Boot Mode (FINE)	Low → High*	Low	FINE Interface

*For FINE interface, the MD pin must be Low at reset release and then switched to High within 20 to 100 ms.

Table 18: SCI9 Firmware Configuration(MPC Settings)

Note: Pins are multiplexed. Firmware must explicitly configure the Multi-Function Pin Controller (MPC) as follows:

Step	Configuration
1. UNLOCK MPC	Unlock registers using the Write-Protect Register (PWPR)
2. PIN FUNCTION SELECT	**PIN 78 (PB7) → TXD9:** Set PB7PFS.PSEL[5:0] = 001010b (0x0A) **PIN 79 (PB6) → RXD9:** Set PB6PFS.PSEL[5:0] = 001010b (0x0A)
3. ENABLE PERIPHERAL	Set Port Mode Register (PMR) bit to '1' for both PB7 and PB6

USB-UART IC Configuration: Use Cypress configuration utility to enable BUSDETECT.

3.5 Raspberry Pi 5 Pinout

Status: Pin assignments are pending hardware verification and will be documented once validated.

3.6 Additional Peripheral Connections

Status: Peripheral connections are pending hardware verification and will be documented once validated.

== == == == == == == == == == == == = =

4 Protocol Buffer Style Guide

This section defines naming standards and conventions for all Protocol Buffer definitions in the STAR project, following Boston Dynamics style guidelines.

4.1 Terminology Standard

The project uses inclusive terminology following OSHWA (Open Source Hardware Association) standards.

Table 19: Communication Bus Terminology

Use	Avoid	Context
Controller / Peripheral	Master / Slave	I2C, SPI, 1 - Wire bus roles
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
Primary / Main	Master	Configuration structures

4.2 Protocol Buffer Naming Conventions(Boston Dynamics Style)

The STAR project follows Boston Dynamics' Protocol Buffer style guide for all .proto definitions.

Table 20: Protocol Buffer Naming Conventions

Element	Convention	Example
Package	Versioned namespace	`star.v1`
Messages	PascalCase	`VelocityCommand`, `BatteryState`
Fields	snake_case + unit suffix	`velocity_mps`, `temperature_celsius`
Enums	SCREAMING_SNAKE_CASE	`MOTOR_STATE_RUNNING`
Enum Zero Value	_UNKNOWN suffix	`STATUS_UNKNOWN = 0`
Enum Values	Type name prefix	`ROBOT_MODE_MANUAL`
RPC Pattern	{Verb} Request/Response	`SetVelocityRequest`

4.3 Unit Suffixes(MKS System)

All numeric fields use SI units with explicit suffixes .This eliminates unit ambiguity and enables automatic documentation generation.

Table 21: Standard Unit Suffixes

Suffix	Unit	Proto Example	C Example
`_m`	meters	`x_m`	`distance_m`
`_mps`	meters / second	`velocity_mps`	`speed_mps`
`_mps2`	m / s^2	`accel_x_mps2`	`acceleration_mps2`
`_rad`	radians	`angle_rad`	`heading_rad`
`_rad_per_s`	rad / second	`omega_rad_per_s`	`angular_vel_rad_per_s`
`_celsius`	degrees C	`temperature_celsius`	`motor_temp_celsius`
`_deci_celsius`	0.1°C	`temp_deci_celsius`	(BMS native format)
`_us`	microseconds	`timestamp_us`	`elapsed_us`
`_ms`	milliseconds	`timeout_ms`	`period_ms`
`_ma`	milliamps	`current_ma`	`motor_current_ma`
`_mv`	millivolts	`cell_mv`	`pack_voltage_mv`
`_mw`	milliwatts	`power_mw`	`consumption_mw`
`_percent`	0 –100%	`soc_percent`	`duty_cycle_percent`

4.4 General Documentation Standards

- Use Doxygen-compatible comments for all public APIs
- Document the “why” not just the “what”
- Keep configuration centralized in header files
- Avoid magic numbers – use named constants with comments
- Version hardware and firmware together using git tags

== == == == == == == == == == == == = =

5 C Style Guide

This section defines comprehensive style guidelines for C firmware development in the STAR project. All firmware code for the RX72N and future platforms must follow these conventions to ensure consistency, maintainability, and safety in embedded systems development.

In this project, **Assembly code** and **Inline Assembly** should be used with extreme caution. While it can offer performance improvements or enable access to low-level hardware features, its use can make code harder to understand, maintain, and port. Always prefer writing in C unless absolutely necessary.

5.1 General Guidelines for ASM and Inline ASM

- **Avoid ASM When Possible** : Always try to write in C before resorting to assembly language.C compilers are highly optimized and can often produce efficient machine code that rivals hand - written assembly.
- **Document ASM Code Thoroughly** : Assembly code is often harder to understand than C, so it should be accompanied by thorough comments that explain **why** ASM was used, as well as what the code does.

```
    /* Set register to zero using ASM */
    mov r0,
    #0 /* Initialize r0 to 0 */
```

- **Use Inline ASM Sparingly** : Inline assembly allows you to embed assembly instructions directly in C code.It should only be used when absolutely necessary, such as when interacting with hardware registers or performing CPU - specific optimizations.
- **Constrain the Scope of Inline ASM** : When using inline assembly, constrain its scope to as few lines as possible and avoid intermixing it with complex C logic .This helps isolate potential issues and reduces the risk of bugs.

```
void set_flag(void) {
  asm volatile("mov r0, #1" : : : "r0"); /* Set flag in register r0 */
}
```

- **Always Use `volatile` for Inline ASM** : Inline assembly should be marked as `volatile` to prevent the compiler from optimizing it out.
- **Do Not Use Inline ASM for Optimizations** : Modern C compilers perform aggressive optimizations. Inline ASM should not be used to try and optimize code performance unless absolutely necessary, as it is often more error-prone than compiler optimizations.
- **Use Constraints Correctly**: When using inline ASM in C, use the appropriate constraints to inform the compiler how the assembly instructions interact with C variables.

```c
int add(int a, int b) {
  int result;
  asm volatile("add %[r], %[a], %[b]"
               : [r] "=r"(result)       /* Output */
               : [a] "r"(a), [b] "r"(b) /* Inputs */
  );
  return result;
}
```

5.2 When to Use ASM and Inline ASM

- **Hardware Access**: Use ASM when direct hardware access is required and C alone cannot provide the necessary control (e.g., setting or clearing specific registers).
- **Performance Critical Code**: ASM may be justified in performance-critical sections where C compiler optimizations fall short, but this should be documented and used as a last resort.
- **CPU-Specific Instructions**: Use ASM for executing instructions that are unique to the target CPU (e.g., specific ARM or x86 instructions that are not exposed in C).

In this project, **assertions** are used as a tool to catch programming errors and invalid states during development. They help ensure that assumptions made in the code are correct, and they provide valuable debugging information when things go wrong. However, assertions should never be relied on to handle runtime errors or expected conditions in production code.

5.3 Guidelines

- **Use Assertions for Debugging** : Assertions are intended to catch conditions that should never occur during normal program execution. Use them to assert assumptions about the state of the program, such as parameter validation or invariants within algorithms.

```c
#include <assert.h>

        void
        process_data(int *data) {
      assert(data != NULL); /* Assert that data is not NULL */
      /* Continue processing */
   }
```

5.4 Platform-Specific Error Checking Macros

Different platforms provide error checking macros for development and debugging:

- **ESP32 (ESP-IDF)**: `ESP_ERROR_CHECK` aborts on error, `ESP_ERROR_CHECK_WITHOUT_ABORT` logs but continues
- **RX72N**: Custom error checking macros following similar patterns
- **General Pattern**: Check return values and handle errors appropriately for your platform

Note : While platform - specific macros are useful during development, production code should use explicit error handling with proper cleanup and recovery mechanisms .

Proper brace placement makes code cleaner and easier to read.This section establishes consistent brace conventions used throughout the STAR firmware codebase.

5.5 General Rules

- **Always Use Braces for Control Statements** : Every `if` , `for` , `while`, or similar control structure must have braces, even for single statements.
- **Same Line for Control Structures and Blocks** : For control structures like `if` , `for` , `while` , and blocks like structs and enums, always place the opening brace `{` on the same line as the statement.
- **Functions Have Braces on a New Line**: The only exception to the same-line rule is for functions, where the opening brace goes on its own line.

5.6 Control Structures

Opening brace on the **same line** as the control statement.

```
        /* CORRECT - if/else with braces on same line */
        if (ret != RX_OK) {
      RX_LOG_ERROR(s_TAG, "Operation failed: %s", rx_err_to_name(ret));
      return ret;
  }

   if (manager == NULL) {
     return RX_ERR_INVALID_ARG;
  } else if (manager->mutex == NULL) {
     return RX_ERR_INVALID_STATE;
  } else {
     /* Proceed with operation */
  }

   /* CORRECT - Single statement still requires braces */
   if (pin < 0 || pin >= GPIO_NUM_MAX) {
     return RX_OK; /* Not an error, pin is not used */
  }

   /* WRONG - Missing braces */
   if (ret != RX_OK)
     return ret;

   /* WRONG - Brace on new line */
   if (ret != RX_OK) {
     return ret;
  }
```

5.6.1 For Loops

```
        /* CORRECT - for loop with brace on same line */
        for (star_bus_config_t *curr = manager->buses; curr;
             curr = curr->next) {
      if (curr->name && strcmp(curr->name, name) == 0) {
        found_bus = curr;
        break;
      }
    }

    for (uint32_t i = 0; i < SPI_HOST_MAX; ++i) {
      manager->spi_host_initialized[i] = false;
      manager->spi_device_count[i] = 0;
    }

    /* WRONG */
    for (uint32_t i = 0; i < count; ++i)
      process_item(i); /* Missing braces */
```

5.6.2 While Loops

```
        /* CORRECT - while loop with brace on same line */
        while (curr) {
      star_bus_config_t *next = curr->next;
      esp_err_t destroy_result = star_bus_config_destroy(curr);
      if (destroy_result != RX_OK && first_error == RX_OK) {
        first_error = destroy_result;
      }
      curr = next;
    }

    while (error_handler_can_retry(&handler)) {
      ret = attempt_operation();
      if (ret == RX_OK) {
        break;
      }
    }

    /* WRONG */
    while (curr) {
      process_node(curr); /* Brace on new line */
      curr = curr->next;
    }
```

5.7 Functions

Opening brace on a **new line** for function definitions.

```
          /* CORRECT - Function brace on new line */
          rx_err_t
          star_bus_manager_init(star_bus_manager_t * manager, const
              char *tag,
                                star_error_interface_t *error_iface,
```

```
                                    star_pin_interface_t *pin_iface) {
        RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                           "Manager pointer is NULL");

        manager->buses = NULL;
        manager->error_iface = error_iface;
        manager->pin_iface = pin_iface;

        /* ... */

        return RX_OK;
    }

    static rx_err_t internal_register_bus_pin(
        star_pin_interface_t * pin_iface, gpio_num_t pin,
        const char *bus_name, const char *usage, bool can_be_shared) {
      if (pin < 0 || pin >= GPIO_NUM_MAX) {
        return RX_OK;
    }
      /* ... */
}

    /* WRONG - Function brace on same line */
    rx_err_t star_bus_manager_init(star_bus_manager_t * manager, ...) {
      /* ... */
}
```

5.8 Structs and Enums

Opening brace on the **same line** as the typedef / struct / enum keyword.

```
        /* CORRECT - Struct brace on same line */
        typedef struct star_bus_manager {
      star_bus_config_t *buses;
      SemaphoreHandle_t mutex;
      const char *tag;
      star_error_interface_t *error_iface;
      star_pin_interface_t *pin_iface;
      bool spi_host_initialized[SPI_HOST_MAX];
      int spi_device_count[SPI_HOST_MAX];
} star_bus_manager_t;

    /* CORRECT - Enum brace on same line */
    typedef enum {
      k_star_bus_type_none,
      k_star_bus_type_i2c,
      k_star_bus_type_spi,
      k_star_bus_type_gpio,
      k_star_bus_type_adc,
      k_star_bus_type_count,
} star_bus_type_t;

    /* CORRECT - Union brace on same line */
    union {
```

```
        star_i2c_bus_config_t i2c;
        star_spi_bus_config_t spi;
        star_gpio_bus_config_t gpio;
        star_adc_bus_config_t adc;
    } proto;

    /* WRONG - Brace on new line */
    typedef struct star_bus_manager {
        /* ... */
    } star_bus_manager_t;
```

5.9 Initializer Lists

Opening brace on the **same line** as the assignment.

```
        /* CORRECT - Initializer brace on same line */
        star_bus_event_t event = {
        .bus_type = k_star_bus_type_i2c,
        .bus_name = config->name,
        .i2c = {
            .port = port, .address = address, .is_write = true, .len =
                len}};

    spi_device_interface_config_t spi_dev_cfg = {
        .clock_speed_hz = 10 * 1000 * 1000,
        .mode = 0,
        .spics_io_num = GPIO_NUM_5,
        .queue_size = 7,
        .pre_cb = NULL,
        .post_cb = NULL,
    };

    /* CORRECT - Short initializers can be on one line */
    i2c_config_t i2c_conf = {.mode = I2C_MODE_MASTER};

    /* WRONG - Brace on new line */
    star_bus_event_t event = {
        .bus_type = k_star_bus_type_i2c,
        /* ... */
    };
```

5.10 Closing Brace Placement

Closing braces are always on their own line, except for `else`, `else if` , and `while` in `do -while` .

```
    /* CORRECT - else on same line as closing brace */
    if (ret != RX_OK) {
  handle_error(ret);
}
else {
  continue_operation();
}
```

```c

/* CORRECT - do-while */
do {
  ret = try_operation();
} while (ret == RX_ERR_TIMEOUT && retries-- > 0);

/* CORRECT - else if chain */
if (config->type == k_star_bus_type_i2c) {
  /* Handle I2C */
} else if (config->type == k_star_bus_type_spi) {
  /* Handle SPI */
} else {
  /* Handle other */
}
```

5.11 Summary Table

Construct	Opening Brace Position
Functions	New line
if/else/else if	Same line
for	Same line
while	Same line
do-while	Same line
switch	Same line
struct	Same line
enum	Same line
union	Same line
Initializer lists	Same line

Switch statements follow specific formatting rules to ensure consistency and readability throughout the STAR firmware codebase.

5.12 General Rules

- **Same Line for Switch and Opening Brace** : The opening brace `{` for a `switch` statement should be on the same line as the `switch` keyword.
- **Braces for Case Blocks** : Each `case` should have its own block of code enclosed in braces `{}`.
- **Break Statements**: Every case must end with `break`, `return`, or a fall-through comment.
- **Default Case**: Always include a `default` case, even if it only logs a warning.

5.13 Basic Switch Structure

```c
      /* CORRECT - from star_bus_manager.c */
      switch (config->type) {
        case k_star_bus_type_i2c: {
  ret =
```

```
      internal_register_bus_pin(pin_iface,
          config->proto.i2c.config.sda_io_num,
                                config->name, "I2C SDA", true);
  if (ret != RX_OK && first_error == RX_OK) {
    first_error = ret;
 }

  ret =
      internal_register_bus_pin(pin_iface,
          config->proto.i2c.config.scl_io_num,
                                config->name, "I2C SCL", true);
  if (ret != RX_OK && first_error == RX_OK) {
    first_error = ret;
 }
  break;
}

case k_star_bus_type_spi: {
  ret = internal_register_bus_pin(pin_iface,
      config->proto.spi.copi_io_num,
                                  config->name, "SPI COPI", true);
  if (ret != RX_OK && first_error == RX_OK) {
    first_error = ret;
 }
  /* ... more pin registrations ... */
  break;
}

default: {
  RX_LOG_DEBUG(s_TAG, "No pin registration for bus type: %d",
      config->type);
  break;
}
}
```

5.14 Case Block Braces

Each case must have its own braces.This prevents variable scope issues and improves readability.

```
        /* CORRECT - Each case has braces */
        switch (bus_type) {
          case k_star_bus_type_i2c: {
      i2c_port_t port = config->proto.i2c.port; /* Local variable */
      uint8_t address = config->proto.i2c.address;
      RX_LOG_INFO(s_TAG, "I2C bus on port %d, address 0x%02X", port,
          address);
      break;
   }
    case k_star_bus_type_spi: {
      spi_host_device_t host = config->proto.spi.host; /* Different scope
          */
      RX_LOG_INFO(s_TAG, "SPI bus on host %d", host);
      break;
   }
```

```
    case k_star_bus_type_gpio: {
      uint8_t pin_count = config->proto.gpio.pin_count;
      RX_LOG_INFO(s_TAG, "GPIO bus with %d pins", pin_count);
      break;
  }
    default: {
      RX_LOG_WARN(s_TAG, "Unknown bus type: %d", bus_type);
      break;
  }
  }

    /* WRONG - Missing braces */
    switch (bus_type) {
    case k_star_bus_type_i2c:
      handle_i2c();
      break;
    case k_star_bus_type_spi:
      handle_spi();
      break;
  }
```

5.15 Break Statement Requirements

Every case must have a clear exit: `break`, `return`, or explicit fall - through.

```
                          /* CORRECT - Using break */
                          switch (config->type) {
    case k_star_bus_type_i2c: {
      process_i2c(config);
      break;
  }
    case k_star_bus_type_spi: {
      process_spi(config);
      break;
  }
    default: {
      break;
  }
  }

    /* CORRECT - Using return */
    const char *star_bus_type_to_string(star_bus_type_t type) {
      switch (type) {
      case k_star_bus_type_none: {
        return "NONE";
    }
      case k_star_bus_type_i2c: {
        return "I2C";
    }
      case k_star_bus_type_spi: {
        return "SPI";
    }
      case k_star_bus_type_gpio: {
        return "GPIO";
```

```
    }
     case k_star_bus_type_adc: {
       return "ADC";
    }
     case k_star_bus_type_count: {
       return "COUNT";
    }
     default: {
       return "UNKNOWN";
    }
    }
  }

   /* CORRECT - Explicit fall-through (rare, must be commented) */
   switch (priority) {
   case PRIORITY_CRITICAL: {
     log_critical(message);
     /* FALLTHROUGH */
  }
   case PRIORITY_HIGH: {
     notify_admin(message);
     /* FALLTHROUGH */
  }
   case PRIORITY_NORMAL: {
     log_message(message);
     break;
  }
   default: {
     break;
  }
  }
```

5.16 Default Case Handling

Always include a default case. Use it for error handling or logging unexpected values.

```
/* CORRECT - Default case logs warning */
switch (curr->type) {
  case k_star_bus_type_i2c: {
  reg_result =
      register_pin_if_valid(curr->proto.i2c.config.sda_io_num, bus_name,
                            "I2C SDA", reg_func, user_ctx, true);
  /* ... */
  break;
}
case k_star_bus_type_spi: {
  reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
     bus_name,
                                     "SPI COPI", reg_func, user_ctx,
                                        true);
  /* ... */
  break;
}
default: {
```

```
  RX_LOG_WARN(manager->tag,
              "Skipping pin registration for "
              "unsupported bus type: %d",
              curr->type);
  break;
}
}

/* CORRECT - Default case returns error for invalid input */
rx_err_t process_bus_type(star_bus_type_t type) {
  switch (type) {
  case k_star_bus_type_i2c: {
    return process_i2c();
 }
  case k_star_bus_type_spi: {
    return process_spi();
 }
  default: {
    RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
    return RX_ERR_INVALID_ARG;
 }
 }
}
```

5.17 Switch on Enum Types

When switching on an enum, consider handling all values explicitly.

```
    /* CORRECT - All enum values handled */
    switch (config->type) {
case k_star_bus_type_none: {
  RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
  break;
}
case k_star_bus_type_i2c: {
  /* Initialize I2C */
  break;
}
case k_star_bus_type_spi: {
  /* Initialize SPI */
  break;
}
case k_star_bus_type_gpio: {
  /* Initialize GPIO */
  break;
}
case k_star_bus_type_adc: {
  /* Initialize ADC */
  break;
}
case k_star_bus_type_count: {
  /* Sentinel value - should never be used */
  RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
  return RX_ERR_INVALID_ARG;
```

```
}
  /* No default needed when all enum values are handled */
  /* Compiler warns if new enum values are added */
}
```

5.18 Indentation

Case labels are at the same indentation level as the switch.Case body is indented one level.

```
    /* CORRECT indentation */
    switch (state) {
case STATE_IDLE: {
  /* Case body indented */
  if (condition) {
    /* Nested code further indented */
 }
  break;
}
case STATE_RUNNING: {
  process_running();
  break;
}
default: {
  break;
}
}

/* WRONG - Case labels indented */
switch (state) {
case STATE_IDLE: {
  break;
}
}
```

5.19 Summary

- Opening brace on same line as `switch`
- Each `case` has its own braces `{ ...}`
- Every case ends with `break`, `return`, or fall - through comment
- Always include `default` case
- Case labels at switch indentation level
- Case body indented one level inside braces

The maximum line length is **80 characters** .Lines longer than this should be split.

- **Break at Natural Points**: Break lines at natural points, such as after operators.
- **Indentation for Continuation Lines** : When breaking lines, ensure the continued line is indented properly.

All functions must include Doxygen comments. This section establishes comprehensive commenting standards for the STAR firmware codebase.

5.20 General Commenting Guidelines

- **Use Block Comments (`/* */`)**: Always prefer block comments over line comments (`//`).
- **Explain Why, Not What** : Comments should explain the reasoning behind code, not what the code does(which should be clear from the code itself) .
- **Keep Comments Updated** : Outdated comments are worse than no comments .
- **English Only** : All comments must be in English.

```
      /* CORRECT - Block comment style */
      /* Initialize the mutex for thread-safe access */
      manager->mutex = xSemaphoreCreateMutex();

  /* WRONG - Line comment style */
  // Initialize the mutex for thread-safe access
  manager->mutex = xSemaphoreCreateMutex();
```

5.21 File Header Comments

Every source and header file should begin with a file path comment, followed by include guards (for headers) and a `@file` Doxygen block for headers with significant content.

```
      /* lib/star_bus/include/star_bus_manager.h */

#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

#include "esp_err.h"
#include "star_bus_manager_types.h"

  /**
   * @file star_bus_manager.h
   * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
      protocols
   *
   * The bus manager provides a central registry for managing multiple bus
   * configurations. It supports dependency injection of error handlers
      and
   * pin validators for loose coupling.
   *
   * Key Features:
   * - Thread-safe bus management with mutex protection
   * - Support for multiple bus types (I2C, SPI, GPIO, DHT22, etc.)
   * - Automatic pin registration and conflict detection
```

```
25     * - Safe bus access via callback pattern (prevents use-after-free)
26     *
27     * Example Usage:
28     * @code
29     * // === Basic Bus Manager Setup ===
30     *
31     * #include "star_bus_manager.h"
32     * #include "star_bus_config.h"
33     * #include "star_error_handler.h"
34     *
35     * // 1. Create error handler and pin validator interfaces
36     * error_handler_t error_handler;
37     * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
38     *
39     * star_error_interface_t error_iface;
40     * star_pin_interface_t pin_iface;
41     * error_handler_get_interface(&error_iface, &error_handler);
42     * pin_validator_get_interface(&pin_iface);
43     *
44     * // 2. Initialize bus manager with dependency injection
45     * star_bus_manager_t bus_manager;
46     * star_bus_manager_init(&bus_manager, "main", &error_iface,
       &pin_iface);
47     * @endcode
48     */
```

5.22 Source File Header

Source files begin with a file path comment.

```
1          /* lib/star_bus/src/star_bus_manager.c */
2
3  #include "star_bus_manager.h"
4
5  #include "freertos/FreeRTOS.h"
6  #include "freertos/semphr.h"
7        /* ... */
```

5.23 Doxygen Documentation

5.23.1 Function Documentation

All public functions must have Doxygen documentation in the header file.

```
1        /**
2         * @brief Initialize a bus manager instance.
3         *
4         * Allocates resources (like a mutex) for the manager. Must be
            called
5         * before any other manager functions.
6         *
7         * @param[out] manager     Pointer to bus manager structure to
            initialize.
```

```
8         *                              Must not be NULL.
9         * @param[in]  tag               Optional tag string for logging
              associated with
10        *                              this manager instance. If NULL or empty,
              a
11        *                              default tag is used.
12        * @param[in]  error_iface       Optional error handler interface (can be
              NULL).
13        * @param[in]  pin_iface         Optional pin validator interface (can be
              NULL).
14        * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
15        *                    is NULL, ESP_ERR_NO_MEM if mutex creation
              fails.
16        */
17       rx_err_t
18       star_bus_manager_init(star_bus_manager_t * manager, const char *tag,
19                             star_error_interface_t *error_iface,
20                             star_pin_interface_t *pin_iface);
```

5.23.2 Parameter Direction Markers

Use `[in]`, `[out]`, or `[in,out]` to indicate parameter direction.

```
1  /**
2  * @brief Write data to an I2C device register.
3  *
4  * @param[in]  config        Pointer to I2C bus configuration.
5  * @param[in]  data          Pointer to data buffer to write.
6  * @param[in]  len           Number of bytes to write.
7  * @param[in]  reg_addr      Target register address.
8  * @param[out] bytes_written Pointer to store actual bytes written.
9  *                           Can be NULL if not needed.
10 * @return rx_err_t RX_OK on success.
11 */
12 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
       config,
13 const uint8_t*           data,
14 size_t                   len,
15 uint8_t                  reg_addr,
16 size_t*                 bytes_written);
17
18   /**
19    * @brief Record an error and manage retry/reset logic
20    *
21    * @param[in,out] handler     Pointer to error handler structure
22    * @param[in]     error       Error code that occurred
23    * @param[in]     description Human-readable description of the error
24    * @param[in]     file        Source file where error occurred
25    * @param[in]     line        Line number where error occurred
26    * @param[in]     func        Function name where error occurred
27    * @return RX_OK if error was handled, or the original error code.
28    */
29   rx_err_t error_handler_record_error(error_handler_t * handler, rx_err_t
         error,
```

```
                                        const char *description, const char
                                            *file,
                                        int32_t line, const char *func);
```

5.23.3 Code Examples with @code / @endcode

Include usage examples in documentation blocks.

```
    /**
     * @file star_error_interface.h
     * @brief Error handler interface for dependency inversion
     *
     * Example Usage:
     * @code
     * // === Creating and Using Error Interface ===
     *
     * #include "star_error_interface.h"
     * #include "star_error_handler.h"
     *
     * // 1. Create concrete error handler
     * error_handler_t error_handler;
     * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
     *
     * // 2. Get interface from implementation
     * star_error_interface_t error_iface;
     * error_handler_get_interface(&error_iface, &error_handler);
     *
     * // 3. Inject into components that need error handling
     * star_bus_manager_init(&bus_manager, "main", &error_iface,
        &pin_iface);
     *
     * // 4. Use the interface
     * if (error_iface.can_retry(error_iface.ctx)) {
     *     // Retry operation...
     *}
     * @endcode
     */
```

5.23.4 Struct Member Documentation

Document struct members inline with Doxygen.

```
    /**
     * @brief Bus configuration structure (common part).
     */
    typedef struct star_bus_config {
    const char *name;      /**< Unique name for this bus/device instance */
    star_bus_type_t type; /**< Type of bus (I2C, SPI, etc.) */
    bool initialized;      /**< Whether this bus has been initialized */
    void *handle;          /**< Driver/device handle */
    void *user_ctx;        /**< User context pointer for callbacks */

    union {
```

```
        star_i2c_bus_config_t i2c;   /**< I2C-specific configuration */
        star_spi_bus_config_t spi;   /**< SPI-specific configuration */
        star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
        star_adc_bus_config_t adc;   /**< ADC-specific configuration */
    } proto;

    struct star_bus_config *next; /**< Next bus config in linked list */
} star_bus_config_t;
```

5.24 Special Comment Keywords

Use these keywords to mark special comments for tracking.

```
/* TODO: Implement retry logic for I2C bus reset */

/* FIXME: Memory leak when bus removal fails mid-operation */

/* BUG: Race condition possible if mutex timeout is too short */

/* XXX: This code assumes little-endian byte order */

/* NOTE: ESP-IDF uses mosi_io_num internally - we map to COPI here */

/* WARNING: Must hold mutex before calling this function */
```

5.24.1 Keyword Definitions

- **TODO** : Work that needs to be done but is not urgent.
- **FIXME** : Known bugs or issues that need to be fixed.
- **BUG** : Confirmed bugs in the code.
- **XXX** : Dangerous or hacky code that works but should be reviewed.
- **NOTE** : Important information about the code.
- **WARNING** : Critical information that must be understood before modifying.

5.25 Section Comments

Use section comments to organize code within files.

```
    /* === Includes === */
#include "freertos/FreeRTOS.h"
#include "star_bus_manager.h"

    /* === Constants === */
    static const char *s_TAG = "STAR_BUS_MANAGER";

/* === Private Function Prototypes === */
static esp_err_t internal_register_bus_pin(...);

```

```
/* === Public Functions === */
esp_err_t star_bus_manager_init(...) { /* ... */}

/* === Private Functions === */
static rx_err_t internal_register_bus_pin(...) { /* ... */}

/* --- Pin Registration Helpers --- */
/* Subsection within a section */
```

5.26 Aligned Comments

Align comments after declarations for improved readability.

```
typedef struct error_handler {
  uint32_t error_count;          /**< Total errors recorded */
  uint32_t max_retries;          /**< Max retry attempts */
  uint32_t current_retry;        /**< Current retry counter */
  uint32_t base_retry_delay;     /**< Base delay (ms) */
  uint32_t max_retry_delay;      /**< Max delay (ms) */
  uint32_t current_retry_delay; /**< Current delay with backoff */
  rx_err_t last_error;           /**< Last recorded error code */
  bool in_error_state;           /**< Whether in error state */
  void *reset_context;           /**< Context for reset function */
  SemaphoreHandle_t mutex;       /**< Mutex for thread-safety */

  rx_err_t (*reset_fn)(void *context); /**< Optional reset function */
} error_handler_t;
```

5.27 Implementation Comments

Comments within function bodies explain non - obvious logic.

```
rx_err_t star_bus_manager_add_bus(
        star_bus_manager_t * manager, star_bus_config_t * config) {
  /* Validate inputs using ESP-IDF macro */
  RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                     "Manager or config pointer is NULL");

  rx_err_t ret = RX_OK;

  /* Acquire mutex with timeout to prevent deadlock */
  if (xSemaphoreTake(manager->mutex,
                     pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
                        !=
      pdTRUE) {
    RX_LOG_ERROR(manager->tag, "Timeout acquiring mutex");
    return RX_ERR_TIMEOUT;
 }

  /* Check for duplicate name - bus names must be unique */
  for (star_bus_config_t *curr = manager->buses; curr; curr = curr->next)
     {
    if (curr->name && strcmp(curr->name, config->name) == 0) {
```

```
      ret = RX_ERR_INVALID_STATE;
      goto add_give_mutex; /* Single exit point pattern */
    }
  }

  /* Initialize hardware before adding to list
   * This ensures manager's SPI host tracking is updated correctly */
  if (!config->initialized) {
    ret = star_bus_config_init(config, manager);
    if (ret != RX_OK) {
      goto add_give_mutex;
    }
  }

  /* Add to head of list (O(1) insertion) */
  config->next = manager->buses;
  manager->buses = config;

add_give_mutex:
  xSemaphoreGive(manager->mutex);
  return ret;
}
```

5.28 Private Function Documentation

Static functions should have brief documentation in the source file.

```
    /**
     * @brief Helper to register a single pin via the interface
     */
    static rx_err_t
    internal_register_bus_pin(star_pin_interface_t * pin_iface,
       gpio_num_t pin,
                              const char *bus_name, const char *usage,
                              bool can_be_shared) {
  if (pin < 0 || pin >= GPIO_NUM_MAX) {
    return RX_OK; /* Not an error, pin is not used */
  }

  if (!pin_iface || !pin_iface->register_pin) {
    return RX_OK; /* No interface, skip registration */
  }

  /* Create description: "BusName: Usage" (e.g., "I2C0: SDA") */
  char desc[64];
  snprintf(desc, sizeof(desc), "%s: %s", bus_name, usage);

  return pin_iface->register_pin(pin_iface->ctx, pin, desc,
     can_be_shared);
}
```

5.29 Summary

- Use block comments (`/* */`), never line comments (`//`)
- Document all public functions with Doxygen in headers
- Use `[in]`, `[out]`, `[ in, out ]` for parameter direction
- Include `@code`/`@endcode` examples in file and complex function docs
- Use TODO, FIXME, BUG, XXX, NOTE, WARNING keywords for tracking
- Align comments after struct members for readability
- Use section comments to organize source files

This section outlines the best practices for ensuring thread-safe and efficient concurrent programming.

5.30 RTOS Types, Mutexes, Semaphores, and Atomic Operations

- **RTOS Types**: Use RTOS types when developing for embedded systems like ESP32.
- **Mutexes**: Use a mutex when you need **exclusive access** to a shared resource.
- **Semaphores**: Use semaphores when you need to manage **shared resources** that multiple threads can access simultaneously.
- **Atomic Operations**: Use atomic operations when working with **simple data types** such as counters or flags.

This section establishes the standards for defining and using enumerations in the STAR firmware codebase.

5.31 General Rules

- **Enum Names Should End in `_t`** : All enum types should end with `_t` .
- **Use Snake Case for Enum Values** : All enum values should follow `snake_case` naming conventions.
- **Enum Values Should Start with `k_`**: To distinguish enum members from other constants, values should start with `k_`.
- **Use typedef**: Always use `typedef` to define the enum type.
- **Include Count Sentinel**: Include a `_count` sentinel value for iteration.

5.32 Basic Enum Structure

```
        /* CORRECT - from star_bus_common_types.h */
        typedef enum {
          k_star_bus_type_none,  /**< No bus type specified */
          k_star_bus_type_i2c,   /**< I2C bus */
          k_star_bus_type_spi,   /**< SPI bus */
          k_star_bus_type_gpio,  /**< GPIO bus */
          k_star_bus_type_adc,   /**< ADC bus */
          k_star_bus_type_count, /**< Number of bus types (sentinel) */
       } star_bus_type_t;

    /* WRONG - Various style violations */
    typedef enum {
      STAR_BUS_TYPE_I2C,  /* Missing k_ prefix */
      StarBusTypeSpi,     /* PascalCase not allowed */
      star_bus_type_gpio, /* Missing k_ prefix */
   } bad_enum_style;     /* Missing _t suffix */
```

5.33 Explicit Value Assignment

Assign explicit values when needed for stability or compatibility.

```
/* CORRECT - Explicit values for protocol compatibility */
typedef enum {
  k_error_none  = 0,  /**< No error */
  k_error_minor = 1,  /**< Minor error, operation can continue */
  k_error_major = 2,  /**< Major error, operation should stop */
  k_error_fatal = 3,  /**< Fatal error, system needs reset */
  k_error_count = 4,  /**< Number of error levels */
} error_level_t;

/* CORRECT - Bit flags with explicit values */
typedef enum {
  k_bus_flag_none = 0x00,        /**< No flags */
  k_bus_flag_initialized = 0x01, /**< Bus is initialized */
  k_bus_flag_busy = 0x02,        /**< Bus is busy */
  k_bus_flag_error = 0x04,       /**< Bus has error */
  k_bus_flag_dma = 0x08,         /**< DMA enabled */
} bus_flags_t;
```

5.34 Documentation

Document each enum value with Doxygen comments.

```
    /**
     * @brief Types of supported bus interfaces.
     */
    typedef enum {
      k_star_bus_type_none,  /**< No bus type specified */
      k_star_bus_type_i2c,   /**< I2C bus */
      k_star_bus_type_spi,   /**< SPI bus */
      k_star_bus_type_gpio,  /**< GPIO bus */
```

```
      k_star_bus_type_adc,   /**< ADC bus */
      k_star_bus_type_count, /**< Number of bus types */
  } star_bus_type_t;
```

5.35 Enum to String Functions

Provide a string conversion function for each enum type.

```
/* In header file */
const char* star_bus_type_to_string(star_bus_type_t type);

/* In source file */
const char *star_bus_type_to_string(star_bus_type_t type) {
  switch (type) {
  case k_star_bus_type_none: {
    return "NONE";
 }
  case k_star_bus_type_i2c: {
    return "I2C";
 }
  case k_star_bus_type_spi: {
    return "SPI";
 }
  case k_star_bus_type_gpio: {
    return "GPIO";
 }
  case k_star_bus_type_adc: {
    return "ADC";
 }
  case k_star_bus_type_count: {
    return "COUNT";
 }
  default: {
    return "UNKNOWN";
 }
 }
}

/* Usage */
RX_LOG_INFO(s_TAG, "Added bus config: %s (Type: %s)", config->name,
            star_bus_type_to_string(config->type));
```

5.36 Using the Count Sentinel

The `_count` value enables iteration and array sizing.

```
    /* Array indexed by enum */
    static const char *s_bus_type_names[k_star_bus_type_count] = {
    [k_star_bus_type_none] = "NONE", [k_star_bus_type_i2c] = "I2C",
    [k_star_bus_type_spi] = "SPI",   [k_star_bus_type_gpio] = "GPIO",
    [k_star_bus_type_adc] = "ADC",
};

```

```c
/* Iteration using count */
for (star_bus_type_t type = k_star_bus_type_none; type <
    k_star_bus_type_count;
      ++type) {
  RX_LOG_INFO(s_TAG, "Bus type %d: %s", type, s_bus_type_names[type]);
}

/* Validation using count */
if (type >= k_star_bus_type_count) {
  RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
  return RX_ERR_INVALID_ARG;
}
```

5.37 Switch Statement Pattern

Always handle all enum values or include a default case.

```c
        /* All values handled explicitly */
        switch (config->type) {
          case k_star_bus_type_none: {
      RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
      break;
    }
     case k_star_bus_type_i2c: {
       ret = init_i2c_bus(config);
       break;
    }
     case k_star_bus_type_spi: {
       ret = init_spi_bus(config);
       break;
    }
     case k_star_bus_type_gpio: {
       ret = init_gpio_bus(config);
       break;
    }
     case k_star_bus_type_adc: {
       ret = init_adc_bus(config);
       break;
    }
     case k_star_bus_type_count: {
       /* Sentinel value should never be used */
       RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
       return RX_ERR_INVALID_ARG;
    }
       /* No default - compiler warns if new enum values added */
}
```

5.38 Forward Declaration

Enums can be forward-declared for header dependency reduction.

```c
/* Define enum with explicit underlying type (C23) before typedef */
enum star_bus_type : int32_t {
```

```
  k_star_bus_type_none,
  k_star_bus_type_i2c,
  /* ... */
};
typedef enum star_bus_type star_bus_type_t;

/* Or forward-declare in header, define in source (C23 with underlying
    type) */
/* In header: */
enum star_bus_type : int32_t;  /* Forward declaration with explicit type
    */
typedef enum star_bus_type star_bus_type_t;

/* In source: */
enum star_bus_type : int32_t {
  k_star_bus_type_none,
  k_star_bus_type_i2c,
  /* ... */
};
```

5.39 Naming Convention Summary

Element	Convention	Example
Type name	`snake_case_t`	`star_bus_type_t`
Member prefix	`k_`	`k_star_bus_type_i2c`
Member naming	`snake_case`	`k_star_bus_type_i2c`
Count sentinel	`_count`	`k_star_bus_type_count`
String function	`_to_string`	`star_bus_type_to_string`

This section establishes error handling patterns for the STAR firmware codebase (RX72N and future targets).

5.40 General Principles

- **Return Error Codes**: Functions should return error codes via `esp_err_t`.
- **Check All Return Values**: Never ignore return values from functions that can fail.
- **Use `goto` for Clean - up**: Use `goto` to jump to a clean-up section when multiple resources need release.
- **Fail Fast**: Return early on invalid arguments or precondition failures.

5.41 Platform Error Codes

Use platform-specific error codes consistently. STAR defines generic error types that map to platform implementations.

```
/* Generic STAR error codes (map to platform-specific implementations) */
RX_OK                   /* Success */
RX_ERR_FAIL             /* Generic failure */
RX_ERR_NO_MEM           /* Out of memory */
```

```
RX_ERR_INVALID_ARG       /* Invalid argument */
RX_ERR_INVALID_STATE     /* Invalid state for operation */
RX_ERR_INVALID_SIZE      /* Invalid size */
RX_ERR_NOT_FOUND         /* Requested resource not found */
RX_ERR_NOT_SUPPORTED     /* Operation not supported */
RX_ERR_TIMEOUT           /* Operation timed out */

/* Usage example */
rx_err_t star_bus_manager_init(star_bus_manager_t* manager, ...)
{
      if (manager == NULL) {
        return RX_ERR_INVALID_ARG;
     }

      manager->mutex = xSemaphoreCreateMutex();
      if (manager->mutex == NULL) {
        return RX_ERR_NO_MEM;
     }

      return RX_OK;
}
```

5.42 Validation Macros

Use platform-provided validation macros for parameter checking and early returns. These provide consistent error handling patterns across the codebase.

5.42.1 RX_RETURN_ON_FALSE

Returns an error if condition is false, with logging. ESP32 platforms use `ESP_RETURN_ON_FALSE`, RX72N uses `RX_RETURN_ON_FALSE`.

```
    /* Platform-specific: RX72N uses rx_check.h */
#include "rx_check.h"

rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      /* Validate required parameter with automatic logging */
      RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                         "Manager pointer is NULL");

      /* Multiple validations */
      RX_RETURN_ON_FALSE(manager->mutex, RX_ERR_INVALID_STATE,
         manager->tag,
                         "Manager mutex not initialized");

      RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
                             config->type < k_star_bus_type_count,
                         RX_ERR_INVALID_ARG, manager->tag,
                         "Invalid bus type in config");
```

```
21
22      return RX_OK;
23  }
```

5.42.2 ESP_GOTO_ON_ERROR

Jumps to a label on error, with logging. Use for cleanup patterns.

```
1   static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
        config,
2   const uint8_t*           data,
3   size_t                   len,
4   uint8_t                  reg_addr,
5   size_t*                  bytes_written)
6   {
7       rx_err_t ret = RX_OK;
8       i2c_cmd_handle_t cmd = NULL;
9
10      cmd = i2c_cmd_link_create();
11      RX_RETURN_ON_FALSE(cmd, RX_ERR_NO_MEM, s_TAG,
12                         "Failed to create I2C command link");
13
14      ret = i2c_master_start(cmd);
15      ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': START
            failed: %s",
16                        config->name, rx_err_to_name(ret));
17
18      ret = i2c_master_write_byte(cmd, (address << 1) | I2C_MASTER_WRITE,
            true);
19      ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
20                        "Write '%s': Address (W) failed: %s",
                              config->name,
21                        rx_err_to_name(ret));
22
23      ret = i2c_master_write(cmd, data, len, true);
24      ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
25                        "Write '%s': Data write failed: %s", config->name,
26                        rx_err_to_name(ret));
27
28      ret = i2c_master_stop(cmd);
29      ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': STOP failed:
            %s",
30                        config->name, rx_err_to_name(ret));
31
32      /* Execute command */
33      ret = i2c_master_cmd_begin(port, cmd, pdMS_TO_TICKS(1000));
34      ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': CMD failed",
35                        config->name);
36
37      /* Success path */
38      i2c_cmd_link_delete(cmd);
39      if (bytes_written) {
40        *bytes_written = len;
41     }
```

```
    return RX_OK;

write_fail:
    if (cmd) {
        i2c_cmd_link_delete(cmd);
    }
    return ret;
}
```

5.42.3 ESP_ERROR_CHECK

Use during development to catch fatal errors.

```
/* Development only - aborts on error */
/* Platform-specific error checking example (ESP32 shown) */
ESP_ERROR_CHECK(i2c_driver_install(port, I2C_MODE_MASTER, 0, 0, 0));

/* Generic pattern: check return value and handle errors */
rx_err_t ret = optional_init();
if (ret != RX_OK) {
    RX_LOG_WARN(TAG, "Optional init failed: %d", ret);
}
```

5.43 NULL Safety Patterns

Always validate pointer parameters.

```
/* At function start - validate all pointers */
rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
const char*                 name,
const uint8_t*              data,
size_t                      len,
uint8_t                     reg_addr,
size_t*                     bytes_written)
{
    RX_RETURN_ON_FALSE(manager && name && data, RX_ERR_INVALID_ARG,
        s_TAG,
                       "Manager, name, or data is NULL");
    RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
                       "Write length must be > 0");

    /* Clear output parameter initially */
    if (bytes_written) {
        *bytes_written = 0;
    }

    /* ... rest of implementation ... */
}

/* Check optional interface before use */
if (manager->error_iface && manager->error_iface->record_error) {
    manager->error_iface->record_error(manager->error_iface->ctx, ret,
```

```
                                          "Bus hardware initialization
                                             failed",
                                          __FILE__, __LINE__, __func__);
}

    /* Use convenience macro for safe interface calls */
#define STAR_IFACE_RECORD_ERROR(iface, err, msg)
    \\
    ((iface) && (iface)->record_error
                                                \\
        ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
                \\
                                __LINE__, __func__)
                                                                \\
        : (err))

/* Usage */
STAR_IFACE_RECORD_ERROR(manager->error_iface, ret, "Operation failed");
```

5.44 Mutex Error Handling

Handle mutex acquisition failures properly.

```
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t*  config)
{
      /* Validate arguments first */
      RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                         "Invalid arguments");
      RX_RETURN_ON_FALSE(manager->mutex, ESP_ERR_INVALID_STATE,
          manager->tag,
                         "Mutex not initialized");

      rx_err_t ret = RX_OK;

      /* Acquire mutex with timeout */
      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        RX_LOG_ERROR(manager->tag,
                     "Timeout acquiring mutex for adding bus '%s'",
                     config->name);
        return RX_ERR_TIMEOUT;
     }

    /* Critical section */
    /* ... operations ... */

    /* Single exit point ensures mutex release */
    add_give_mutex:
      xSemaphoreGive(manager->mutex);
      return ret;
}
```

5.45 Error Interface Dependency Injection

Components receive error handlers through dependency injection.

```
/* Interface definition */
typedef struct star_error_interface {
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
         *message,
                               const char *file, int line, const char
                                  *func);
      bool (*can_retry)(void *ctx);
      rx_err_t (*reset_state)(void *ctx);
      void *ctx;
} star_error_interface_t;

/* Injecting error handler into component */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*              tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      manager->error_iface = error_iface; /* Can be NULL */
      /* ... */
}

/* Using the error interface */
if (ret != RX_OK) {
      if (manager->error_iface && manager->error_iface->record_error) {
        manager->error_iface->record_error(manager->error_iface->ctx, ret,
                                           "Bus initialization failed",
                                           __FILE__, __LINE__, __func__);
     }
}
```

5.46 Cleanup Patterns

Use goto for consistent cleanup when multiple resources are acquired.

```
rx_err_t complex_init(complex_t* obj)
{
      rx_err_t ret = RX_OK;

      /* Allocate resources */
      obj->buffer = malloc(BUFFER_SIZE);
      if (!obj->buffer) {
        ret = RX_ERR_NO_MEM;
        goto cleanup;
     }

      obj->mutex = xSemaphoreCreateMutex();
      if (!obj->mutex) {
```

```
        ret = RX_ERR_NO_MEM;
        goto cleanup;
    }

    ret = hardware_init(obj);
    if (ret != RX_OK) {
        goto cleanup;
    }

    return RX_OK;

cleanup:
    /* Release resources in reverse order */
    if (obj->mutex) {
        vSemaphoreDelete(obj->mutex);
        obj->mutex = NULL;
    }
    if (obj->buffer) {
        free(obj->buffer);
        obj->buffer = NULL;
    }
    return ret;
}
```

5.47 Error Logging

Use appropriate log levels for different error severities.

```
/* RX_LOG_ERROR - Errors that prevent operation */
RX_LOG_ERROR(s_TAG, "Failed to initialize bus '%s': %s",
config->name, rx_err_to_name(ret));

/* RX_LOG_WARN - Warnings that don't prevent operation */
RX_LOG_WARN(manager->tag, "Failed to register pins for bus '%s': %s "
"(bus still added)", config->name, rx_err_to_name(ret));

/* RX_LOG_INFO - Informational messages */
RX_LOG_INFO(manager->tag, "Added bus config: %s (Type: %s)",
config->name, star_bus_type_to_string(config->type));

/* RX_LOG_DEBUG - Debug messages (compiled out in release) */
RX_LOG_DEBUG(s_TAG, "Write Success: %zu bytes to '%s' (Addr 0x%02X)",
len, config->name, address);
```

5.48 Summary

- Return `rx_err_t` from all functions that can fail
- Use `ESP_RETURN_ON_FALSE` for parameter validation
- Use `ESP_GOTO_ON_ERROR` for cleanup patterns
- Always validate pointers before dereferencing

- Handle mutex timeouts as `ESP_ERR_TIMEOUT`
- Use dependency injection for error handlers
- Release resources in reverse order of acquisition
- Log errors with appropriate severity levels

This section establishes standards for function design, implementation, and documentation in the STAR firmware codebase.

5.49 Return Value Conventions

- If the name of a function is an action (or an imperative command), it should return an **error-code integer** (`esp_err_t`).
- If the name is a predicate (a function that returns true/false), it should return a **succeeded boolean**.

```
/* Action functions return rx_err_t */
rx_err_t star_bus_manager_init(...);
rx_err_t star_bus_manager_add_bus(...);
rx_err_t star_bus_config_destroy(...);

/* Predicate functions return bool */
bool error_handler_can_retry(error_handler_t* handler);
bool is_bus_initialized(const star_bus_config_t* config);

/* Getter functions return the value or pointer */
star_bus_config_t* star_bus_manager_find_bus(...);
const char* star_bus_type_to_string(star_bus_type_t type);
```

5.50 Private and Exposed Private Functions

5.50.1 Private Functions (static)

Functions internal to a single source file should be declared `static` with the `internal_` prefix.

```
/* In .c file - private static functions */
static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage,
bool                  can_be_shared)
{
      if (pin < 0 || pin >= GPIO_NUM_MAX) {
        return RX_OK; /* Not an error, pin is not used */
     }
      /* ... implementation ... */
}

static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
   config,
```

```
i2c_port_t                port,
i2c_cmd_handle_t          cmd,
const char*               operation_name)
{
      /* ... implementation ... */
}

/* Declare prototypes at top of file */
static rx_err_t internal_register_bus_pin(...);
static rx_err_t internal_unregister_bus_pin(...);
static rx_err_t internal_register_config_pins(...);
```

5.50.2 Exposed Private Functions (priv_)

If a function cannot be `static` and must be exposed across files within a module (but not publicly), prefix its name with `priv_`.

```
/* In internal header (e.g., star_bus_internal.h) */
rx_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
void      priv_star_bus_release_mutex(star_bus_manager_t* manager);
```

5.51 Parameter Validation

Validate all parameters at the start of public functions.

```
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t*  config)
{
      /* Validate required parameters */
      ESP_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                          "Manager or config pointer is NULL");

      /* Validate config contents */
      ESP_RETURN_ON_FALSE(config->name && strlen(config->name) > 0,
                          RX_ERR_INVALID_ARG, manager->tag,
                          "Config name is NULL or empty");

      /* Validate enum range */
      RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
                             config->type < k_star_bus_type_count,
                         RX_ERR_INVALID_ARG, manager->tag,
                         "Invalid bus type in config");

      /* Validate state */
      RX_RETURN_ON_FALSE(manager->mutex, RX_ERR_INVALID_STATE,
         manager->tag,
                         "Manager mutex not initialized");

      /* ... rest of implementation ... */
}

rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
```

```
const char*                 name,
const uint8_t*              data,
size_t                      len,
uint8_t                     reg_addr,
size_t*                     bytes_written)
{
      /* Validate all required pointers */
      RX_RETURN_ON_FALSE(manager && name && data, RX_ERR_INVALID_ARG,
          s_TAG,
                         "Manager, name, or data is NULL");

      /* Validate numeric constraints */
      RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
                         "Write length must be > 0");

      /* Clear output parameters initially */
      if (bytes_written) {
        *bytes_written = 0;
     }

      /* ... rest of implementation ... */
}
```

5.52 Function Declaration Formatting

Align parameters vertically when they span multiple lines.

```
/* CORRECT - Aligned parameters */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*              tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);

star_bus_config_t* star_bus_config_create_spi_device(
const char*                          name,
spi_host_device_t                    host,
gpio_num_t                           copi_pin,
gpio_num_t                           cipo_pin,
gpio_num_t                           sclk_pin,
int32_t                              dma_chan,
const spi_device_interface_config_t* dev_cfg);

/* Function pointers in structs */
typedef struct star_error_interface {
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
          *message,
                               const char *file, int line, const char
                                  *func);
      bool (*can_retry)(void *ctx);
      rx_err_t (*reset_state)(void *ctx);
      void *ctx;
} star_error_interface_t;
```

5.53 Doxygen Documentation

All public functions must have complete Doxygen documentation in header files.

```
/**
* @brief Add a new bus/device configuration to the manager.
*
* Adds a previously created configuration (via star_bus_config_create_*)
* to the manager's internal list. The manager takes ownership conceptually,
* but the configuration still needs to be destroyed eventually (typically via
* star_bus_manager_remove_bus or star_bus_manager_deinit).
*
* The configuration must have a unique name. This function is thread-safe.
*
* @param[in] manager Pointer to the initialized bus manager.
*                    Must not be NULL.
* @param[in] config  Pointer to the bus configuration structure to add.
*                    Must not be NULL and must have a valid name.
* @return rx_err_t RX_OK on success,
*                  ESP_ERR_INVALID_ARG if manager or config is invalid,
*                  ESP_ERR_INVALID_STATE if a bus with the same name
*                      already exists or mutex error,
*                  ESP_ERR_TIMEOUT if mutex times out.
*/
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t*  config);
```

5.54 Private Function Documentation

Static functions should have brief documentation in the source file.

```
/**
* @brief Helper: Execute I2C command with retry logic
*
* Wraps i2c_master_cmd_begin() with automatic retry for transient errors.
*
* @param[in] config         Pointer to I2C bus configuration
* @param[in] port           I2C port number
* @param[in] cmd            I2C command handle
* @param[in] operation_name Name of operation for logging
* @return rx_err_t Final result after retries
*/
static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
    config,
i2c_port_t               port,
i2c_cmd_handle_t         cmd,
const char*              operation_name);
```

5.55 Function Implementation Patterns

5.55.1 Single Exit Point with Cleanup

```c
rx_err_t star_bus_manager_remove_bus(star_bus_manager_t* manager,
const char*         name)
{
      ESP_RETURN_ON_FALSE(manager && name && strlen(name) > 0,
                          RX_ERR_INVALID_ARG, s_TAG, "Invalid arguments");

      star_bus_config_t *to_remove = NULL;
      star_bus_config_t *prev = NULL;
      rx_err_t ret = RX_ERR_NOT_FOUND;

      /* Acquire mutex */
      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      /* Find and unlink */
      for (star_bus_config_t *curr = manager->buses; curr;
           prev = curr, curr = curr->next) {
        if (curr->name && strcmp(curr->name, name) == 0) {
          to_remove = curr;
          if (prev == NULL) {
            manager->buses = curr->next;
         } else {
            prev->next = curr->next;
         }
          to_remove->next = NULL;
          ret = RX_OK;
          break;
       }
     }

      if (ret == ESP_ERR_NOT_FOUND) {
        goto remove_give_mutex;
     }

      /* Cleanup operations */
      rx_err_t destroy_result = star_bus_config_destroy(to_remove);
      if (destroy_result != RX_OK) {
        ret = destroy_result;
     }

    remove_give_mutex:
      xSemaphoreGive(manager->mutex);
      return ret;
}
```

5.55.2 Factory Function Pattern

```c
star_bus_config_t* star_bus_config_create_i2c(const char* name,
```

```
i2c_port_t  port,
uint8_t     address,
gpio_num_t  sda_pin,
gpio_num_t  scl_pin,
uint32_t    clk_speed)
{
      if (name == NULL || strlen(name) == 0) {
        RX_LOG_ERROR(s_TAG, "Invalid name for I2C bus config");
        return NULL;
     }

      star_bus_config_t *config = calloc(1, sizeof(star_bus_config_t));
      if (config == NULL) {
        RX_LOG_ERROR(s_TAG, "Failed to allocate memory for I2C config");
        return NULL;
     }

      /* Initialize common fields */
      config->name = strdup(name);
      config->type = k_star_bus_type_i2c;
      config->initialized = false;
      config->handle = NULL;
      config->next = NULL;

      /* Initialize I2C-specific fields */
      config->proto.i2c.port = port;
      config->proto.i2c.address = address;
      config->proto.i2c.config.sda_io_num = sda_pin;
      config->proto.i2c.config.scl_io_num = scl_pin;
      config->proto.i2c.config.master.clk_speed = clk_speed;

      /* Set default operations */
      star_bus_i2c_init_default_ops(&config->proto.i2c.ops);

      return config;
}
```

5.56 Callback Functions

```
/* Callback type definition */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
void*              user_ctx);

/* Function using callback */
rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
const char*         name,
star_bus_callback_t callback,
void*               user_ctx)
{
      ESP_RETURN_ON_FALSE(manager && name && callback, RX_ERR_INVALID_ARG,
                          s_TAG, "Invalid arguments");

      /* ... find bus and invoke callback while holding mutex ... */
```

```
        rx_err_t callback_result = callback(found_bus, user_ctx);

        xSemaphoreGive(manager->mutex);
        return callback_result;
}

/* Using the callback pattern */
rx_err_t my_bus_callback(star_bus_config_t* bus, void* ctx) {
        ESP_LOGI(TAG, "Bus type: %s", star_bus_type_to_string(bus->type));
        return RX_OK;
}

star_bus_manager_with_bus(&manager, "sensor_i2c", my_bus_callback, NULL);
```

5.57 Summary

- Action functions return `esp_err_t`; predicates return `bool`
- Use `static` with `internal_` prefix for file-local functions
- Use `priv_` prefix for exposed-but-not-public functions
- Validate all parameters at function start
- Align parameters vertically in multi-line declarations
- Document all public functions with complete Doxygen
- Use single exit point with cleanup for complex functions
- Use factory functions for object creation

Global variables should be avoided when possible. This section establishes patterns for when they are necessary.

5.58 General Principles

- **Avoid Globals**: Global variables are strongly discouraged. Use dependency injection instead.
- **File-Scoped When Necessary**: If a global is needed, limit its scope to a single file using `static`.
- **Use Prefix Conventions**: `s_` for static file-scoped, `g_` for true globals.
- **Prefer `static const` Over `#define`**: For typed constants, use `static const`.

5.59 Static File-Scoped Variables (s_ prefix)

Use `static` with `s_` prefix for variables that should be accessible only within a single source file.

```
/* CORRECT - Static file-scoped variables */

/* Logging tag - most common pattern */
static const char* s_TAG = "STAR_BUS_MANAGER";
```

```c

/* Configuration constants - prefer static const over #define */
static const uint32_t s_i2c_timeout_ms = 1000;
static const uint32_t s_default_retry_count = 3;

/* State variables scoped to this file */
static bool s_module_initialized = false;

/* From star_bus_i2c.c */
static const char* s_TAG = "STAR_BUS_I2C";
static const uint32_t s_i2c_timeout_ms = 1000;

/* From star_bus_config.c */
static const char* s_TAG = "STAR_BUS_CONFIG";
```

5.60 Why static const Over #define

`static const` provides type safety and scope control that `#define` lacks.

```c
/* PREFERRED - Type-safe, scoped, debugger-visible */
static const uint32_t s_timeout_ms = 1000;
static const size_t   s_buffer_size = 256;

/* Use the constant with type checking */
uint32_t timeout = s_timeout_ms;  /* Compiler checks type */

    /* AVOID - No type safety, global namespace pollution */
#define TIMEOUT_MS 1000
#define BUFFER_SIZE 256

/* Issues with #define: */
/* 1. No type checking */
/* 2. Can collide with other defines */
/* 3. Not visible in debugger */
/* 4. Can have unexpected macro expansion */
```

5.61 When to Use #define

Use `#define` for:

- Compile-time constants for array sizes
- Conditional compilation
- Macros that cannot be functions

```c
    /* Array sizes must be compile-time constants */
#define STAR_BUS_NAME_MAX_LEN (32)

char bus_name[STAR_BUS_NAME_MAX_LEN];  /* OK - compile-time constant */

    /* Conditional compilation */
#ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
```

```
RX_LOG_DEBUG(s_TAG, "Debug info: %d", value);
#endif

    /* Macros with special features */
#define RECORD_ERROR(handler, code, desc)
   \\
      do {

         \\
        error_handler_record_error((handler), (code), (desc), __FILE__,
                 \\
                                    __LINE__, __func__);
                                                          \\
     } while (0)
```

5.62 Explicit Type Usage - Always Use Fixed - Width Integer Types

CRITICAL RULE: Never use implicit types like `int`, `char`, `short`, or `long`. Always use explicit fixed-width types from `<stdint.h>`.

5.62.1 Why Explicit Types Matter

- **Platform Independence**: `int` can be 16-bit or 32-bit depending on platform
- **Clear Intent**: `uint32_t` explicitly shows the variable is unsigned and 32-bit
- **Prevents Bugs**: Implicit integer promotion and sign extension bugs are avoided
- **Embedded Safety**: Embedded systems require precise control over data sizes
- **Code Portability**: Code works identically on ESP32, RX72N, and future platforms

5.62.2 Correct Type Usage

```
#include <stdbool.h>
#include <stdint.h>

/* CORRECT - Always use explicit fixed-width types */

/* Unsigned integers */
uint8_t   byte_value    = 0xFF;      /* 8-bit unsigned (0 to 255) */
uint16_t  word_value    = 0xFFFF;    /* 16-bit unsigned (0 to 65535) */
uint32_t  counter       = 0;          /* 32-bit unsigned */
uint64_t  timestamp_us  = 0;          /* 64-bit unsigned */

/* Signed integers */
int8_t    temperature   = -40;        /* 8-bit signed (-128 to 127) */
int16_t   offset        = -1000;      /* 16-bit signed */
int32_t   position      = 0;          /* 32-bit signed */
int64_t   delta_time_us = 0;          /* 64-bit signed */

/* Boolean (not int!) */
bool      is_enabled    = false;      /* Use stdbool.h */
```

```

/* Size and pointer types */
size_t    buffer_size   = 256;        /* For sizes and counts */
uintptr_t address       = 0x20000000; /* For storing addresses */

/* WRONG - Never use these implicit types */
int       bad_counter;     /* Platform-dependent size! */
unsigned  bad_value;       /* Ambiguous width! */
char      bad_byte;        /* Is it signed or unsigned? */
short     bad_small;       /* 16-bit? Not guaranteed! */
long      bad_large;       /* 32-bit or 64-bit? Depends on platform! */
```

5.62.3 Special Case: Characters and Strings

Only use `char` for actual character strings, never for byte data.

```
/* CORRECT - char only for strings */
const char* message = "Hello, STAR!";
char        buffer[64];
char        letter = 'A';

/* WRONG - Don't use char for byte data */
char data[256];  /* Should be uint8_t data[256] */
char byte = 0xFF;  /* Should be uint8_t byte = 0xFF */

/* CORRECT - uint8_t for byte data */
uint8_t data_buffer[256];
uint8_t byte_value = 0xFF;
```

5.62.4 Loop Counters and Array Indices

Use `uint32_t` or `size_t` for loop counters and array indices.

```
/* CORRECT - Explicit types for loops */
for (uint32_t i = 0; i < array_length; i++) {
      process_element(array[i]);
}

/* For sizes from sizeof() or string length */
for (size_t i = 0; i < buffer_size; i++) {
      buffer[i] = 0;
}

/* WRONG - Never use int for loops */
for (int i = 0; i < count; i++) {  /* What if count > INT_MAX? */
      /* ... */
}
```

5.62.5 Function Parameters and Return Types

Always use explicit types in function signatures.

```c
/* CORRECT - Explicit types in function signatures */
uint32_t calculate_checksum(const uint8_t* data, size_t length);
int32_t  read_temperature_celsius(uint8_t sensor_id);
bool     is_motor_running(uint8_t motor_id);

/* WRONG - Implicit types */
int calculate_value(char* data, int len);  /* Ambiguous! */
```

5.62.6 Type Casting for Enum Comparisons

When comparing enums of different types, cast to the appropriate fixed-width type.

```c
typedef enum {
  k_channel_0 = 0,
  k_channel_1 = 1,
  k_channel_max = 2,
} channel_t;

/* CORRECT - Cast to explicit type for comparison */
if ((int32_t)channel >= k_channel_max) {
      return ERROR_INVALID_CHANNEL;
}

/* WRONG - Direct enum comparison can trigger warnings */
if (channel >= k_channel_max) {  /* -Werror=enum-compare */
      return ERROR_INVALID_CHANNEL;
}
```

5.62.7 Summary: Type Usage Rules

- **ALWAYS**: Use `uint8_t`, `int32_t`, etc. from `<stdint.h>`
- **NEVER**: Use `int`, `unsigned`, `short`, `long`
- **ONLY**: Use `char` for actual character strings
- **USE**: `bool` from `<stdbool.h>` for boolean values
- **USE**: `size_t` for sizes and counts from sizeof()
- **CAST**: Enums to `int32_t` when comparing different enum types

5.63 True Global Variables (g_ prefix)

If a truly global variable is unavoidable, use the `g_` prefix. This should be rare.

```c
/* DISCOURAGED but sometimes necessary */
/* g_ prefix for true globals (visible across files) */
g_system_state_t g_system_state;

/* Prefer dependency injection instead */
/* BETTER approach: */
typedef struct app_context {
```

```
      star_bus_manager_t bus_manager;
      error_handler_t error_handler;
      /* ... */
} app_context_t;

/* Pass context to functions that need it */
void app_process_sensors(app_context_t* ctx);
```

5.64 Dependency Injection Alternative

Replace globals with dependency injection.

```
/* WRONG - Global error handler */
error_handler_t g_error_handler;  /* Global - bad */

void some_function(void) {
      if (error) {
        RECORD_ERROR(&g_error_handler, err, "Failed"); /* Uses global */
     }
}

/* CORRECT - Dependency injection */
typedef struct star_bus_manager {
      star_error_interface_t *error_iface; /* Injected dependency */
      /* ... */
} star_bus_manager_t;

rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,  /* Injected */
star_pin_interface_t*   pin_iface)    /* Injected */
{
      manager->error_iface = error_iface;
      /* ... */
}

/* Usage */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
error_handler_get_interface(&error_iface, &error_handler);

star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, NULL);
```

5.65 Static Variables for Module State

When a module needs to maintain state, prefer explicit initialization and deinitialization.

```
/* Module-private state */
static struct {
      bool initialized;
```

```
      uint32_t transaction_count;
      /* ... */
} s_module_state = {
  .initialized = false,
  .transaction_count = 0,
};

rx_err_t module_init(void)
{
      if (s_module_state.initialized) {
        return RX_ERR_INVALID_STATE;
     }

      /* Initialize module */
      s_module_state.initialized = true;
      return RX_OK;
}

rx_err_t module_deinit(void)
{
      if (!s_module_state.initialized) {
        return RX_ERR_INVALID_STATE;
     }

      /* Clean up */
      s_module_state.initialized = false;
      s_module_state.transaction_count = 0;
      return RX_OK;
}
```

5.66 Thread Safety for Shared State

If static variables are accessed from multiple threads, protect with mutex.

```
/* Thread-safe module state */
static struct {
      bool initialized;
      SemaphoreHandle_t mutex;
      uint32_t counter;
} s_shared_state;

rx_err_t safe_increment_counter(void)
{
      if (xSemaphoreTake(s_shared_state.mutex, pdMS_TO_TICKS(1000)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      s_shared_state.counter++;

      xSemaphoreGive(s_shared_state.mutex);
      return RX_OK;
}
```

5.67 Summary

Type	Prefix	Usage
Static file-scoped	`s_`	Prefer this for module-internal state
True global	`g_`	Avoid; use dependency injection
`static const`	`s_`	Type-safe constants
`#define`	None	Array sizes, macros, conditional compilation

- Use `static const` for type-safe constants
- Use `s_TAG` for module logging tags
- Prefer dependency injection over global variables
- Protect shared state with mutexes
- Limit variable scope to the smallest necessary

This section establishes standards for header file organization and content in the STAR firmware codebase.

5.68 General Principles

- **Header Files Should Only Contain Declarations**: No implementations except for inline functions.
- **Minimal Includes in Headers**: Include only what is necessary for the declarations.
- **Self-Contained Headers**: Each header should compile independently.
- **One Purpose Per Header**: Split large headers into focused sub-headers.

5.69 Include Guard Naming Convention

All header files must use include guards following the pattern: `STAR_COMPONENT_FILENAME_H`

```
    /* CORRECT - from star_bus_manager.h */
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

        /* ... declarations ... */

#endif /* STAR_COMPONENT_BUS_MANAGER_H */

    /* CORRECT - from star_bus_config.h */
#ifndef STAR_COMPONENT_BUS_CONFIG_H
#define STAR_COMPONENT_BUS_CONFIG_H
    /* ... */
#endif /* STAR_COMPONENT_BUS_CONFIG_H */

    /* CORRECT - from star_error_interface.h */
#ifndef STAR_ERROR_INTERFACE_H
#define STAR_ERROR_INTERFACE_H
    /* ... */
```

```
#endif /* STAR_ERROR_INTERFACE_H */

    /* CORRECT - from star_pin_interface.h */
#ifndef STAR_PIN_INTERFACE_H
#define STAR_PIN_INTERFACE_H
    /* ... */
#endif /* STAR_PIN_INTERFACE_H */
```

5.70 C++ Compatibility

All headers must include C++ extern "C" blocks for compatibility with C++ compilers.

```
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

        /* All declarations go here */

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.71 Header File Structure

Headers should follow this organization:

```
        /* lib/star_bus/include/star_bus_manager.h */

#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

    /* === 1. Includes === */
#include "esp_err.h"
#include "star_bus_manager_types.h"

    /* === 2. File Documentation === */
    /**
     * @file star_bus_manager.h
     * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
        protocols
     *
     * The bus manager provides a central registry for managing multiple
     * bus configurations. It supports dependency injection of error
        handlers
```

```
     * and pin validators for loose coupling.
     *
     * Example Usage:
     * @code
     * // Initialize bus manager with dependency injection
     * star_bus_manager_t bus_manager;
     * star_bus_manager_init(&bus_manager, "main", &error_iface,
        &pin_iface);
     * @endcode
     */

    /* === 3. Forward Declarations (if needed) === */
    /* Forward declarations reduce header dependencies */

    /* === 4. Macros and Constants === */
#define STAR_BUS_NAME_MAX_LEN (32)
     /* === 5. Type Definitions === */
     /* (Usually in separate _types.h header) */

     /* === 6. Function Declarations === */

     /**
      * @brief Initialize a bus manager instance.
      *
      * @param[out] manager     Pointer to bus manager to initialize.
      * @param[in]  tag         Optional tag string for logging.
      * @param[in]  error_iface Optional error handler interface.
      * @param[in]  pin_iface   Optional pin validator interface.
      * @return rx_err_t RX_OK on success.
      */
     rx_err_t star_bus_manager_init(
         star_bus_manager_t * manager, const char *tag,
         star_error_interface_t *error_iface, star_pin_interface_t
            *pin_iface);

         /* ... more function declarations ... */

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.72 Include Order

Includes should be ordered in groups, separated by blank lines:

```
    /* 1. Primary header (in .c file, include corresponding .h first) */
#include "star_bus_manager.h"

    /* 2. FreeRTOS includes */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"
#include "freertos/task.h"
```

```

    /* 3. Standard library includes */
#include <inttypes.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

    /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */
#include "driver/gpio.h"
#include "driver/i2c.h"
    /* Platform-specific: RX72N uses rx_check.h */
#include "esp_err.h"
#include "esp_log.h"
#include "rx_check.h"
#include "sdkconfig.h"

    /* 5. Project includes */
#include "star_bus_config.h"
#include "star_bus_i2c.h"
#include "star_bus_types.h"
#include "star_error_interface.h"
```

5.73 Forward Declarations

Use forward declarations to minimize header dependencies.

```
/* In star_bus_config.h - forward declare instead of including */
/* Forward declaration */
struct star_bus_manager;  /* Don't need full definition */

/* Function using forward-declared type */
rx_err_t star_bus_config_init(star_bus_config_t*      config,
struct star_bus_manager* manager);

/* In star_bus_common_types.h */
/* Forward declarations for pointer-only usage */
typedef struct star_bus_config  star_bus_config_t;
typedef struct star_bus_manager star_bus_manager_t;
```

5.74 Separating Types from Functions

For complex modules, split type definitions into dedicated headers.

```
/* File organization for star_bus module */
star_bus/include/
star_bus_manager.h        /* Main API (functions) */
star_bus_manager_types.h  /* Types for manager */
star_bus_common_types.h   /* Shared types */
star_bus_protocol_types.h /* Protocol-specific types */
star_bus_function_types.h /* Function pointer types */
```

```
star_bus_event_types.h    /* Event structures */
star_bus_types.h          /* Controller - includes all */

    /* Controller header pattern */
    /* star_bus_types.h */
#ifndef STAR_COMPONENT_BUS_TYPES_H
#define STAR_COMPONENT_BUS_TYPES_H

#ifdef __cplusplus
extern "C" {
#endif

    /* Include all type headers */
#include "star_bus_common_types.h"
#include "star_bus_event_types.h"
#include "star_bus_function_types.h"
#include "star_bus_manager_types.h"
#include "star_bus_protocol_types.h"

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_TYPES_H */
```

5.75 Doxygen File Documentation

Include `@file` documentation for headers with significant content.

```
/**
* @file star_error_handler.h
* @brief Error handling with retry logic and exponential backoff
*
* This module provides a concrete implementation of the error interface
* with automatic retry management, exponential backoff delays, and
* optional reset function callbacks.
*
* Key Features:
* - Configurable max retries and backoff delays
* - Exponential backoff between retries
* - Optional custom reset function for recovery
* - Thread-safe with mutex protection
*
* Example Usage:
* @code
* error_handler_t error_handler;
* error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
*
* if (operation_failed()) {
  *     RECORD_ERROR(&error_handler, ESP_ERR_TIMEOUT, "Timeout");
  *}
  * @endcode
  */
```

5.76 Complete Header Template

```c
        /* lib/star_module/include/star_module.h */

#ifndef STAR_COMPONENT_MODULE_H
#define STAR_COMPONENT_MODULE_H

#ifdef __cplusplus
extern "C" {
#endif

    /* === Includes === */
#include <stdbool.h>
#include <stdint.h>

#include "esp_err.h"
      /* === File Documentation === */
      /**
       * @file star_module.h
       * @brief Brief description of this module
       *
       * Detailed description of what this module provides and how to use
          it.
       */

      /* === Forward Declarations === */
      typedef struct star_module star_module_t;

    /* === Macros === */
#define STAR_MODULE_DEFAULT_TIMEOUT_MS (1000)

      /* === Type Definitions === */
      typedef struct star_module {
        bool initialized;
        uint32_t timeout_ms;
        void *user_ctx;
     } star_module_t;

      /* === Function Declarations === */

      /**
       * @brief Initialize the module.
       *
       * @param[out] module  Pointer to module structure to initialize.
       * @param[in]  timeout Timeout in milliseconds.
       * @return rx_err_t RX_OK on success.
       */
      esp_err_t star_module_init(star_module_t * module, uint32_t
          timeout);

      /**
       * @brief Deinitialize the module.
       *
```

```
50          * @param[in,out] module Pointer to module structure to
                deinitialize.
51          * @return rx_err_t RX_OK on success.
52          */
53         esp_err_t star_module_deinit(star_module_t * module);
54
55 #ifdef __cplusplus
56 }
57 #endif
58
59 #endif /* STAR_COMPONENT_MODULE_H */
```

5.77 Summary

- Use include guards: `STAR_COMPONENT_FILENAME_H`
- Include C++ extern "C" blocks
- Order includes: RTOS, standard library, platform HAL, project
- Use forward declarations to minimize dependencies
- Split types into separate _types.h headers for complex modules
- Document files with `@file` Doxygen blocks
- Keep headers focused on declarations, not implementations

This section establishes standards for horizontal spacing in the STAR firmware codebase.

5.78 General Rules

- **Space After Keywords**: Add space after `if`, `for`, `while`, `switch`.
- **Binary Operators**: Add a single space around binary operators.
- **No Space After Function Names**: No space between function name and opening parenthesis.
- **Alignment**: Align related declarations and comments.

5.79 Space After Control Keywords

Add a space between keywords and opening parenthesis.

```
1 /* CORRECT - Space after keywords */
2 if (ret != RX_OK) {
3       return ret;
4 }
5
6 for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
7       process(curr);
8 }
9
```

```c
while (retry_count > 0) {
      attempt_operation();
      retry_count--;
}

switch (config->type) {
    case k_star_bus_type_i2c: {
      /* ... */
      break;
   }
}

/* WRONG - No space after keywords */
if(ret != RX_OK) {          /* Missing space */
      return ret;
}

for(int i = 0; i < 10; i++) {  /* Missing space */
      /* ... */
}
```

5.80 No Space After Function Names

Do not add space between function name and opening parenthesis.

```c
/* CORRECT - No space after function name */
rx_err_t ret = star_bus_manager_init(&manager, "main", NULL, NULL);
RX_LOG_INFO(s_TAG, "Initialized bus manager");
free(buffer);

/* WRONG - Space after function name */
esp_err_t ret = star_bus_manager_init (&manager, "main", NULL, NULL);
RX_LOG_INFO (s_TAG, "Initialized bus manager");
free (buffer);
```

5.81 Binary Operators

Add a single space around binary operators.

```c
/* CORRECT - Spaces around binary operators */
int result = a + b;
bool is_valid = (count > 0) && (size <= MAX_SIZE);
uint32_t mask = flags | FLAG_ENABLED;
int shifted = value << 2;
size_t remaining = total - processed;

/* Assignment operators */
count += 1;
flags |= FLAG_INITIALIZED;
value >>= 4;

/* Comparison operators */
if (ret != RX_OK) { /* ... */}
```

```
if (count >= threshold) { /* ... */}
if (ptr == NULL) { /* ... */}

/* WRONG - Missing spaces */
int result = a+b;
bool is_valid = (count>0)&&(size<=MAX_SIZE);
if (ret!=ESP_OK) { /* ... */}
```

5.82 Unary Operators

No space between unary operator and operand.

```
/* CORRECT - No space with unary operators */
count++;
--remaining;
bool is_null = !ptr;
int negative = -value;
uint32_t complement = ~mask;
size_t size = sizeof(star_bus_config_t);
int* ptr = &value;
int deref = *ptr;

/* WRONG - Space with unary operators */
count ++;
-- remaining;
bool is_null = ! ptr;
```

5.83 Pointer Declarations

Place asterisk with the type, not the variable name.

```
/* CORRECT - Asterisk with type */
star_bus_config_t* config;
const char* name;
void* user_ctx;

/* Function parameters */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*              tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);

/* Multiple pointers on same line (avoid when possible) */
int *a, *b;  /* Both are pointers */

/* WRONG - Various incorrect styles */
star_bus_config_t *config;     /* Asterisk with variable */
star_bus_config_t * config;    /* Spaces around asterisk */
```

5.84 Comma and Semicolon Spacing

Space after comma and semicolon, not before.

```c
/* CORRECT */
star_bus_manager_init(&manager, "main", &error_iface, &pin_iface);

for (int i = 0; i < count; i++) {
      /* ... */
}

int array[] = {1, 2, 3, 4, 5};

/* WRONG */
star_bus_manager_init(&manager ,"main" ,&error_iface ,&pin_iface);
for (int i = 0 ;i < count ;i++) { /* ... */}
int array[] = {1 ,2 ,3 ,4 ,5};
```

5.85 Parentheses Spacing

No space after opening parenthesis or before closing parenthesis.

```c
/* CORRECT */
if (ret != RX_OK) {
      return ret;
}

result = calculate(a, b, c);

/* WRONG - Spaces inside parentheses */
if ( ret != RX_OK ) {
      return ret;
}

result = calculate( a, b, c );
```

5.86 Parameter Alignment

Align parameters vertically in multi-line function declarations and calls.

```c
/* CORRECT - Aligned parameters */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);

star_bus_config_t* star_bus_config_create_spi_device(
const char*                         name,
spi_host_device_t                   host,
gpio_num_t                          copi_pin,
gpio_num_t                          cipo_pin,
gpio_num_t                          sclk_pin,
int32_t                             dma_chan,
const spi_device_interface_config_t* dev_cfg);

/* Long function calls */
```

```
ESP_RETURN_ON_FALSE(manager && config,
ESP_ERR_INVALID_ARG,
s_TAG,
"Manager or config pointer is NULL");
```

5.87 Variable and Comment Alignment

Align related variables and their comments.

```
/* CORRECT - Aligned declarations and comments */
typedef struct error_handler {
      uint32_t error_count;          /**< Total errors recorded */
      uint32_t max_retries;          /**< Max retry attempts */
      uint32_t current_retry;        /**< Current retry counter */
      uint32_t base_retry_delay;     /**< Base delay (ms) */
      uint32_t max_retry_delay;      /**< Max delay (ms) */
      uint32_t current_retry_delay; /**< Current delay with backoff */
      rx_err_t last_error;           /**< Last recorded error code */
      bool in_error_state;           /**< Whether in error state */
      void *reset_context;           /**< Context for reset function */
      SemaphoreHandle_t mutex;       /**< Mutex for thread-safety */
} error_handler_t;

/* Aligned local variables */
i2c_port_t       port    = config->proto.i2c.port;
uint8_t          address = config->proto.i2c.address;
rx_err_t        ret     = RX_OK;
i2c_cmd_handle_t cmd     = NULL;
```

5.88 Struct Initializer Alignment

Align designated initializers.

```
/* CORRECT - Aligned initializers */
star_bus_event_t event = {
  .bus_type = k_star_bus_type_i2c,
  .bus_name = config->name,
  .i2c      = {
    .port     = port,
    .address  = address,
    .is_write = true,
    .len      = len
 }
};

spi_device_interface_config_t spi_cfg = {
  .clock_speed_hz = 10 * 1000 * 1000,
  .mode           = 0,
  .spics_io_num   = GPIO_NUM_5,
  .queue_size     = 7,
  .pre_cb         = NULL,
  .post_cb        = NULL,
};
```

5.89 Summary

Context	Rule
After `if`, `for`, `while`, `switch`	Space
After function name	No space
Around binary operators	Space
With unary operators	No space
After comma/semicolon	Space
Inside parentheses	No space
Pointer asterisk	With type
Multi-line parameters	Align vertically

This section establishes standards for include statement organization and ordering in the STAR firmware codebase.

5.90 General Rules

- **Use Angle Brackets for System/Library Headers**: `<stdio.h>`.
- **Use Quotation Marks for Project Headers**: `"my_project.h"`.
- **Minimal Includes in Header Files**: Only include what is necessary.
- **Group and Order Includes**: Follow consistent ordering.

5.91 Include Order

Includes should be ordered in these groups, separated by blank lines:

1. Primary header (in .c files)
2. FreeRTOS includes
3. Standard library includes
4. Platform HAL includes
5. Project includes

```
    /* lib/star_bus/src/star_bus_i2c.c */

    /* 1. Primary header - always first in .c file */
#include "star_bus_i2c.h"

    /* 2. FreeRTOS includes */
#include "freertos/FreeRTOS.h"

    /* 3. Standard library includes */
#include <inttypes.h>
#include <string.h>

    /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */
    /* Platform-specific: RX72N uses rx_check.h */
```

```
#include "esp_log.h"
#include "rx_check.h"

    /* 5. Project includes */
#include "star_bus_manager.h"
#include "star_bus_types.h"
```

5.92 Complete Example from Codebase

```
    /* lib/star_bus/src/star_bus_manager.c */

    /* Primary header */
#include "star_bus_manager.h"

    /* FreeRTOS */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"

    /* Standard library */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

    /* Platform HAL (ESP-IDF, Renesas, etc.) */
    /* Platform-specific: RX72N uses rx_check.h */
#include "esp_log.h"
#include "rx_check.h"
#include "sdkconfig.h"

    /* Project includes */
#include "star_bus_adc.h"
#include "star_bus_config.h"
#include "star_bus_gpio.h"
#include "star_bus_i2c.h"
#include "star_bus_spi.h"
```

5.93 FreeRTOS Include Order

FreeRTOS requires specific include ordering. Always include `FreeRTOS.h` first.

```
    /* CORRECT - FreeRTOS.h must come first */
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "freertos/task.h"

    /* WRONG - Other FreeRTOS headers before FreeRTOS.h */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h" /* Error: FreeRTOS.h not included first */
```

5.94 System Includes

Use angle brackets for system and standard library headers.

```
    /* Standard C library */
#include <inttypes.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

5.95 Platform HAL Includes

Platform HAL headers (ESP-IDF, Renesas, etc.) use quotation marks and are grouped separately.

```
    /* Platform-specific driver includes (example: ESP32) */
#include "driver/gpio.h"
#include "driver/i2c.h"
#include "driver/spi_master.h"
#include "driver/uart.h"

    /* Platform system includes */
    /* Platform-specific: RX72N uses rx_check.h */
#include "platform_system.h"
#include "rx_check.h"
#include "rx_err.h" /* Platform error types */
#include "rx_log.h" /* Platform logging */

    /* Platform peripheral includes (if needed) */
#include "esp_adc/adc_oneshot.h"
#include "platform_adc.h"

    /* Configuration */
#include "sdkconfig.h"
```

5.96 Project Includes

Use quotation marks for project headers. Alphabetize within each group.

```
    /* Project includes - alphabetized */
#include "star_bus_adc.h"
#include "star_bus_config.h"
#include "star_bus_gpio.h"
#include "star_bus_i2c.h"
#include "star_bus_manager.h"
#include "star_bus_spi.h"
#include "star_bus_types.h"
#include "star_error_interface.h"
#include "star_pin_interface.h"
```

5.97 Header File Includes

Header files should minimize includes and use forward declarations when possible.

```
    /* star_bus_manager.h - Header file includes */
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

    /* Only include what's needed for declarations */
#include "esp_err.h"
#include "star_bus_manager_types.h"
      /* Use forward declarations for pointer-only usage */
      struct star_bus_config; /* Don't include star_bus_config.h */

      /* Function using forward-declared type */
      rx_err_t star_bus_manager_add_bus(star_bus_manager_t * manager,
                                        struct star_bus_config * config);

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.98 Conditional Includes

Use `#ifdef` for platform-specific or optional includes.

```
    /* Platform-specific includes */
#ifdef CONFIG_IDF_TARGET_ESP32S3
#include "esp32s3/rom/gpio.h"
#else
#include "esp32/rom/gpio.h"
#endif

    /* Optional feature includes */
#ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
#include "esp_debug_helpers.h"
#endif
```

5.99 Include Comments

Add comments for non-obvious includes explaining why they are needed.

```
#include "star_bus_manager.h"

#include "freertos/FreeRTOS.h"

#include <inttypes.h>
#include <string.h>
```

```
7
8       /* Platform-specific: RX72N uses rx_check.h */
9   #include "esp_log.h"
10  #include "rx_check.h"
11  #include "star_bus_manager.h" /* Needed for star_bus_manager_find_bus */
12  #include "star_bus_types.h"   /* Needed for star_bus_config_t and union */
```

5.100 Avoiding Circular Dependencies

Structure your includes to avoid circular dependencies.

```
1   /* Problem: Circular dependency */
2   /* star_bus_manager.h includes star_bus_config.h */
3   /* star_bus_config.h includes star_bus_manager.h */
4
5   /* Solution: Use forward declarations */
6   /* star_bus_config.h */
7   struct star_bus_manager;  /* Forward declare, don't include */
8
9   rx_err_t star_bus_config_init(star_bus_config_t* config,
10  struct star_bus_manager* manager);
11
12      /* star_bus_config.c - Include full header here */
13  #include "star_bus_config.h"
14  #include "star_bus_manager.h" /* Full definition needed in .c */
```

5.101 Summary

Order	Category	Style
1	Primary header (.c files)	`"..."`
2	FreeRTOS	`"freertos/..."`
3	Standard library	`<...>`
4	Platform HAL	`"..."`
5	Project	`"..."`

- Always include `FreeRTOS.h` before other FreeRTOS headers
- Separate groups with blank lines
- Alphabetize within each group
- Minimize includes in headers; use forward declarations
- Add comments for non-obvious include dependencies

This project follows the rule of **2 spaces** for indentation, without using tabs.
Unix-style line endings ('LF', represented as '\n') are mandatory.
Logging is handled using platform-specific logging macros (e.g., `RX_LOG_*` for STAR, `ESP_LOG*` for ESP-IDF compatibility layer).

- **RX_LOG_ERROR / ESP_LOGE** - Error

- **RX_LOG_WARN / ESP_LOGW** - Warning
- **RX_LOG_INFO / ESP_LOGI** - Info
- **RX_LOG_DEBUG / ESP_LOGD** - Debug
- **RX_LOG_VERBOSE / ESP_LOGV** - Verbose

This section establishes comprehensive naming standards for all identifiers in the STAR firmware codebase. Consistent naming improves readability, maintainability, and reduces cognitive load when navigating the codebase.

5.102 General Rules

- **Language**: All identifiers must be in English.
- **Clarity**: Names should be descriptive and self-documenting.
- **Abbreviations**: Avoid abbreviations unless they are widely understood (e.g., `GPIO`, `I2C`, `SPI`).

5.103 Functions and Variables

Functions and variables use `snake_case`—all lowercase with words separated by underscores.

```
/* CORRECT - snake_case for functions and variables */
esp_err_t star_bus_manager_init(star_bus_manager_t* manager, const char*
    tag);
uint32_t  error_count;
size_t    bytes_written;
bool      is_initialized;

/* WRONG - other naming styles */
rx_err_t StarBusManagerInit();   /* PascalCase */
rx_err_t starBusManagerInit();   /* camelCase */
uint32_t  ErrorCount;             /* PascalCase */
```

5.104 Module Prefixes

All public functions must use the module prefix pattern: `star_[module]_[function]`.

```
/* From star_bus_manager.h */
rx_err_t star_bus_manager_init(...);
rx_err_t star_bus_manager_add_bus(...);
rx_err_t star_bus_manager_remove_bus(...);
rx_err_t star_bus_manager_deinit(...);

/* From star_bus_config.h */
star_bus_config_t* star_bus_config_create_i2c(...);
star_bus_config_t* star_bus_config_create_spi_device(...);
rx_err_t           star_bus_config_destroy(...);

/* From star_bus_i2c.h */
rx_err_t star_bus_i2c_write(...);
```

```
rx_err_t star_bus_i2c_read(...);
void      star_bus_i2c_init_default_ops(...);

/* From star_error_handler.h */
rx_err_t error_handler_init(...);
rx_err_t error_handler_record_error(...);
bool      error_handler_can_retry(...);
```

5.105 Constants and Macros

Constants and macros use `SCREAMING_SNAKE_CASE`—all uppercase with words separated by underscores.

```
    /* CORRECT - SCREAMING_SNAKE_CASE for macros and constants */
#define STAR_BUS_NAME_MAX_LEN (32)
#define CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS (1000)
#define I2C_MASTER_WRITE 0
#define I2C_MASTER_READ 1
#define GPIO_NUM_MAX 48

    /* Convenience macro with proper naming */
#define RECORD_ERROR(handler, code, desc)
   \\
      do {

         \\
        error_handler_record_error((handler), (code), (desc), __FILE__,
                 \\
                                    __LINE__, __func__);
                                                                \\
     } while (0)

#define STAR_IFACE_RECORD_ERROR(iface, err, msg)
   \\
      ((iface) && (iface)->record_error
                                                \\
           ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
                   \\
                                    __LINE__, __func__)
                                                          \\
           : (err))

    /* WRONG */
#define star_bus_name_max_len 32 /* lowercase */
#define StarBusNameMaxLen 32     /* PascalCase */
```

5.106 Type Names

All custom types must use `snake_case` with the `_t` suffix.

```
/* CORRECT - snake_case_t for type names */
typedef struct star_bus_config  star_bus_config_t;
typedef struct star_bus_manager star_bus_manager_t;
```

```
typedef struct error_handler    error_handler_t;
typedef enum star_bus_type      star_bus_type_t;

/* Callback function types also use _t suffix */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus, void*
    user_ctx);
typedef rx_err_t (*pin_registration_function_t)(gpio_num_t  pin_num,
const char* bus_name,
const char* pin_usage,
void*       user_ctx);

/* WRONG */
typedef struct StarBusConfig StarBusConfig;  /* PascalCase */
typedef struct star_bus_config star_bus_config;  /* missing _t suffix */
```

5.107 Enum Members

Enum members use the `k_` prefix followed by `snake_case`.

```
/* CORRECT - k_prefix with snake_case */
typedef enum {
  k_star_bus_type_none,    /**< No bus type specified */
  k_star_bus_type_i2c,     /**< I2C bus */
  k_star_bus_type_spi,     /**< SPI bus */
  k_star_bus_type_gpio,    /**< GPIO bus */
  k_star_bus_type_adc,     /**< ADC bus */
  k_star_bus_type_count,   /**< Number of bus types (sentinel) */
} star_bus_type_t;

/* Usage in switch statements */
switch (config->type) {
    case k_star_bus_type_i2c: {
      /* Handle I2C */
      break;
   }
    case k_star_bus_type_spi: {
      /* Handle SPI */
      break;
   }
    default: {
      RX_LOG_WARN(s_TAG, "Unknown bus type: %d", config->type);
      break;
   }
}

/* WRONG */
typedef enum {
  STAR_BUS_TYPE_I2C,    /* SCREAMING_SNAKE_CASE without k_ */
  StarBusTypeI2c,       /* PascalCase */
  starBusTypeI2c,       /* camelCase */
} bad_enum_t;
```

5.108 Static Variables (File-Scoped)

Static file-scoped variables use the `s_` prefix followed by `snake_case`.

```
/* CORRECT - s_ prefix for static file-scoped variables */
static const char* s_TAG = "STAR_BUS_MANAGER";
static const uint32_t s_i2c_timeout_ms = 1000;

/* From star_bus_i2c.c */
static const char* s_TAG = "STAR_BUS_I2C";
static const uint32_t s_i2c_timeout_ms = 1000;

/* From star_bus_config.c */
static const char* s_TAG = "STAR_BUS_CONFIG";

/* WRONG - missing s_ prefix */
static const char* TAG = "MODULE";
static uint32_t timeout_ms = 1000;
```

5.109 Global Variables

Global variables use the `g_` prefix followed by `snake_case`. However, global variables are **strongly discouraged**. Prefer dependency injection or static file-scoped variables.

```
/* CORRECT (but discouraged) - g_ prefix for globals */
g_system_state_t g_system_state;
uint32_t g_error_count;

/* PREFERRED - use dependency injection instead */
typedef struct {
      star_error_interface_t *error_iface; /* Injected */
      star_pin_interface_t *pin_iface;     /* Injected */
} star_bus_manager_t;

/* Initialize with injected dependencies */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);
```

5.110 Private Functions

5.110.1 File-Scoped Static Functions (internal_ prefix)

Static functions that are internal to a single source file use the `internal_` prefix.

```
/* From star_bus_manager.c - internal helper functions */
static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage,
bool                  can_be_shared)
{
      if (pin < 0 || pin >= GPIO_NUM_MAX) {
```

```
        return RX_OK; /* Not an error, pin is not used */
    }
    /* ... implementation ... */
}

static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
    pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage)
{
    /* ... implementation ... */
}

static rx_err_t internal_register_config_pins(star_pin_interface_t*
    pin_iface,
const star_bus_config_t* config)
{
    /* ... implementation ... */
}

/* From star_bus_i2c.c */
static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
    config,
const uint8_t*          data,
size_t                  len,
uint8_t                 reg_addr,
size_t*                 bytes_written);

static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
    config,
i2c_port_t              port,
i2c_cmd_handle_t        cmd,
const char*             operation_name);
```

5.110.2 Exposed Private Functions (priv_ prefix)

Functions that must be exposed across files within a module (but are not part of the public API) use the `priv_` prefix.

```
/* In private header (e.g., star_bus_internal.h) */
/* These functions are used by multiple .c files in the same module
but should not be called by external code */

rx_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
void     priv_star_bus_release_mutex(star_bus_manager_t* manager);

/* Used in implementation files within the module */
rx_err_t star_bus_config_init(star_bus_config_t* config,
struct star_bus_manager* manager)
{
    /* Validate using private helper */
```

```
    esp_err_t ret = priv_star_bus_validate_config(config);
    if (ret != RX_OK) {
      return ret;
    }
    /* ... rest of implementation ... */
}
```

5.111 Summary Table

Identifier Type	Convention	Example
Functions	`snake_case`	`star_bus_manager_init`
Variables	`snake_case`	`bytes_written`
Constants/Macros	`SCREAMING_SNAKE_CASE`	`STAR_BUS_NAME_MAX_LEN`
Type names	`snake_case_t`	`star_bus_config_t`
Enum members	`k_snake_case`	`k_star_bus_type_i2c`
Static variables	`s_snake_case`	`s_TAG`
Global variables	`g_snake_case`	`g_system_state`
Internal functions	`internal_snake_case`	`internal_register_bus_pin`
Private functions	`priv_snake_case`	`priv_star_bus_validate_config`
Module prefix	`star_[module]_`	`star_bus_`, `star_error_`

This section describes the patterns and conventions for defining custom types in the STAR firmware codebase. Consistent type definition patterns improve code readability and enable better tooling support.

5.112 Typedef Struct Pattern

All structures should be defined using the typedef pattern with the `_t` suffix.

5.112.1 Standard Pattern

```
/* Forward declaration (in header for dependencies) */
typedef struct star_bus_config  star_bus_config_t;
typedef struct star_bus_manager star_bus_manager_t;

/* Full definition (in header or implementation) */
typedef struct star_bus_config {
    const char *name;     /**< Unique name for this bus/device instance
       */
    star_bus_type_t type; /**< Type of bus (I2C, SPI, etc.) */
    bool initialized;     /**< Whether this bus has been initialized */
    void *handle;         /**< Driver/device handle */
    void *user_ctx;       /**< User context pointer for callbacks */

    /* Protocol-specific configuration (see union section) */
    union {
      star_i2c_bus_config_t i2c;
      star_spi_bus_config_t spi;
      star_gpio_bus_config_t gpio;
      star_adc_bus_config_t adc;
```

```
    } proto;

    struct star_bus_config *next; /**< Linked list pointer */
} star_bus_config_t;
```

5.112.2 Error Handler Example

```
/* From star_error_handler.h */
typedef struct error_handler {
    uint32_t error_count;          /**< Total number of errors recorded */
    uint32_t max_retries;          /**< Maximum retry attempts allowed */
    uint32_t current_retry;        /**< Current retry attempt counter */
    uint32_t base_retry_delay;     /**< Base delay between retries (ms) */
    uint32_t max_retry_delay;      /**< Maximum retry delay (ms) */
    uint32_t current_retry_delay; /**< Current delay with backoff */
    rx_err_t last_error;           /**< Last recorded error code */
    bool in_error_state;           /**< Whether in error state */
    void *reset_context;           /**< Context for reset function */
    SemaphoreHandle_t mutex;       /**< Mutex for thread-safety */

    rx_err_t (*reset_fn)(void *context); /**< Optional reset function */
} error_handler_t;
```

5.113 Union Usage for Protocol Configurations

Unions are used to store protocol-specific configurations efficiently. This pattern allows a single configuration structure to support multiple bus types.

```
/* From star_bus_manager_types.h */
typedef struct star_bus_config {
    const char *name;
    star_bus_type_t type; /* Discriminator for the union */
    bool initialized;
    void *handle;
    void *user_ctx;

    /* Union holds protocol-specific configuration */
    union {
      star_i2c_bus_config_t i2c;   /**< I2C-specific configuration */
      star_spi_bus_config_t spi;   /**< SPI-specific configuration */
      star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
      star_adc_bus_config_t adc;   /**< ADC-specific configuration */
    } proto;

    struct star_bus_config *next;
} star_bus_config_t;

/* Accessing union members based on type discriminator */
switch (config->type) {
    case k_star_bus_type_i2c: {
```

```
        i2c_port_t port = config->proto.i2c.port;
        uint8_t address = config->proto.i2c.address;
        /* ... */
        break;
    }
     case k_star_bus_type_spi: {
        spi_host_device_t host = config->proto.spi.host;
        gpio_num_t copi_pin = config->proto.spi.copi_io_num;
        /* ... */
        break;
    }
        /* ... other cases ... */
}
```

5.113.1 Protocol - Specific Structures

```
/* I2C bus configuration (from star_bus_protocol_types.h) */
typedef struct star_i2c_bus_config {
        i2c_port_t port;                /**< I2C port number */
        i2c_config_t config;            /**< Platform I2C configuration */
        uint8_t address;                /**< 7-bit I2C device address */
        star_i2c_callbacks_t callbacks; /**< User callbacks */
        star_i2c_ops_t ops;             /**< Operation function pointers */
} star_i2c_bus_config_t;

/* SPI bus configuration */
typedef struct star_spi_bus_config {
        spi_host_device_t host;                 /**< SPI host number */
        bool is_peripheral;                     /**< True if SPI peripheral
            mode */
        spi_bus_config_t bus_cfg;               /**< Platform SPI bus config
            */
        spi_device_interface_config_t dev_cfg; /**< Device config */
        star_spi_callbacks_t callbacks;
        star_spi_ops_t ops;

        /* Inclusive terminology pin names (see Terminology section) */
        gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
        gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
        gpio_num_t sclk_io_num; /**< Serial Clock */
        gpio_num_t cs_io_num;   /**< Chip Select */
        /* ... */
} star_spi_bus_config_t;
```

5.114 Callback Function Types

Callback function pointer types follow a consistent pattern with the `_t` suffix.

```
/* From star_bus_common_types.h - Pin registration callback */
typedef rx_err_t (*pin_registration_function_t)(gpio_num_t  pin_num,
const char* bus_name,
const char* pin_usage,
```

```
void*       user_ctx);

/* From star_bus_manager.h - Bus access callback */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
void*              user_ctx);

/* From star_bus_function_types.h - I2C operation callbacks */
typedef rx_err_t (*star_i2c_write_fn_t)(const star_bus_config_t* config,
const uint8_t*           data,
size_t                   len,
uint8_t                  reg_addr,
size_t*                  bytes_written);

typedef rx_err_t (*star_i2c_read_fn_t)(const star_bus_config_t* config,
uint8_t*                 data,
size_t                   len,
uint8_t                  reg_addr,
size_t*                  bytes_read);

/* Event callbacks */
typedef void (*star_i2c_transfer_complete_cb_t)(const star_bus_event_t*
   event,
void*                   user_ctx);

typedef void (*star_i2c_error_cb_t)(const star_bus_event_t* event,
rx_err_t               error,
void*                   user_ctx);

/* Grouping related callbacks in a struct */
typedef struct star_i2c_callbacks {
      star_i2c_transfer_complete_cb_t on_transfer_complete;
      star_i2c_error_cb_t on_error;
} star_i2c_callbacks_t;
```

5.115 Interface Patterns (Dependency Injection)

Interfaces use structs with function pointers and an opaque context pointer. This pattern enables dependency injection and testability.

```
/* From star_error_interface.h */
typedef struct star_error_interface {
      /**
       * @brief Record an error
       * @param ctx Implementation context
       * @param error Error code
       * @param message Error message
       * @param file Source file
       * @param line Line number
       * @param func Function name
       * @return ESP_OK on success
       */
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
         *message,
```

```
                                const char *file, int line, const char
                                    *func);

        /**
         * @brief Check if retry is possible
         * @param ctx Implementation context
         * @return true if retry is allowed
         */
        bool (*can_retry)(void *ctx);

        /**
         * @brief Reset error state
         * @param ctx Implementation context
         * @return ESP_OK on success
         */
        rx_err_t (*reset_state)(void *ctx);

        /**
         * @brief Implementation context (opaque pointer)
         */
        void *ctx;
} star_error_interface_t;

/* From star_pin_interface.h */
typedef struct star_pin_interface {
        rx_err_t (*register_pin)(void *ctx, int pin, const char
            *description,
                                 bool shared);
        rx_err_t (*unregister_pin)(void *ctx, int pin, const char
            *description);
        rx_err_t (*validate)(void *ctx);
        void *ctx;
} star_pin_interface_t;
```

5.115.1 Interface Usage Pattern

```
/* Creating an interface from concrete implementation */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
error_handler_get_interface(&error_iface, &error_handler);

/* Injecting interfaces into components */
star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

/* Using interface through function pointers */
if (error_iface.can_retry(error_iface.ctx)) {
        /* Retry operation */
}

/* Or using the convenience macro */
```

```
STAR_IFACE_RECORD_ERROR(&error_iface, ESP_ERR_TIMEOUT, "Operation timed
    out");
```

5.116 Opaque Handle Patterns

For complex objects that should not be directly manipulated, use opaque handles.

```
/* Forward declaration creates an opaque type */
typedef struct star_bus_manager star_bus_manager_t;

/* Public API only uses pointers to the opaque type */
esp_err_t star_bus_manager_init(star_bus_manager_t* manager, ...);
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager, ...);

/* Full definition is only in the .c file or private header */
/* Users cannot access internal fields directly */

/* Example usage - users allocate storage but don't access internals */
star_bus_manager_t bus_manager;  /* Stack allocation */
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

/* Or heap allocation */
star_bus_manager_t* manager_ptr = malloc(sizeof(star_bus_manager_t));
star_bus_manager_init(manager_ptr, "main", &error_iface, &pin_iface);
```

5.117 Operations Structure Pattern

Group related function pointers in an operations structure for pluggable implementations.

```
/* From star_bus_protocol_types.h */
typedef struct star_i2c_ops {
      star_i2c_write_fn_t write;                 /**< Write with register
          */
      star_i2c_read_fn_t read;                   /**< Read with register
          */
      star_i2c_write_command_fn_t write_command; /**< Write command byte
          */
      star_i2c_read_raw_fn_t read_raw;           /**< Read raw bytes */
} star_i2c_ops_t;

typedef struct star_spi_ops {
      star_spi_transmit_fn_t transmit; /**< Transmit only */
      star_spi_receive_fn_t receive;   /**< Receive only */
      star_spi_transfer_fn_t transfer; /**< Full-duplex transfer */
} star_spi_ops_t;

typedef struct star_gpio_ops {
      rx_err_t (*write_digital)(const struct star_bus_config *config,
                                uint8_t pin_index, uint8_t value);
      rx_err_t (*read_digital)(const struct star_bus_config *config,
                               uint8_t pin_index, uint8_t *value);
} star_gpio_ops_t;

```

```c
/* Initializing default operations */
void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
{
      if (ops == NULL) {
        RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
        return;
     }
      ops->write = internal_star_bus_i2c_write;
      ops->read = internal_star_bus_i2c_read;
      ops->write_command = internal_star_bus_i2c_write_command;
      ops->read_raw = internal_star_bus_i2c_read_raw;
}
```

5.118 Manager Structure Pattern

Manager structures coordinate multiple resources with thread-safe access.

```c
/* From star_bus_manager_types.h */
typedef struct star_bus_manager {
      star_bus_config_t *buses;            /**< Linked list of bus
          configs */
      SemaphoreHandle_t mutex;             /**< Mutex for thread-safety */
      const char *tag;                     /**< Logging tag */
      star_error_interface_t *error_iface; /**< Injected error handler */
      star_pin_interface_t *pin_iface;     /**< Injected pin validator */

      /* Resource tracking */
      bool spi_host_initialized[SPI_HOST_MAX];
      int spi_device_count[SPI_HOST_MAX];
} star_bus_manager_t;
```

5.119 Best Practices

- **Always use _t suffix**: Makes types easily distinguishable from variables.
- **Use forward declarations**: Minimize header dependencies and compilation time.
- **Document all fields**: Use Doxygen comments for struct members.
- **Keep unions discriminated**: Always pair unions with a type field.
- **Prefer static const over #define**: For typed constants within structures.
- **Use opaque types for encapsulation**: Hide implementation details from users.
- **Group related function pointers**: Use ops structs for pluggable implementations.

This section describes the standard file and directory structure for the STAR firmware codebase. Consistent organization improves navigability and maintainability.

5.120 Library Directory Structure

Each library follows a standard component structure (PlatformIO/ESP-IDF on ESP32, CMake on RX72N):

```
lib/star_component/
    CMakeLists.txt       # CMake build configuration
    library.json         # PlatformIO library metadata
    include/             # Public headers (API)
        star_component.h
        star_component_types.h
    src/                 # Implementation files
        star_component.c
```

5.120.1 Example: star_bus Library

```
lib/star_bus/
    CMakeLists.txt
    library.json
    include/
        star_bus_adc.h
        star_bus_common_types.h
        star_bus_config.h
        star_bus_event_types.h
        star_bus_function_types.h
        star_bus_gpio.h
        star_bus_i2c.h
        star_bus_manager.h
        star_bus_manager_types.h
        star_bus_protocol_types.h
        star_bus_spi.h
        star_bus_types.h
    src/
        star_bus_adc.c
        star_bus_config.c
        star_bus_gpio.c
        star_bus_i2c.c
        star_bus_manager.c
        star_bus_spi.c
        star_bus_types.c
```

5.121 CMakeLists.txt Structure

```
#lib / star_bus / CMakeLists.txt

idf_component_register(
SRCS
"src/star_bus_adc.c"
"src/star_bus_config.c"
```

```
"src/star_bus_gpio.c"
"src/star_bus_i2c.c"
"src/star_bus_manager.c"
"src/star_bus_spi.c"
"src/star_bus_types.c"
INCLUDE_DIRS
"include"
REQUIRES
driver
esp_adc
freertos
star_core
)
```

5.122 Header File Organization

Header files must follow a consistent internal structure.

5.122.1 Header File Template

```
        /* lib/star_bus/include/star_bus_manager.h */

#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

    /* === Includes === */
#include "esp_err.h"
#include "star_bus_manager_types.h"
      /**
       * @file star_bus_manager.h
       * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
          protocols
       *
       * The bus manager provides a central registry for managing multiple
       * bus configurations. It supports dependency injection of error
          handlers
       * and pin validators for loose coupling.
       *
       * Key Features:
       * - Thread-safe bus management with mutex protection
       * - Support for multiple bus types (I2C, SPI, GPIO, etc.)
       * - Automatic pin registration and conflict detection
       * - Safe bus access via callback pattern
       *
       * Example Usage:
       * @code
       * // Initialize bus manager with dependency injection
       * star_bus_manager_t bus_manager;
```

```
31          * star_bus_manager_init(&bus_manager, "main", &error_iface,
                &pin_iface);
32          *
33          * // Create and add an I2C bus
34          * star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
35          *     "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22,
                400000);
36          * star_bus_manager_add_bus(&bus_manager, i2c_bus);
37          * @endcode
38          */
39
40        /* === Forward Declarations === */
41        /* (if needed beyond what's in included headers) */
42
43        /* === Public Functions === */
44
45        /**
46         * @brief Initialize a bus manager instance.
47         *
48         * @param[out] manager     Pointer to bus manager to initialize.
                Must not
49         * be NULL.
50         * @param[in]  tag         Optional tag string for logging. NULL
                uses
51         * default.
52         * @param[in]  error_iface Optional error handler interface (can be
                NULL).
53         * @param[in]  pin_iface   Optional pin validator interface (can be
                NULL).
54         * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
                is
55         * NULL, ESP_ERR_NO_MEM if mutex creation fails.
56         */
57        rx_err_t star_bus_manager_init(
58            star_bus_manager_t * manager, const char *tag,
59            star_error_interface_t *error_iface, star_pin_interface_t
                *pin_iface);
60
61            /* ... more function declarations ... */
62
63  #ifdef __cplusplus
64  }
65  #endif
66
67  #endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.122.2 Header Section Order

1. **File path comment**: Identifies the file location
2. **Include guard**: `#ifndef STAR_COMPONENT_FILENAME_H`
3. **C++ extern "C" opening**: For C++ compatibility

4. **Includes**: System, then platform HAL, then project headers
5. **File documentation**: `@file` Doxygen block with overview
6. **Forward declarations**: If needed
7. **Constants and macros**: Public defines
8. **Type definitions**: Enums, structs, typedefs
9. **Function declarations**: Grouped by category
10. **C++ extern "C" closing**: Matches opening
11. **Include guard closing**: `#endif /* ... */`

5.123 Include Guard Naming Convention

Include guards follow the pattern: `STAR_COMPONENT_FILENAME_H`

```
    /* star_bus_manager.h */
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H
    /* ... */
#endif /* STAR_COMPONENT_BUS_MANAGER_H */

    /* star_bus_config.h */
#ifndef STAR_COMPONENT_BUS_CONFIG_H
#define STAR_COMPONENT_BUS_CONFIG_H
    /* ... */
#endif /* STAR_COMPONENT_BUS_CONFIG_H */

    /* star_error_interface.h */
#ifndef STAR_ERROR_INTERFACE_H
#define STAR_ERROR_INTERFACE_H
    /* ... */
#endif /* STAR_ERROR_INTERFACE_H */

    /* star_pin_interface.h */
#ifndef STAR_PIN_INTERFACE_H
#define STAR_PIN_INTERFACE_H
    /* ... */
#endif /* STAR_PIN_INTERFACE_H */
```

5.124 Source File Organization

Source files follow a consistent internal structure.

5.124.1 Source File Template

```
    /* lib/star_bus/src/star_bus_manager.c */

    /* === Primary Header === */
#include "star_bus_manager.h"
```

```

    /* === System Includes === */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

    /* === Platform HAL Includes === */
    /* Platform-specific: RX72N uses rx_check.h */
#include "esp_log.h"
#include "rx_check.h"
#include "sdkconfig.h"

    /* === Project Includes === */
#include "star_bus_adc.h"
#include "star_bus_config.h"
#include "star_bus_gpio.h"

/* === Constants === */

static const char* s_TAG = "STAR_BUS_MANAGER";

/* === Private Function Prototypes === */

static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage,
bool                  can_be_shared);

static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
   pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage);

/* === Public Functions === */

rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                         "Manager pointer is NULL");

      manager->buses = NULL;
      manager->error_iface = error_iface;
      manager->pin_iface = pin_iface;

      /* ... rest of implementation ... */

```

```
58        return RX_OK;
59  }
60
61  /* ... more public functions ... */
62
63  /* === Private Functions === */
64
65  static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
66  gpio_num_t             pin,
67  const char*            bus_name,
68  const char*            usage,
69  bool                   can_be_shared)
70  {
71        if (pin < 0 || pin >= GPIO_NUM_MAX) {
72          return RX_OK; /* Not an error, pin is not used */
73       }
74        /* ... implementation ... */
75  }
```

5.124.2 Source Section Order

1. **File path comment**: Identifies the file location
2. **Primary header**: The corresponding .h file for this .c file
3. **System includes**: FreeRTOS, standard library (`<...>`)
4. **Platform HAL includes**: Platform-specific components
5. **Project includes**: Other project headers (`"..."`)
6. **Constants**: `static const` variables, including `s_TAG`
7. **Private function prototypes**: Forward declarations of static functions
8. **Public functions**: Implementation of API functions
9. **Private functions**: Implementation of static helper functions

5.125 Types Header Separation

For complex modules, separate type definitions into dedicated headers:

```
star_bus/include/
    star_bus_manager.h        # Main API (functions)
    star_bus_manager_types.h  # Types for manager
    star_bus_common_types.h   # Shared types (enums, forward decls)
    star_bus_protocol_types.h # Protocol-specific types
    star_bus_function_types.h # Function pointer types
    star_bus_event_types.h    # Event structures
    star_bus_types.h          # Controller header (includes all)
```

5.125.1 Controller Header Pattern

```c
    /* star_bus_types.h - includes all type headers */
#ifndef STAR_COMPONENT_BUS_TYPES_H
#define STAR_COMPONENT_BUS_TYPES_H

#ifdef __cplusplus
extern "C" {
#endif

    /* Controller include file for all public types */
#include "star_bus_common_types.h"
#include "star_bus_event_types.h"
#include "star_bus_function_types.h"
#include "star_bus_manager_types.h"
#include "star_bus_protocol_types.h"

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_TYPES_H */
```

5.126 Project Root Structure

```
star-esp32-firmware/
    CLAUDE.md              # Claude Code instructions
    platformio.ini         # PlatformIO configuration
    sdkconfig.defaults     # ESP-IDF default config
    lib/                   # Libraries/components
        star_core/         # Core interfaces
        star_bus/          # Bus abstraction
        star_error_handler/
        star_pin_validator/
        star_sensor_*/     # Sensor drivers
        star_motor/
        star_pid/
    src/                   # Main application
        main.c
    partitions/            # Partition tables
        partitions_4mb.csv
        partitions_16mb.csv
    docs/                  # Documentation
        style_guide.tex
        style_guide_sections/
    scripts/               # Build/utility scripts
    test/                  # Test files
```

5.127 Best Practices

- **One header per component**: Each component has its primary public header.
- **Minimize header dependencies**: Use forward declarations to reduce includes.
- **Self-contained headers**: Each header should compile independently.
- **Consistent naming**: Files match the component they contain.
- **Group related files**: Keep related headers and sources together.
- **Separate types from functions**: For complex modules, split into _types.h.
- **Document file purpose**: Use `@file` Doxygen blocks.

This section establishes the OSHWA (Open Source Hardware Association) inclusive terminology standards used throughout the STAR firmware codebase. These standards replace legacy terminology with more inclusive alternatives.

5.128 Overview

The STAR project follows inclusive terminology guidelines to ensure our codebase is welcoming to all contributors and avoids language with harmful historical connotations. This is particularly relevant in embedded systems where certain legacy terms have been deeply entrenched.

5.129 SPI Bus Terminology

5.129.1 COPI / CIPO(Not MOSI / MISO)

STAR Term	Legacy Term	Description
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
SCLK	SCLK	Serial Clock (unchanged)
CS	SS	Chip Select (unchanged)

```
/* CORRECT - OSHWA terminology */
typedef struct star_spi_bus_config {
      gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
      gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
      gpio_num_t sclk_io_num; /**< Serial Clock */
      gpio_num_t cs_io_num;   /**< Chip Select */
      /* ... */
} star_spi_bus_config_t;

/* Factory function uses inclusive naming */
star_bus_config_t* star_bus_config_create_spi_device(
const char*       name,
spi_host_device_t host,
gpio_num_t        copi_pin,   /* NOT mosi_pin */
gpio_num_t        cipo_pin,   /* NOT miso_pin */
gpio_num_t        sclk_pin,
int32_t           dma_chan,
```

```
const spi_device_interface_config_t* dev_cfg);

/* In documentation and comments */
/* CORRECT */
reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
bus_name,
"SPI COPI",  /* Descriptive usage */
reg_func,
user_ctx,
true);

/* WRONG */
reg_result = register_pin_if_valid(curr->proto.spi.mosi_io_num,
bus_name,
"SPI MOSI",  /* Legacy terminology */
reg_func,
user_ctx,
true);
```

5.130 I2C and General Bus Terminology

5.130.1 Controller / Peripheral(Not Master / Slave)

STAR Term	Legacy Term	Description
Controller	Master	Device that initiates communication
Peripheral	Slave	Device that responds to controller
Primary	Master	For configuration structures
Main	Master	For general reference

```
/* CORRECT - Controller/Peripheral terminology */
typedef struct star_spi_bus_config {
      spi_host_device_t host;
      bool is_peripheral; /**< True if operating as SPI peripheral */
      /* ... */
} star_spi_bus_config_t;

/* Factory function for peripheral mode */
star_bus_config_t* star_bus_config_create_spi_peripheral(
const char*       name,
spi_host_device_t host,
gpio_num_t        copi_pin,
gpio_num_t        cipo_pin,
gpio_num_t        sclk_pin,
gpio_num_t        cs_pin,
int32_t           queue_size,
uint8_t           mode);

/* In comments and documentation */
/**
* @brief I2C Controller operations
*
* The controller device initiates all transactions on the bus.
```

```
* Peripheral devices respond when addressed by the controller.
*/

/* WRONG */
bool is_slave;              /* Use is_peripheral */
star_bus_config_create_spi_slave();  /* Use _peripheral */
```

5.131 Mapping Platform Legacy APIs

Some platform HALs (like ESP-IDF) still use legacy terminology internally. When interfacing with these APIs, document the mapping clearly.

```
/* From star_bus_protocol_types.h */
typedef struct star_spi_bus_config {
      /* ... */

      /* SPI pin naming follows OSHWA standard:
       * - COPI (Controller Out, Peripheral In) - replaces MOSI
       * - CIPO (Controller In, Peripheral Out) - replaces MISO
       * This modern terminology removes master/slave language.
       * Some platforms (like ESP-IDF) use legacy mosi/miso terminology,
       * which we map here.
       */
      gpio_num_t
          copi_io_num; /**< GPIO for COPI (maps to platform mosi_io_num)
              */
      gpio_num_t
          cipo_io_num; /**< GPIO for CIPO (maps to platform miso_io_num)
              */
      /* ... */
} star_spi_bus_config_t;

/* When setting up platform HAL structures (example: ESP32) */
static rx_err_t internal_setup_spi_bus(star_bus_config_t* config)
{
      spi_bus_config_t bus_cfg = {
          /* Map our inclusive terminology to platform legacy names */
          .mosi_io_num = config->proto.spi.copi_io_num,
          .miso_io_num = config->proto.spi.cipo_io_num,
          .sclk_io_num = config->proto.spi.sclk_io_num,
          .quadwp_io_num = -1,
          .quadhd_io_num = -1,
          .max_transfer_sz = config->proto.spi.max_transfer_sz,
     };
      /* ... */
}

/* Similarly for I2C */
/* Some platforms use I2C_MODE_SLAVE/MASTER, we document as
   peripheral/controller */
i2c_config_t i2c_conf = {
  .mode = I2C_MODE_MASTER,  /* Controller mode in STAR terminology */
  /* or */
```

```
  .mode = I2C_MODE_SLAVE,    /* Peripheral mode in STAR terminology */
  /* ... */
};
```

5.132 Documentation Guidelines

When writing comments, documentation, or user-facing text:

```
/* CORRECT - Use inclusive terminology in all documentation */

/**
* @brief Configure ESP32 as SPI peripheral device
*
* The peripheral will respond to transactions initiated by an
* external SPI controller. Data flows:
* - COPI: Data received from controller
* - CIPO: Data sent to controller
*
* @param[in] name Unique name for this SPI peripheral instance
* @param[in] host SPI host device
* @param[in] copi_pin GPIO for Controller Out, Peripheral In
* @param[in] cipo_pin GPIO for Controller In, Peripheral Out
* ...
*/
star_bus_config_t* star_bus_config_create_spi_peripheral(...);

/* WRONG - Legacy terminology */
/**
* @brief Configure ESP32 as SPI slave device
*
* The slave will respond to transactions initiated by the
* SPI master. Data flows:
* - MOSI: Data from master to slave
* - MISO: Data from slave to master
*/
```

5.133 Code Review Checklist

When reviewing code, check for:

- **Variable names**: No “master”, “slave”, “mosi”, “miso”
- **Function names**: Use “controller”, “peripheral”, “copi”, “cipo”
- **Comments**: Update any legacy terminology
- **Documentation**: Ensure Doxygen uses inclusive terms
- **String literals**: Check log messages and error strings
- **Platform HAL mapping**: Document when interfacing with legacy APIs

5.134 Common Patterns

```
/* Logging with inclusive terminology */
RX_LOG_INFO(s_TAG, "SPI peripheral initialized on host %d", host);
RX_LOG_INFO(s_TAG, "I2C controller configured on port %d", port);

/* Error messages */
ESP_LOGE(s_TAG, "Failed to initialize SPI peripheral: %s",
rx_err_to_name(ret));
ESP_LOGE(s_TAG, "I2C controller transaction failed");

/* Pin registration descriptions */
internal_register_bus_pin(pin_iface, config->proto.spi.copi_io_num,
config->name, "SPI COPI", true);
internal_register_bus_pin(pin_iface, config->proto.spi.cipo_io_num,
config->name, "SPI CIPO", true);
```

5.135 Summary

Use This	Not This
Controller	Master
Peripheral	Slave
COPI	MOSI
CIPO	MISO
Primary	Master (for configs)
Main	Master (general)
Allowlist	Whitelist
Blocklist	Blacklist

Following these guidelines ensures our codebase is inclusive and welcoming while maintaining technical clarity. The terminology accurately describes the functional roles of devices without relying on harmful historical language.

This section describes the key architectural patterns used throughout the STAR firmware codebase. These patterns enable loose coupling, testability, and maintainability in embedded systems.

5.136 Dependency Injection (DIP)

The Dependency Inversion Principle (DIP) is the foundational architectural pattern of the STAR framework. High-level modules depend on abstract interfaces rather than concrete implementations.

5.136.1 Interface Definition

Interfaces are defined as structs containing function pointers and an opaque context pointer.

```
/* From star_error_interface.h */
typedef struct star_error_interface {
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
         *message,
                               const char *file, int line, const char
                                  *func);
      bool (*can_retry)(void *ctx);
```

```
      rx_err_t (*reset_state)(void *ctx);
      void *ctx; /* Opaque context pointer */
} star_error_interface_t;

/* From star_pin_interface.h */
typedef struct star_pin_interface {
      rx_err_t (*register_pin)(void *ctx, int pin, const char
         *description,
                               bool shared);
      rx_err_t (*unregister_pin)(void *ctx, int pin, const char
         *description);
      rx_err_t (*validate)(void *ctx);
      void *ctx;
} star_pin_interface_t;
```

5.136.2 Interface Implementation

Concrete implementations provide the actual functionality.

```
/* Create error handler and get interface */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
error_handler_get_interface(&error_iface, &error_handler);

/* Implementation of get_interface */
void error_handler_get_interface(star_error_interface_t* iface,
error_handler_t* handler)
{
      iface->record_error = adapter_record_error; /* Adapter function */
      iface->can_retry = adapter_can_retry;
      iface->reset_state = adapter_reset_state;
      iface->ctx = handler; /* Concrete handler as context */
}
```

5.136.3 Dependency Injection Pattern

Components receive their dependencies through initialization functions.

```
/* Component declares interface dependencies */
typedef struct star_bus_manager {
      star_bus_config_t *buses;
      SemaphoreHandle_t mutex;
      const char *tag;
      star_error_interface_t *error_iface; /* Injected */
      star_pin_interface_t *pin_iface;     /* Injected */
      /* ... */
} star_bus_manager_t;

/* Initialization receives dependencies */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
```

```
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                         "Manager pointer is NULL");

      manager->buses = NULL;
      manager->error_iface = error_iface; /* Can be NULL */
      manager->pin_iface = pin_iface;     /* Can be NULL */
      /* ... */
}

/* Usage with injected dependencies */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
star_pin_interface_t pin_iface;
error_handler_get_interface(&error_iface, &error_handler);
pin_validator_get_interface(&pin_iface);

star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
```

5.136.4 NULL Interface Handling

Components gracefully handle NULL interfaces.

```
/* Check interface before use */
if (manager->error_iface && manager->error_iface->record_error) {
      manager->error_iface->record_error(manager->error_iface->ctx, ret,
                                         "Operation failed", __FILE__,
                                            __LINE__,
                                         __func__);
}

    /* Or use convenience macro */
#define STAR_IFACE_RECORD_ERROR(iface, err, msg)
   \\
      ((iface) && (iface)->record_error
                                                \\
           ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
                   \\
                                   __LINE__, __func__)
                                                          \\
           : (err))
```

5.137 Bus Abstraction Pattern

The bus abstraction provides a unified interface for different communication protocols.

5.137.1 Unified Configuration

```c
/* Single config type with protocol union */
typedef struct star_bus_config {
      const char *name;
      star_bus_type_t type; /* Discriminator */
      bool initialized;
      void *handle;
      void *user_ctx;

      union {
        star_i2c_bus_config_t i2c;
        star_spi_bus_config_t spi;
        star_gpio_bus_config_t gpio;
        star_adc_bus_config_t adc;
     } proto;

      struct star_bus_config *next;
} star_bus_config_t;
```

5.137.2 Factory Functions

Factory functions create configurations for specific protocols.

```c
/* I2C bus factory */
star_bus_config_t* star_bus_config_create_i2c(
const char* name,
i2c_port_t  port,
uint8_t     address,
gpio_num_t  sda_pin,
gpio_num_t  scl_pin,
uint32_t    clk_speed);

/* SPI device factory */
star_bus_config_t* star_bus_config_create_spi_device(
const char*                          name,
spi_host_device_t                    host,
gpio_num_t                           copi_pin,
gpio_num_t                           cipo_pin,
gpio_num_t                           sclk_pin,
int32_t                              dma_chan,
const spi_device_interface_config_t* dev_cfg);

/* GPIO bus factory */
star_bus_config_t* star_bus_config_create_gpio(
const char* name,
gpio_num_t* pins,
uint8_t     pin_count);
```

5.137.3 Manager Pattern

The bus manager coordinates multiple bus configurations.

```c
/* Initialize manager */
```

```
star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

/* Add buses */
star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
"sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22, 400000);
star_bus_manager_add_bus(&bus_manager, i2c_bus);

/* Find and use buses */
star_bus_config_t* bus = star_bus_manager_find_bus(&bus_manager,
   "sensor_i2c");

/* Safe access with callback (prevents use-after-free) */
rx_err_t my_callback(star_bus_config_t* bus, void* ctx) {
      /* Access bus safely while mutex is held */
      return RX_OK;
}
star_bus_manager_with_bus(&bus_manager, "sensor_i2c", my_callback, NULL);
```

5.138 Operations Structure Pattern

Group related function pointers for pluggable implementations.

```
/* Define operations structure */
typedef struct star_i2c_ops {
      star_i2c_write_fn_t write;
      star_i2c_read_fn_t read;
      star_i2c_write_command_fn_t write_command;
      star_i2c_read_raw_fn_t read_raw;
} star_i2c_ops_t;

/* Initialize with default implementations */
void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
{
      if (ops == NULL) {
        RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
        return;
     }
      ops->write = internal_star_bus_i2c_write;
      ops->read = internal_star_bus_i2c_read;
      ops->write_command = internal_star_bus_i2c_write_command;
      ops->read_raw = internal_star_bus_i2c_read_raw;
}

/* Use through function pointers */
rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
const char* name, ...)
{
      star_bus_config_t *bus_config = star_bus_manager_find_bus(manager,
         name);
      /* ... validation ... */
      return bus_config->proto.i2c.ops.write(bus_config, data, len,
         reg_addr,
                                             bytes_written);
```

```
}
```

5.139 Thread-Safe Callback Pattern

Safely access shared resources through callbacks.

```
/* Callback type */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
void* user_ctx);

/* Thread-safe access function */
rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
const char*         name,
star_bus_callback_t callback,
void*               user_ctx)
{
      ESP_RETURN_ON_FALSE(manager && name && callback, RX_ERR_INVALID_ARG,
                          s_TAG, "Invalid arguments");

      /* Acquire mutex */
      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      /* Find bus while holding mutex */
      star_bus_config_t *found_bus = NULL;
      for (star_bus_config_t *curr = manager->buses; curr; curr =
          curr->next) {
        if (curr->name && strcmp(curr->name, name) == 0) {
          found_bus = curr;
          break;
       }
     }

      if (found_bus == NULL) {
        xSemaphoreGive(manager->mutex);
        return ESP_ERR_NOT_FOUND;
     }

      /* Invoke callback while holding mutex - prevents use-after-free */
      rx_err_t callback_result = callback(found_bus, user_ctx);

      xSemaphoreGive(manager->mutex);
      return callback_result;
}
```

5.140 Thread Safety Patterns

5.140.1 Mutex Timeout Convention

Standard mutex timeout is 1000ms (configurable via Kconfig).

```
/* Standard mutex acquisition pattern */
if (xSemaphoreTake(manager->mutex,
pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
!= pdTRUE) {
      ESP_LOGE(manager->tag, "Timeout acquiring mutex");
      return ESP_ERR_TIMEOUT;
}

/* Critical section */
/* ... protected operations ... */

xSemaphoreGive(manager->mutex);
```

5.140.2 Mutex with Cleanup(goto pattern)

```
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t* config)
{
      RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                         "Invalid arguments");

      rx_err_t ret = RX_OK;

      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      /* Check for duplicate name */
      for (star_bus_config_t *curr = manager->buses; curr; curr =
          curr->next) {
        if (curr->name && strcmp(curr->name, config->name) == 0) {
          ret = RX_ERR_INVALID_STATE;
          goto add_give_mutex; /* Single exit point */
       }
     }

      /* Initialize bus hardware */
      ret = star_bus_config_init(config, manager);
      if (ret != RX_OK) {
        goto add_give_mutex;
     }

      /* Add to list */
      config->next = manager->buses;
      manager->buses = config;

```

```
    add_give_mutex:
      xSemaphoreGive(manager->mutex);
      return ret;
}
```

5.141 Error Handler Ownership Pattern

Components track whether they own their error handler for proper cleanup.

```
/* Sensor handle with ownership tracking */
typedef struct mpu6050_handle {
      star_bus_manager_t *bus_manager;
      const char *bus_name;
      star_error_interface_t *error_iface;
      bool owns_error_handler; /* Ownership flag */
      SemaphoreHandle_t mutex;
      bool initialized;
      /* ... */
} mpu6050_handle_t;

/* Initialization with optional interface */
rx_err_t star_sensor_mpu6050_init(mpu6050_handle_t* handle,
star_bus_manager_t* bus_manager,
const char* bus_name,
star_error_interface_t* error_iface,
const mpu6050_config_t* config)
{
      if (error_iface != NULL) {
        /* Use provided interface - we don't own it */
        handle->error_iface = error_iface;
        handle->owns_error_handler = false;
     } else {
        /* Create default - we own it */
        handle->error_iface = star_error_interface_create_default();
        handle->owns_error_handler = true;
     }
      /* ... */
}

/* Cleanup respects ownership */
rx_err_t star_sensor_mpu6050_deinit(mpu6050_handle_t* handle)
{
      /* Clean up error interface if we own it */
      star_error_interface_cleanup(handle->error_iface,
                                   handle->owns_error_handler);
      handle->error_iface = NULL;
      /* ... */
}

/* Helper for cleanup */
void star_error_interface_cleanup(star_error_interface_t* error_iface,
bool owns_handler)
{
      if (!owns_handler || !error_iface) {
```

```
        return; /* Not our responsibility */
    }
    error_handler_t *handler = (error_handler_t *)error_iface->ctx;
    error_handler_destroy_default(handler);
    free(error_iface);
}
```

5.142 Linked List Management

Bus configurations are managed as a singly-linked list.

```
/* Add to head of list (O(1)) */
config->next = manager->buses;
manager->buses = config;

/* Remove from list (O(n)) */
star_bus_config_t* prev = NULL;
for (star_bus_config_t* curr = manager->buses; curr; prev = curr, curr =
    curr->next) {
    if (strcmp(curr->name, name) == 0) {
        if (prev == NULL) {
            manager->buses = curr->next; /* Remove head */
        } else {
            prev->next = curr->next; /* Remove middle/tail */
        }
        curr->next = NULL;
        /* ... cleanup curr ... */
        break;
    }
}

/* Iterate through list */
for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
    /* Process each configuration */
}
```

5.143 Best Practices Summary

- **Use dependency injection**: Components receive interfaces, not concrete types.
- **Handle NULL interfaces**: Always check before dereferencing.
- **Track ownership**: Know who is responsible for cleanup.
- **Use callbacks for safe access**: Prevents use-after-free with shared resources.
- **Consistent mutex timeout**: 1000ms standard timeout.
- **Single exit point with goto**: Ensures mutex release on all paths.
- **Factory functions**: Create configurations with proper initialization.
- **Operations structures**: Enable pluggable implementations.

5.144 Prefer Alternatives to Macros

Rule: Avoid macros unless absolutely necessary. Use modern C alternatives:

- `static const` for constants
- `inline` functions for simple operations
- `enum` for related constants

Rationale: Macros lack type safety, scope control, and debugger visibility.
Good:

```
static const uint32_t k_max_buffer_size = 256;
static const float k_pi = 3.14159f;

static inline int max(int a, int b) {
      return (a > b) ? a : b;
}
```

Bad:

```
#define MAX_BUFFER_SIZE 256
#define PI 3.14159
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

5.145 When Macros Are Necessary

Use macros only for:

1. Array sizes (compile-time constants)
2. Conditional compilation (`#ifdef`, `#if`)
3. Token pasting and stringification
4. Platform-specific code paths

5.146 Macro Safety Rules

5.146.1 Wrap Statement - Like Macros in `do -while (0)`

Rule: Always wrap multi-statement macros in `do { ... } while (0)`.
Why: Prevents bugs when used in if-statements without braces.
Correct:

```
#define LOG_ERROR(msg)                                          \\
      do {                                                        \\
        log_timestamp();                                          \\
        log_message(msg);                                         \\
     } while (0)
```

Incorrect(causes bugs) :

```
#define LOG_ERROR(msg)                                                  \\
      log_timestamp();                                                      \\
      log_message(msg)

/* Bug example: */
if (error)
    LOG_ERROR("fail");  /* Only log_timestamp() is conditional! */
else
    handle_success();
```

5.146.2 Parenthesize All Macro Arguments

Rule: Wrap all argument uses and the entire expression in parentheses.

Correct:

```
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Incorrect(causes bugs) :

```
#define SQUARE(x) x *x

int result = SQUARE(2 + 3);  /* Expands to: 2 + 3 * 2 + 3 = 11, not 25! */
```

5.146.3 Beware of Side Effects and Multiple Evaluation

Rule: Document if macro arguments are evaluated multiple times. Never pass arguments with side effects.

Dangerous:

```
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int i = 5;
int result = MAX(i++, 10);  /* i is incremented TWICE! */
```

Solution: Use inline functions or add warning comment:

```
    /* WARNING: Arguments evaluated multiple times - no side effects! */
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

5.147 Multi - line Macro Formatting

Rules:

- Use backslash continuation aligned consistently
- Indent continuation lines for readability
- Place opening brace on same line as `do`

Example:

```
#define RECORD_ERROR(handler, code, desc)                                    \\
      do {                                                                       \\
        if ((handler)->error_iface != NULL) {                                    \\
          (handler)->error_iface->record_error((code), (desc), __FILE__,       \\
                                                __LINE__);                       \\
       }                                                                        \\
     } while (0)
```

5.148 Naming Conventions

Rule: Use `SCREAMING_SNAKE_CASE` for all macros.

Examples:

```
    /* Simple constants */
#define MAX_RETRY_COUNT (3)
#define MOTOR_PWM_FREQ_HZ (20000)

    /* Function-like macros (always use parentheses) */
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

    /* Multi-statement macros (always use do-while(0)) */
#define LOG_ERROR(msg)                                                       \\
      do {                                                                       \\
        log_timestamp();                                                         \\
        log_message(msg);                                                        \\
     } while (0)

#define RECORD_ERROR(handler, code, desc)                                    \\
      do {                                                                       \\
        error_handler_record_error((handler), (code), (desc), __FILE__,       \\
                                   __LINE__, __func__);                        \\
     } while (0)
```

5.149 Include Guards

Use traditional include guards for maximum portability:

```
#ifndef STAR_COMPONENT_FILENAME_H
#define STAR_COMPONENT_FILENAME_H

        /* File contents */

#endif /* STAR_COMPONENT_FILENAME_H */
```

Do not use `#pragma once` (see Section 12 for details).

5.150 Conditional Compilation

Best Practices:

- Document why conditionals exist
- Keep conditional blocks small
- Use configuration headers instead of scattered `#ifdef`

Good:

```
    /* Platform-specific GPIO initialization */
#if defined(CONFIG_STAR_BOARD_ESP32_S3)
    configure_esp32s3_gpio();
#elif defined(CONFIG_STAR_BOARD_ESP32_WROOM)
    configure_esp32_wroom_gpio();
#else
#error "Unsupported board configuration"
#endif
```

This section establishes the unit suffix naming standard used throughout the STAR firmware codebase. All numeric fields use SI (International System of Units) units with explicit suffixes to eliminate unit ambiguity and enable automatic documentation generation.

5.151 Overview

The firmware uses the MKS (Meter-Kilogram-Second) system as the foundation for all physical measurements. Every numeric variable that represents a physical quantity must include a unit suffix that clearly identifies its meaning.

5.152 Benefits

- **Eliminates Unit Ambiguity**: No confusion about whether a value is in meters or millimeters, degrees or radians
- **Self-Documenting Code**: Variable names explicitly state their units
- **Protocol Buffer Compatibility**: Matches naming conventions used in `star-proto/` messages
- **Prevents Conversion Errors**: Explicit units reduce the risk of unit mismatch bugs

5.153 Standard Unit Suffixes

Table 22: Standard Unit Suffixes for STAR Firmware

Suffix	Unit	Proto Example	C Example
_m	meters	x_m	distance_m
_mps	meters/second	velocity_mps	speed_mps
_mps2	m/s^2	accel_x_mps2	acceleration_mps2
_rad	radians	angle_rad	heading_rad
_rad_per_s	rad/second	omega_rad_per_s	angular_vel_rad_per_s
_celsius	degrees C	temperature_celsius	motor_temp_celsius
_deci_celsius	0.1°C	temp_deci_celsius	(BMS native format)
_us	microseconds	timestamp_us	elapsed_us
_ms	milliseconds	timeout_ms	period_ms
_ma	milliamps	current_ma	motor_current_ma
_mv	millivolts	cell_mv	pack_voltage_mv
_mw	milliwatts	power_mw	consumption_mw
_percent	0–100%	soc_percent	duty_cycle_percent

5.154 Usage Examples

5.154.1 Distance and Velocity

```
/* CORRECT - Distance and velocity with units */
float distance_m = 1.5f;            /* 1.5 meters */
float velocity_mps = 2.0f;          /* 2.0 meters per second */
float acceleration_mps2 = 9.81f;    /* 9.81 m/s^2 (gravity) */

/* Motor velocity tracking */
float target_velocity_mps = 0.5f;
float current_velocity_mps;
star_encoder_get_velocity_rpm(&encoder, 10.0f, 500,
   &current_velocity_mps);

/* WRONG - No unit suffix */
float distance = 1.5f;      /* What unit? Meters? Millimeters? Feet? */
float velocity = 2.0f;      /* Meters per second? RPM? */
```

5.154.2 Angular Measurements

```
/* CORRECT - Angular measurements in radians */
float heading_rad = 1.57f;              /* 90 degrees in radians */
float angular_velocity_rad_per_s = 3.14f; /* pi radians per second */

/* Robot pose */
typedef struct {
      float x_m;
      float y_m;
      float theta_rad; /* Heading angle */
```

```
} robot_pose_t;

/* WRONG - Ambiguous angle units */
float heading = 90.0f;  /* Degrees or radians? */
float omega = 3.14f;    /* What unit? */
```

5.154.3 Temperature

```
/* CORRECT - Temperature with units */
float motor_temp_celsius;
star_sensor_ds18b20_read_temp(&temp_sensor, &motor_temp_celsius);

/* BMS uses deci-celsius (0.1 degree resolution) */
int16_t cell_temp_deci_celsius = 235;  /* 23.5 degrees C */

    /* Thermal shutdown threshold */
#define MOTOR_THERMAL_SHUTDOWN_CELSIUS (85.0f)

if (motor_temp_celsius > MOTOR_THERMAL_SHUTDOWN_CELSIUS) {
      star_drv8243_stop(&motor, true); /* Emergency brake */
}

/* WRONG */
float motor_temp = 75.0f;  /* Celsius? Fahrenheit? Kelvin? */
```

5.154.4 Time

```
/* CORRECT - Time units */
uint32_t timeout_ms = 1000;         /* 1 second timeout */
uint64_t timestamp_us;              /* Microsecond timestamp */
star_encoder_get_timestamp_us(&encoder, &timestamp_us);

/* Timing for control loops */
const uint32_t control_period_ms = 1;     /* 1ms = 1000Hz */
const uint32_t sensor_poll_us = 10000;    /* 10ms in microseconds */

/* WRONG */
uint32_t timeout = 1000;    /* Milliseconds? Seconds? Ticks? */
uint64_t timestamp;         /* What resolution? */
```

5.154.5 Electrical Measurements

```
/* CORRECT - Electrical units */
uint32_t motor_current_ma;
star_drv8243_read_current(&motor, &motor_current_ma);

uint32_t cell_voltage_mv = 3700;   /* 3.7V in millivolts */
uint32_t pack_voltage_mv = 11100;  /* 11.1V (3S LiPo) */
uint32_t power_mw = 7400;          /* 7.4W in milliwatts */

```

```
    /* Current limiting */
#define MAX_MOTOR_CURRENT_MA (2000) /* 2A limit */

if (motor_current_ma > MAX_MOTOR_CURRENT_MA) {
      ESP_LOGW(s_TAG, "Motor current limit exceeded: %lu mA",
         motor_current_ma);
      star_drv8243_set_speed(&motor, 0.0f);
}

/* WRONG */
uint32_t motor_current = 1500;  /* Milliamps? Amps? */
uint32_t voltage = 3700;        /* Millivolts? Volts? */
```

5.154.6 Percentage

```
/* CORRECT - Percentage (0-100 range) */
float duty_cycle_percent = 75.0f;      /* 75% duty cycle */
uint8_t battery_soc_percent = 85;      /* 85% state of charge */
float throttle_percent = 50.0f;        /* 50% throttle */

star_drv8243_set_speed(&motor, duty_cycle_percent);

/* WRONG */
float duty_cycle = 0.75f;    /* Normalized (0-1)? Percentage (0-100)? */
uint8_t battery_soc = 85;    /* Percentage? Raw ADC value? */
```

5.155 Protocol Buffer Integration

The unit suffix convention matches the naming used in Protocol Buffer messages:

```
// From star-proto/proto/robot_command.proto
message VelocityCommand {
      float velocity_mps = 1;                // Meters per second
      float angular_velocity_rad_per_s = 2; // Radians per second
}

message BatteryState {
      repeated uint32 cell_mv = 1;   // Cell voltages in millivolts
      uint32 current_ma = 2;         // Pack current in milliamps
      uint8 soc_percent = 3;         // State of charge (0-100%)
      int16 temperature_celsius = 4; // Pack temperature
}
```

5.156 Conversion Functions

When converting between units, use descriptive function names:

```
/* CORRECT - Clear unit conversion */
float rpm_to_mps(float rpm, float wheel_diameter_m)
{
      float wheel_circumference_m = 3.14159f * wheel_diameter_m;
      float revolutions_per_second = rpm / 60.0f;
```

```
6       return revolutions_per_second * wheel_circumference_m;
7   }
8
9   float celsius_to_deci_celsius(float temp_celsius)
10  {
11      return temp_celsius * 10.0f;
12  }
13
14  float mv_to_volts(uint32_t voltage_mv)
15  {
16      return voltage_mv / 1000.0f;
17  }
18
19  /* WRONG - Ambiguous conversions */
20  float convert_rpm(float rpm);  /* Convert to what? */
21  float scale_temp(float temp);  /* What units? */
```

5.157 Summary

- **Always use unit suffixes** for physical quantities
- **Match Protocol Buffer conventions** for consistency
- **Use SI base units** when possible (meters, radians, seconds)
- **Document conversions** clearly in function names
- **Prefer millivolts/milliamps** over floating-point volts/amps for integer precision

This convention eliminates an entire class of bugs caused by unit ambiguity and makes the codebase more maintainable.

This section establishes memory management rules for STAR firmware across all platforms. Embedded platforms have limited RAM (e.g., RX72N: 1MB SRAM), making proper memory management critical for system stability and performance.

5.158 No Dynamic Allocation

Rule: Never use `malloc()`, `calloc()`, `realloc()`, or `free()` in production firmware.

5.158.1 Rationale

- **Heap Fragmentation**: Repeated allocation/deallocation causes memory fragmentation over time
- **Memory Leaks**: Forgotten `free()` calls lead to gradual RAM exhaustion
- **Unpredictable Timing**: Allocation time is non-deterministic (breaks real-time constraints)
- **Out-of-Memory Failures**: No recovery mechanism when heap is exhausted
- **Debugging Difficulty**: Memory corruption bugs are extremely hard to track down

5.158.2 Alternatives

Use static allocation instead:

```
/* WRONG - Dynamic allocation */
void process_sensor_data(void)
{
      float *samples = (float *)malloc(1000 * sizeof(float));
      if (samples == NULL) {
        RX_LOG_ERROR(s_TAG, "Out of memory!");
        return;
     }

      /* Process samples... */

      free(samples); /* Easy to forget! */
}

/* CORRECT - Static allocation */
static float s_samples[1000];  /* Allocated once at compile time */

void process_sensor_data(void)
{
      /* Use s_samples directly - no allocation, no leaks */
      for (int i = 0; i < 1000; i++) {
        s_samples[i] = read_sensor();
     }
}
```

5.158.3 Platform Heap Functions

Some platforms provide specialized heap functions (e.g., ESP-IDF's `heap_caps_malloc`) that are **only acceptable** for:

- **One-time initialization**: Allocating resources during startup that persist for the lifetime of the program
- **FreeRTOS task stacks**: Created once during task initialization
- **Third-party libraries**: When required by external dependencies (document clearly)

```
/* ACCEPTABLE - One-time allocation during init */
rx_err_t sensor_driver_init(sensor_handle_t* handle)
{
      /* Allocate persistent buffer once */
      handle->rx_buffer = heap_caps_malloc(DMA_BUFFER_SIZE,
          MALLOC_CAP_DMA);
      if (handle->rx_buffer == NULL) {
        return ESP_ERR_NO_MEM;
     }
      handle->owns_buffer = true; /* Track ownership */
      return RX_OK;
}
```

```c

/* Must have corresponding cleanup */
rx_err_t sensor_driver_deinit(sensor_handle_t* handle)
{
      if (handle->owns_buffer && handle->rx_buffer) {
        free(handle->rx_buffer);
        handle->rx_buffer = NULL;
     }
      return RX_OK;
}
```

5.159 Avoid Large Stack Allocations

Rule: Never allocate large arrays on the stack. Use static or global storage instead.

5.159.1 Stack Size Constraints

- Typical embedded task stack: **2KB–8KB** (platform and RTOS dependent)
- ISR stack: Even smaller (~1KB)
- Stack overflow causes silent corruption or hard faults

5.159.2 Examples

```c
/* WRONG - Large stack allocation */
void process_image(void)
{
      uint8_t frame_buffer[640 * 480]; /* 307KB on stack! */
      /* Stack overflow guaranteed! */
}

/* WRONG - Nested large allocations */
void motor_control_loop(void)
{
      float pid_state[100];           /* 400 bytes */
      uint32_t encoder_samples[500]; /* 2000 bytes */
      /* Total: 2400 bytes - may overflow 3KB stack */
}

/* CORRECT - Use static storage */
static uint8_t s_frame_buffer[640 * 480];  /* In .bss, not stack */

void process_image(void)
{
      /* Use s_frame_buffer safely */
      memset(s_frame_buffer, 0, sizeof(s_frame_buffer));
}

/* CORRECT - Small stack allocations are fine */
void calculate_checksum(const uint8_t* data, size_t len)
{
      uint32_t crc = 0; /* 4 bytes - perfectly fine on stack */
```

```
        /* ... */
}
```

5.159.3 Stack Usage Guidelines

Table 23: Stack Allocation Guidelines

Size	Allowed	Storage Location
$<$ 256 bytes	Yes	Stack (automatic)
256 bytes – 1 KB	Caution	Prefer static if persistent
$>$ 1 KB	No	Static or global only
$>$ 10 KB	Never	Static with explicit comment

5.160 Use `const` for Flash Storage

Rule: Place all read-only data in flash/ROM using the `const` keyword.

5.160.1 Rationale

- Embedded platforms typically have large flash but limited RAM (e.g., RX72N: 4MB flash / 1MB RAM)
- `const` data is stored in flash and accessed directly (no RAM copy)
- Saves precious SRAM for runtime variables

5.160.2 Examples

```
/* WRONG - Lookup table in RAM (wastes 256 bytes) */
static uint8_t sine_lookup[256] = {
  0, 3, 6, 9, 12, /* ... */
};

/* CORRECT - Lookup table in flash (zero RAM usage) */
static const uint8_t sine_lookup[256] = {
  0, 3, 6, 9, 12, /* ... */
};

/* WRONG - String literals without const */
static char* error_messages[] = {
  "Sensor timeout",
  "Invalid checksum",
  "Bus error"
};

/* CORRECT - Strings in flash */
static const char* const error_messages[] = {
  "Sensor timeout",
  "Invalid checksum",
  "Bus error"
};
```

```

/* CORRECT - Default configuration in flash */
static const star_pid_config_t default_pid_config = {
  .kp = 1.0f,
  .ki = 0.5f,
  .kd = 0.1f,
  .output_min = -100.0f,
  .output_max = 100.0f,
};

/* Usage: Copy to RAM only when needed */
star_pid_config_t runtime_config;
memcpy(&runtime_config, &default_pid_config, sizeof(runtime_config));
```

5.160.3 Pointer Declarations

Be careful with pointer `const` placement:

```
/* Pointer to const data (data in flash, pointer in RAM) */
const char* error_msg = "Timeout";

/* Const pointer to data (pointer in flash, data in RAM) */
char* const buffer_ptr = buffer;

/* Const pointer to const data (both in flash) */
const char* const error_table[] = {"Error 1", "Error 2"};
```

5.161 Validate All Buffers

Rule: Always validate buffer sizes before `memcpy()`, DMA operations, or ring buffer manipulations.

5.161.1 Rationale

- Buffer overruns cause memory corruption
- Corruption may manifest much later, making debugging extremely difficult
- Real-time systems have no memory protection (unlike Linux)

5.161.2 Validation Patterns

```
/* WRONG - Unchecked memcpy */
void receive_packet(uint8_t* data, size_t len)
{
      memcpy(rx_buffer, data, len); /* What if len > buffer size? */
}

    /* CORRECT - Validate before copy */
#define RX_BUFFER_SIZE (512)
static uint8_t rx_buffer[RX_BUFFER_SIZE];

void receive_packet(const uint8_t* data, size_t len)
```

```
{
      if (data == NULL || len > RX_BUFFER_SIZE) {
        RX_LOG_ERROR(s_TAG, "Invalid packet: len=%zu (max=%d)", len,
                     RX_BUFFER_SIZE);
        return;
     }
      memcpy(rx_buffer, data, len);
}

/* CORRECT - Clamp to buffer size */
void receive_packet_safe(const uint8_t* data, size_t len,
size_t* bytes_copied)
{
      size_t copy_len = (len < RX_BUFFER_SIZE) ? len : RX_BUFFER_SIZE;
      memcpy(rx_buffer, data, copy_len);
      *bytes_copied = copy_len;

      if (len > RX_BUFFER_SIZE) {
        RX_LOG_WARN(s_TAG, "Packet truncated: %zu bytes lost",
                    len - RX_BUFFER_SIZE);
     }
}
```

5.161.3 DMA Buffer Validation

```
/* DMA buffers must be validated AND aligned */
rx_err_t start_dma_transfer(const uint8_t* src, size_t len)
{
      /* Validate buffer pointer */
      if (src == NULL) {
        return RX_ERR_INVALID_ARG;
     }

      /* Validate length */
      if (len == 0 || len > MAX_DMA_TRANSFER_SIZE) {
        RX_LOG_ERROR(s_TAG, "Invalid DMA length: %zu", len);
        return ESP_ERR_INVALID_SIZE;
     }

      /* Check alignment (DMA requires 4-byte alignment) */
      if (((uintptr_t)src & 0x3) != 0) {
        RX_LOG_ERROR(s_TAG, "DMA buffer not 4-byte aligned");
        return RX_ERR_INVALID_ARG;
     }

      /* Proceed with DMA */
      return spi_device_transmit(spi_handle, &transaction);
}
```

5.161.4 Ring Buffer Validation

```c
/* CORRECT - Ring buffer with overflow protection */
typedef struct {
      uint8_t buffer[256];
      size_t head;
      size_t tail;
      size_t count;
} ring_buffer_t;

rx_err_t ring_buffer_write(ring_buffer_t* rb, const uint8_t* data,
size_t len)
{
      if (rb == NULL || data == NULL) {
        return RX_ERR_INVALID_ARG;
     }

      /* Check available space */
      size_t available = sizeof(rb->buffer) - rb->count;
      if (len > available) {
        RX_LOG_WARN(s_TAG, "Ring buffer overflow: %zu bytes lost",
                    len - available);
        return ESP_ERR_NO_MEM;
     }

      /* Safe to write */
      for (size_t i = 0; i < len; i++) {
        rb->buffer[rb->head] = data[i];
        rb->head = (rb->head + 1) % sizeof(rb->buffer);
        rb->count++;
     }

      return RX_OK;
}
```

5.162 Summary

Table 24: Memory Management Rules Summary

Rule	Implementation
No dynamic allocation	Use static/global buffers instead of malloc/free
Avoid large stack arrays	Use static storage for arrays $>$ 256 bytes
Use `const`	Place lookup tables, strings, defaults in flash
Validate all buffers	Check lengths before memcpy, DMA, ring buffers

Following these rules ensures stable, predictable memory usage and prevents an entire class of memory-related bugs that are extremely difficult to debug in embedded systems.

This section establishes best practices for real-time embedded systems development across all STAR platforms. These guidelines are critical for motor control (1000Hz loops), sensor fusion, and safety-critical robotics applications.

5.163 Interrupt Service Routines(ISRs)

5.163.1 Minimize ISR Work

Rule: Keep ISRs as short as possible. Set flags or queue work to background tasks instead of performing complex operations.

```
/* WRONG - Too much work in ISR */
static void IRAM_ATTR gpio_isr_handler(void* arg)
{
      /* Reading I2C sensor blocks for milliseconds! */
      float temperature = read_i2c_sensor();
      update_pid_controller(temperature);
      adjust_motor_speed();
      log_telemetry_to_sd_card(); /* File I/O in ISR! */
}

/* CORRECT - Minimal ISR, queue work to task */
static volatile bool s_encoder_event = false;
static QueueHandle_t s_encoder_queue;

static void IRAM_ATTR encoder_isr_handler(void* arg)
{
      /* Only set flag and queue event */
      s_encoder_event = true;

      encoder_event_t event = {.timestamp_us = esp_timer_get_time()};
      xQueueSendFromISR(s_encoder_queue, &event, NULL);

      /* ISR done in microseconds, not milliseconds */
}

/* Background task processes events */
void encoder_task(void* arg)
{
      encoder_event_t event;
      while (1) {
        if (xQueueReceive(s_encoder_queue, &event, portMAX_DELAY)) {
          /* Heavy processing here, not in ISR */
          process_encoder_event(&event);
       }
     }
}
```

5.163.2 ISR Constraints

- **No blocking operations**: No mutexes, delays, or I/O
- **Use `IRAM_ATTR`**: Places ISR in IRAM for faster execution
- **FromISR variants**: Use `xQueueSendFromISR()`, not `xQueueSend()`
- **Keep stack usage low**: ISR stack is typically 1KB or less
- **No floating-point**: Context save/restore is expensive

5.164 No Blocking Delays

Rule: Never use `vTaskDelay()` or busy-wait loops in time-critical code. Use event-driven design.

```c
/* WRONG - Blocking delay in motor control */
void motor_control_loop(void)
{
      while (1) {
        read_encoder();
        compute_pid();
        update_motor();

        vTaskDelay(pdMS_TO_TICKS(1)); /* Blocks, jitter from scheduler */
    }
}

/* CORRECT - Timer-driven execution */
static void motor_control_timer_callback(void* arg)
{
      /* Called exactly every 1ms by hardware timer */
      read_encoder();
      compute_pid();
      update_motor();
}

void setup_motor_control(void)
{
      esp_timer_create_args_t timer_args = {
          .callback = motor_control_timer_callback, .name = "motor_ctrl"};

      esp_timer_handle_t timer;
      esp_timer_create(&timer_args, &timer);
      esp_timer_start_periodic(timer, 1000); /* 1ms = 1000us */
}

/* CORRECT - Event-driven sensor reading */
void sensor_task(void* arg)
{
      while (1) {
        /* Wait for event, no polling */
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

        /* Event occurred, process it */
        process_sensor_data();
    }
}
```

5.165 Real - Time and Safety

Table 25: Real - Time Best Practices

Practice	Implementation
Know timing constraints	Profile worst-case execution times (WCET)
Use watchdog timers	Automatic reset on firmware lockup
Handle sensor timeouts	Never assume sensors always respond
Fail safe	On error, disable actuators and enter safe state
CRC critical data	Validate packets and stored configuration

5.165.1 Timing Constraints

```
/* Profile execution time using GPIO toggle */
void motor_control_loop(void)
{
      gpio_set_level(DEBUG_PIN, 1); /* Pin HIGH */

      /* Critical timing section */
      read_encoder();
      compute_pid();
      update_motor();

      gpio_set_level(DEBUG_PIN, 0); /* Pin LOW */

      /* Measure with oscilloscope:
       * Pulse width = worst-case execution time (WCET) */
}

/* Or use platform-specific cycle counter (ESP32, RX72N, etc.) */
uint32_t start_cycles = esp_cpu_get_cycle_count();

critical_function();

uint32_t end_cycles = esp_cpu_get_cycle_count();
uint32_t elapsed_us = (end_cycles - start_cycles) / (CPU_FREQ_MHZ);
RX_LOG_INFO(s_TAG, "Execution time: %lu us", elapsed_us);
```

5.165.2 Watchdog Timers

```
    /* Initialize task watchdog */
#include "esp_task_wdt.h"

void motor_control_task(void* arg)
{
      /* Subscribe to watchdog (5 second timeout) */
      esp_task_wdt_add(NULL);

      while (1) {
        /* Kick watchdog to prove we're alive */
```

```
        esp_task_wdt_reset();

        /* Control loop */
        motor_control_iteration();

        vTaskDelay(pdMS_TO_TICKS(1));
    }
}

/* If task hangs, watchdog resets system after timeout
   (platform-specific) */
```

5.165.3 Sensor Timeouts

```
/* WRONG - Assumes sensor always responds */
float read_temperature(void)
{
      send_i2c_command(TEMP_SENSOR_ADDR, READ_TEMP_CMD);
      return receive_i2c_data(TEMP_SENSOR_ADDR); /* Hangs if sensor fails
         */
}

/* CORRECT - Timeout and error handling */
rx_err_t read_temperature_safe(float* temp_celsius)
{
      rx_err_t ret;
      uint8_t data[2];

      /* I2C read with 100ms timeout */
      ret = star_bus_i2c_read(bus_manager, "temp_i2c", data, 2,
         TEMP_READ_REG,
                              NULL);
      if (ret != RX_OK) {
        RX_LOG_WARN(s_TAG, "Temperature sensor timeout");
        return ret;
    }

      *temp_celsius = (data[0] << 8 | data[1]) / 10.0f;
      return RX_OK;
}

/* Use sensor timeout for safety */
rx_err_t ret = read_temperature_safe(&motor_temp);
if (ret != RX_OK) {
      /* Sensor failed - enter safe state */
      star_drv8243_stop(&motor, true);
      system_enter_safe_mode();
}
```

5.165.4 Fail - Safe Design

```c
/* Safe state: all motors stopped, systems halted */
void system_enter_safe_mode(void)
{
      RX_LOG_ERROR(s_TAG, "ENTERING SAFE MODE");

      /* Stop all motors */
      for (int i = 0; i < NUM_MOTORS; i++) {
        star_drv8243_stop(&motors[i], true); /* Brake */
     }

      /* Disable all actuators */
      gpio_set_level(RELAY_ENABLE_PIN, 0);

      /* Set error LED */
      gpio_set_level(ERROR_LED_PIN, 1);

      /* Log error to flash */
      log_critical_error_to_nvs("Safe mode triggered");

      /* Infinite loop - require manual reset */
      while (1) {
        vTaskDelay(pdMS_TO_TICKS(100));
     }
}

/* Trigger safe mode on critical errors */
if (battery_voltage_mv < BATTERY_CRITICAL_MV) {
      system_enter_safe_mode();
}

if (motor_temp_celsius > THERMAL_SHUTDOWN_CELSIUS) {
      system_enter_safe_mode();
}
```

5.165.5 CRC Validation

```c
/* Validate critical configuration with CRC */
typedef struct {
      star_pid_config_t pid;
      uint32_t crc32; /* CRC of pid structure */
} stored_config_t;

rx_err_t save_config_to_nvs(const star_pid_config_t* config)
{
      stored_config_t stored;
      memcpy(&stored.pid, config, sizeof(star_pid_config_t));

      /* Calculate CRC-32 */
      stored.crc32 =
          esp_crc32_le(0, (uint8_t *)&stored.pid,
             sizeof(star_pid_config_t));

```

```
16      /* Save to NVS */
17      return nvs_set_blob(nvs_handle, "pid_config", &stored,
18                          sizeof(stored_config_t));
19  }
20
21  rx_err_t load_config_from_nvs(star_pid_config_t* config)
22  {
23      stored_config_t stored;
24      size_t len = sizeof(stored_config_t);
25
26      esp_err_t ret = nvs_get_blob(nvs_handle, "pid_config", &stored,
            &len);
27      if (ret != RX_OK) {
28        return ret;
29    }
30
31      /* Verify CRC */
32      uint32_t calculated_crc =
33          esp_crc32_le(0, (uint8_t *)&stored.pid,
                sizeof(star_pid_config_t));
34      if (calculated_crc != stored.crc32) {
35        RX_LOG_ERROR(s_TAG, "Config CRC mismatch! Flash corrupted?");
36        return ESP_ERR_INVALID_CRC;
37    }
38
39      memcpy(config, &stored.pid, sizeof(star_pid_config_t));
40      return RX_OK;
41  }
```

5.166 Hardware Abstraction

5.166.1 Use Abstract Bus Interfaces

```
1  /* CORRECT - Driver depends on bus abstraction, not platform HAL */
2  rx_err_t mpu6050_init(mpu6050_handle_t* handle,
3  star_bus_manager_t* bus_manager,
4  const char* bus_name)
5  {
6      handle->bus_manager = bus_manager;
7      handle->bus_name = bus_name;
8
9      /* Access I2C through bus manager */
10     uint8_t whoami;
11     esp_err_t ret = star_bus_i2c_read(bus_manager, bus_name, &whoami, 1,
12                                       MPU6050_WHO_AM_I, NULL);
13
14     /* Easy to mock for testing */
15     return ret;
16 }
```

5.166.2 Debounce Mechanical Inputs

```c
/* Software debounce for button */
typedef struct {
      gpio_num_t pin;
      bool state;
      uint32_t last_change_ms;
      uint32_t debounce_ms;
} debounced_button_t;

bool button_read(debounced_button_t* btn)
{
      bool raw_state = gpio_get_level(btn->pin);
      uint32_t now_ms = xTaskGetTickCount() * portTICK_PERIOD_MS;

      if (raw_state != btn->state) {
        /* State change detected */
        if (now_ms - btn->last_change_ms > btn->debounce_ms) {
          /* Stable for debounce period */
          btn->state = raw_state;
          btn->last_change_ms = now_ms;
       }
     }

      return btn->state;
}
```

5.166.3 Enable Brown - Out Detection

```c
/* In sdkconfig or menuconfig:
* CONFIG_ESP32_BROWNOUT_DET=y
* CONFIG_ESP32_BROWNOUT_DET_LVL_SEL_7=y  (3.08V threshold)
*/

/* ESP32 automatically resets on voltage dip below threshold */
/* Prevents flash corruption and erratic behavior */
```

5.166.4 Flash Wear - Leveling

```c
    /* Use NVS (Non-Volatile Storage) for wear-leveling */
#include "nvs_flash.h"

rx_err_t init_nvs(void)
{
      esp_err_t ret = nvs_flash_init();
      if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
          ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        /* Erase and retry */
        /* Platform-specific: handle flash erase error appropriately */
        nvs_flash_erase();
        ret = nvs_flash_init();
     }
      return ret;
```

```
}

/* NVS automatically rotates writes across sectors */
nvs_handle_t nvs_handle;
nvs_open("storage", NVS_READWRITE, &nvs_handle);
nvs_set_u32(nvs_handle, "boot_count", boot_count);
nvs_commit(nvs_handle);
```

5.166.5 Always Use Pull - Up / Pull - Down

```
/* WRONG - Floating input */
gpio_config_t io_conf = {
  .pin_bit_mask = (1ULL << GPIO_NUM_35),
  .mode = GPIO_MODE_INPUT,
  /* No pull-up or pull-down! Reads random values */
};

/* CORRECT - Enable pull-up */
gpio_config_t io_conf = {
  .pin_bit_mask = (1ULL << GPIO_NUM_35),
  .mode = GPIO_MODE_INPUT,
  .pull_up_en = GPIO_PULLUP_ENABLE,
  .pull_down_en = GPIO_PULLDOWN_DISABLE,
};
gpio_config(&io_conf);

/* For active-low signals (like nFAULT), use pull-up */
/* For active-high signals, use pull-down */
```

5.167 Performance and Real - Time

5.167.1 Benchmark, Don't Guess

```
    /* Use GPIO toggle to measure timing on oscilloscope */
#define BENCHMARK_START() gpio_set_level(BENCHMARK_PIN, 1)
#define BENCHMARK_END() gpio_set_level(BENCHMARK_PIN, 0)

          void
          pid_control_loop(void) {
        BENCHMARK_START();

        float velocity_rpm;
        star_encoder_get_velocity_rpm(&encoder, 1.0f, 500, &velocity_rpm);

        float output;
        star_pid_compute(&pid, setpoint, velocity_rpm, 0.001f, &output);

        star_motor_set_duty(&motor, output);

        BENCHMARK_END();
        /* Oscilloscope shows pulse width = execution time */
    }
```

5.167.2 Avoid Blocking Operations

```
          /* WRONG - Blocking I2C read in control loop */
          void
          control_loop(void) {
        while (1) {
          uint8_t data[6];
          i2c_master_read_from_device(I2C_NUM_0, SENSOR_ADDR, data, 6,
                                      100 / portTICK_PERIOD_MS);
          /* Blocks for up to 100ms! */

          process_data(data);
        }
      }

       /* CORRECT - Non-blocking with DMA */
       void setup_nonblocking_i2c(void) {
         /* Configure I2C interrupt + DMA */
         /* When data ready, ISR triggers */
         /* Background task processes results */
      }
```

5.168 Power Management

5.168.1 Use Sleep Modes

```
          /* Light sleep between sensor reads */
          void
          sensor_polling_task(void *arg) {
        while (1) {
          read_sensors();
          process_data();

          /* Sleep for 100ms instead of busy-wait */
          esp_sleep_enable_timer_wakeup(100000); /* 100ms in us */
          esp_light_sleep_start();
        }
      }
```

5.168.2 Use DMA to Reduce CPU Load

```
          /* DMA-enabled SPI transfer (CPU-free) */
          spi_device_transmit(spi_handle, &transaction);
       /* DMA handles transfer, CPU does other work */
```

5.169 Testing and Debugging

5.169.1 Use Mock Drivers

```
        /* Create mock bus for unit testing */
        star_bus_manager_t mock_bus_manager;
    star_bus_manager_init(&mock_bus_manager, "mock", NULL, NULL);

    /* Test driver without hardware */
    mpu6050_handle_t sensor;
    mpu6050_init(&sensor, &mock_bus_manager, "mock_i2c");
```

5.169.2 Meaningful Logging

```
        /* Include timestamp, module, and error codes */
        ESP_LOGE(s_TAG,
                 "[%lu] Motor fault detected: current=%lu mA (limit=%d
                    mA)",
                 esp_timer_get_time() / 1000, /* Timestamp in ms */
                 current_ma, MAX_MOTOR_CURRENT_MA);
```

5.170 Reliability and Fault Tolerance

5.170.1 Reset Peripherals if Stuck

```
        /* I2C bus recovery */
        rx_err_t
        recover_i2c_bus(i2c_port_t port) {
      ESP_LOGW(s_TAG, "Recovering I2C bus %d", port);

      /* Delete driver */
      i2c_driver_delete(port);

      /* Wait */
      vTaskDelay(pdMS_TO_TICKS(10));

      /* Re-initialize */
      i2c_config_t conf = {/* ... */};
      i2c_param_config(port, &conf);
      return i2c_driver_install(port, conf.mode, 0, 0, 0);
  }
```

5.171 Summary

Following these real - time and embedded best practices ensures :

- **Predictable timing** for motor control and sensor fusion
- **Robust error handling** for fault tolerance
- **Safe operation** in robotics applications
- **Maintainable code** through proper abstractions
- **Testability** with mock drivers and measurement tools

These practices are the foundation of reliable embedded systems for robotics.

This section establishes configuration requirements for Protocol Buffer messages used in STAR embedded firmware. The STAR project uses **nanopb** (Nano Protocol Buffers) for embedded systems, which requires static memory allocation.

5.172 nanopb Overview

nanopb is a small, portable Protocol Buffers implementation designed for embedded systems:

- *Static Allocation* : All buffers allocated at compile time(no malloc)
- *Small Code Size* : Minimal flash / RAM footprint
- *C Language* : Compatible with C99(optional C11 features; not C++)
- *Field Size Constraints* : Maximum sizes configured in `.options` files

5.173 Why Static Allocation ?

STAR firmware follows strict "no dynamic allocation" rules(see Section 25) .nanopb field sizes must be explicitly configured to enable compile - time buffer allocation .

5.174 Field Size Configuration

Field sizes are configured in `.options` files located alongside `.proto` files :

```
#File : star - proto / proto / star.options
            star.v1.RequestHeader.request_id max_size :
                64 star.v1.ResponseHeader.error_message
            max_size : 128 star.v1.FirmwareChunk.data max_size : 1024
```

5.175 Standard Field Size Constraints

Table 26: nanopb Field Size Constraints for Embedded Platforms

Field Type	Constraint	Example
String(UUID)	`max_size : 64`	`request_id`
String(error)	`max_size : 128`	`error _message`
String(version)	`max_size : 32`	`firmware_version`
Repeated(cells)	`max_count : 16`	`cell_mv[]`
Repeated(motors)	`max_count : 4`	`motor_status[]`
Bytes(firmware)	`max_size : 1024`	`chunk_data`

5.176 Examples

5.176.1 String Fields

```
    // File: robot_command.proto
    message RequestHeader {
  string request_id = 1; // UUID (36 chars + null terminator)
}

// File: robot_command.options
RequestHeader.request_id max_size : 64
```

Generated C code :

```
typedef struct _RequestHeader {
  char request_id[64]; /* Static allocation */
} RequestHeader;
```

5.176.2 Repeated Fields

```
    // File: battery_state.proto
    message BatteryState {
  repeated uint32 cell_mv = 1; // Cell voltages in millivolts
}

// File: battery_state.options
BatteryState.cell_mv max_count : 16
```

Generated C code :

```
typedef struct _BatteryState {
  uint32_t cell_mv[16];    /* Static array */
  pb_size_t cell_mv_count; /* Actual element count */
} BatteryState;
```

5.176.3 Bytes Fields

```
    // File: firmware_update.proto
    message FirmwareChunk {
  bytes data = 1; // Binary firmware data
}

// File: firmware_update.options
FirmwareChunk.data max_size : 1024
```

Generated C code :

```
typedef struct _FirmwareChunk {
  pb_bytes_array_t data; /* Contains uint8_t bytes[1024] */
} FirmwareChunk;
```

5.177 Sizing Guidelines

5.177.1 String Sizes

Table 27: Recommended String Field Sizes

Purpose	Size	Example
UUID(RFC 4122)	64 bytes	`request_id, session_id`
Error messages	128 bytes	`error _message`
Version strings	32 bytes	`firmware_version`
Short names	32 bytes	`component_name`
Log messages	256 bytes	`log_text`

5.177.2 Repeated Field Counts

Table 28: Recommended Repeated Field Counts

Purpose	Count	Example
Battery cells(3S LiPo)	3	`cell_mv[]`
Motors(quadruped)	4	`motor_status[]`
Motors(hexapod)	8	`motor_status[]`
Sensor array	8 –16	`temperature_sensors[]`
Telemetry samples	32 –64	`velocity_samples[]`

5.177.3 Bytes Field Sizes

Table 29: Recommended Bytes Field Sizes

Purpose	Size	Example
Firmware chunk	1024 bytes	`chunk_data`
Image thumbnail	4096 bytes	`thumbnail`
Small binary data	256 bytes	`calibration_blob`
Cryptographic key	64 bytes	`aes_key`

5.178 Memory Impact

5.178.1 Buffer Size Calculation

Each message allocates memory for the largest possible size:

```
/* Example: BatteryState with 3 cells (3S LiPo) */
typedef struct {
  uint32_t cell_mv[3];      /* 3 * 4 = 12 bytes */
  pb_size_t cell_mv_count; /* 4 bytes */
  uint32_t current_ma;      /* 4 bytes */
  uint8_t soc_percent;      /* 1 byte */
  /* Total: ~21 bytes + padding */
} BatteryState;

/* Even if only 1 cell used, full array is allocated */
```

5.178.2 RAM Usage Example

```
    /* Static allocation of message buffers */
    static RequestHeader s_request;     /* 64 bytes */
static ResponseHeader s_response;       /* 128 bytes */
static BatteryState s_battery_state;    /* 24 bytes */
static FirmwareChunk s_firmware_chunk; /* 1024 bytes */

/* Total: ~1240 bytes of static RAM */
```

5.179 Best Practices

5.179.1 Right - Size Your Buffers

- **Don't over-allocate**: `max_size:1024` for a 20-byte string wastes 1004 bytes
- **Don't under-allocate**: Truncated data causes silent bugs
- **Measure actual usage** : Profile message sizes in production

5.179.2 Use Fixed - Size Types

```
        /* CORRECT - Fixed-size integer types */
        message MotorCommand {
      int32 speed_percent = 1; // Always 4 bytes
      uint64 timestamp_us = 2; // Always 8 bytes
   }

    /* AVOID - Variable-length encoding */
    message MotorCommand {
      sint32 speed_percent = 1; // 1-5 bytes (varint encoding)
   }
```

5.179.3 Validate Message Sizes

```
        /* Check encoded message size before transmission */
        uint8_t buffer[256];
    pb_ostream_t stream = pb_ostream_from_buffer(buffer, sizeof(buffer));

    bool status = pb_encode(&stream, RequestHeader_fields, &request);
    if (!status) {
      RX_LOG_ERROR(s_TAG, "Encoding failed: %s", PB_GET_ERROR(&stream));
      return RX_ERR_FAIL;
   }

    size_t message_length = stream.bytes_written;
    RX_LOG_INFO(s_TAG, "Encoded message: %zu bytes", message_length);

    /* Verify it fits in SPI transaction buffer */
    if (message_length > SPI_MAX_TRANSFER_SIZE) {
      RX_LOG_ERROR(s_TAG, "Message too large for SPI: %zu > %d",
          message_length,
```

```
                    SPI_MAX_TRANSFER_SIZE);
        return ESP_ERR_INVALID_SIZE;
    }
```

5.180 Integration with Code Generation

5.180.1 buf.gen.yaml Configuration

```
#File : star - proto / buf.gen.yaml
        version : v2 plugins : -local : nanopb_generator out : gen /
                                                               nanopb opt
        : - --extension =.pb
```

5.180.2 Options File Naming

```
#Options file must match.proto file name
        proto /
        robot_command.proto->robot_command.options battery_state.proto
            ->battery_state.options firmware_update.proto->firmware_update
            .options
```

5.181 Common Pitfalls

5.181.1 Missing Options File

```
        /* ERROR: No .options file for string field */
        // robot_command.proto
        message Command {
      string name = 1; // No max_size specified
    }

     /* Generated code uses default (usually too small or error) */
```

Fix: Always create `.options` file for messages with strings, bytes, or repeated fields.

5.181.2 Mismatched Field Names

```
#WRONG - Field name mismatch
#robot_command.options
    Command.command_name max_size : 32 #Field is 'name',
    not 'command_name'

#CORRECT
        Command.name max_size : 32
```

5.181.3 Forgetting Array Count

```
    /* Common mistake: Forgetting to set count field */
    BatteryState battery = {0};
battery.cell_mv[0] = 3700;
battery.cell_mv[1] = 3720;
battery.cell_mv[2] = 3710;
/* BUG: battery.cell_mv_count not set! Encoder sends 0 cells */

/* CORRECT */
battery.cell_mv_count = 3; /* Set actual element count (3S LiPo) */
```

5.182 Summary

- **Always create.options files** for strings, bytes, repeated fields
- **Right-size buffers** based on actual usage (measure, don't guess)
- **Use fixed-size types** to avoid varint encoding overhead
- **Validate encoded sizes** before transmission
- **Set array count fields** for repeated fields

Properly configured nanopb constraints enable reliable, predictable Protocol Buffer communication on resource-constrained embedded hardware.

== == == == == == == == == == == == = =

5.183 NASA Power of 10: Rules for Safety-Critical Code

This section documents the NASA/JPL Power of 10 coding rules developed by Gerard J. Holzmann in 2006 for safety-critical embedded systems. These rules prioritize verification and predictability over coding flexibility.

STAR Compliance Philosophy: The STAR project follows the Power of 10 rules, with one **intentional architectural deviation** for Dependency Inversion Principle (DIP) – a conscious trade-off for testability and modularity.

5.183.1 Compliance Status Legend

Symbol	Meaning
✓ COMPLIANT	STAR follows this rule
△ !INTENTIONAL DEVIATION	Architectural choice(DIP pattern)

5.183.2 Rule 1 : Simplify Control Flow

Rule: Restrict all code to very simple control flow constructs – no `goto`, `setjmp`/`longjmp`, or recursion (direct or indirect).

Rationale: Creates predictable program structure for human understanding and static analysis. Recursion causes unbounded stack usage – dangerous in embedded systems.

STAR Compliance: ✓ COMPLIANT

```
    /* COMPLIANT - No goto, no recursion */
    esp_err_t
    star_motor_init(star_motor_handle_t *handle,
                    const star_motor_config_t *config) {
  if (handle == NULL || config == NULL) {
    return ESP_ERR_INVALID_ARG;
 }

  /* All control flow uses if/while/for */
  esp_err_t ret = configure_timer(handle, config);
  if (ret != ESP_OK) {
    return ret;
 }

  return configure_gpio(handle, config);
}
```

5.183.3 Rule 2 : Fixed Loop Upper - Bounds

Rule : All loops must have a fixed upper bound that static analysis tools can trivially prove.

Rationale : Prevents runaway code, guarantees termination, enables real - time deadline verification.

STAR Compliance: ✓ **COMPLIANT**

```
/* COMPLIANT - Fixed iteration count */
#define MAX_RETRIES (3)

    for (uint8_t i = 0; i < MAX_RETRIES; i++) {
  if (sensor_read() == ESP_OK) {
    break;
 }
  vTaskDelay(pdMS_TO_TICKS(100));
}

/* COMPLIANT - Array with known size */
#define BQ7850_MAX_CELLS (16)

for (uint8_t cell = 0; cell < BQ7850_MAX_CELLS; cell++) {
  read_cell_voltage(cell, &voltages[cell]);
}
```

5.183.4 Rule 3 : No Dynamic Memory After Initialization

Rule : Prohibit `malloc` /`free` after initialization phase.

Rationale : Dynamic allocation causes unpredictable performance, memory leaks, fragmentation, and corruption.

STAR Compliance: ✓ **COMPLIANT**

Best Practice : Use pre - allocated static pools instead of runtime memory requests.

```
/* COMPLIANT - Static allocation with fixed limits */
#define MAX_ITEMS (8)
```

```
#define MAX_DESC_LEN (64)

    typedef struct {
  char items[MAX_ITEMS][MAX_DESC_LEN]; /* Static 2D array */
  uint8_t item_count;
} static_pool_t;

/* Usage */
if (pool->item_count >= MAX_ITEMS) {
  return ESP_ERR_NO_MEM;
}
strncpy(pool->items[pool->item_count], desc, MAX_DESC_LEN - 1);
pool->items[pool->item_count][MAX_DESC_LEN - 1] = '\0';
pool->item_count++;
```

5.183.5 Rule 4 : Keep Functions Short

Rule : Functions should not exceed ~60 lines(one printed page, one statement per line) .

Rationale : Represents single verifiable logical units; improves comprehension, review, and testability.

STAR Compliance: ✓ **MOSTLY COMPLIANT**

```
    /* COMPLIANT - Single responsibility, short function */
    esp_err_t
    star_drv8243_read_current(star_drv8243_handle_t *handle,
                              float *current_ma) {
  if (handle == NULL || current_ma == NULL || !handle->initialized) {
    return ESP_ERR_INVALID_ARG;
 }

  int raw_value = 0;
  esp_err_t ret = read_adc_channel(handle, &raw_value);
  if (ret != ESP_OK) {
    return ret;
 }

  *current_ma = (float)raw_value * handle->ki_propi / 1000.0f;
  return ESP_OK;
}
```

5.183.6 Rule 5 : Use Assertions Generously

Rule : Minimum two assertions per function; assertions must be side - effect - free.

Rationale : Acts as executable documentation verifying pre - conditions, post - conditions, and invariants.

STAR Compliance: ✓ **COMPLIANT**

Note: STAR uses explicit parameter validation instead of assertions for production code (assertions disabled in release builds).

```
/* COMPLIANT - Explicit validation (better than assert for production) */
esp_err_t star_pid_compute(star_pid_handle_t* handle, float setpoint,
                           float measurement, float dt, float* output)
```

```
{
  /* Pre-condition checks (effectively runtime assertions) */
  if (handle == NULL) {
    return ESP_ERR_INVALID_ARG;
 }
  if (!handle->initialized) {
    return ESP_ERR_INVALID_STATE;
 }
  if (output == NULL) {
    return ESP_ERR_INVALID_ARG;
 }
  if (dt <= 0.0f || dt > 1.0f) { /* Sanity check time delta */
    return ESP_ERR_INVALID_ARG;
 }

  /* Function logic */
  float error = setpoint - measurement;
  handle->integral += error * dt;

  /* Post-condition: integral within bounds */
  if (handle->integral > handle->integral_max) {
    handle->integral = handle->integral_max;
 } else if (handle->integral < handle->integral_min) {
    handle->integral = handle->integral_min;
 }

  *output = handle->kp * error + handle->ki * handle->integral;
  return ESP_OK;
}
```

5.183.7 Rule 6 : Declare Data at Smallest Scope

Rule : Minimize variable scope to reduce accidental corruption risks.

Rationale : Limits chances of accidental reference or corruption from other code.

STAR Compliance: ✓ **COMPLIANT**

```
    /* COMPLIANT - Variables declared at minimal scope */
    void
    process_sensors(void) {
  /* Loop variable declared in for statement */
  for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
    /* Temporary variable for single sensor */
    float temp_c = 0.0f;
    if (read_sensor(i, &temp_c) == ESP_OK) {
      /* Process immediately, temp_c out of scope after block */
      update_telemetry(i, temp_c);
   }
 }
}
```

5.183.8 Rule 7 : Check All Return Values and Parameters

Rule : Validate all function returns and input parameters; treat warnings as errors.

Rationale : Forces explicit failure - condition handling rather than ignoring error codes.

STAR Compliance: ✓ **COMPLIANT**

```
    /* COMPLIANT - All returns checked, all parameters validated */
    esp_err_t
    star_bus_manager_add_bus(star_bus_manager_t *manager,
                             star_bus_config_t *config) {
  /* Parameter validation */
  if (manager == NULL || config == NULL) {
    return ESP_ERR_INVALID_ARG;
 }

  /* Check return value and propagate errors */
  esp_err_t ret = star_bus_config_init(config);
  if (ret != ESP_OK) {
    ESP_LOGE(TAG, "Bus initialization failed: %s", esp_err_to_name(ret));
    return ret;
 }

  /* Mutex operation checked */
  if (xSemaphoreTake(manager->mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
    ESP_LOGE(TAG, "Failed to acquire mutex");
    return ESP_ERR_TIMEOUT;
 }

  /* Critical section */
  add_to_list(manager, config);
  xSemaphoreGive(manager->mutex);

  return ESP_OK;
}
```

5.183.9 Rule 8 : Limit Preprocessor Use

Rule : Restrict to header inclusion and simple macros; forbid token pasting, recursive macros, and incomplete syntactic units.

Rationale : Preprocessor complexity obscures operations, fooling developers and static analysis tools.

STAR Compliance: ✓ **COMPLIANT**

```
/* COMPLIANT - Simple constant macros */
#define MAX_RETRY_COUNT (3)
#define MOTOR_PWM_FREQ_HZ (20000)
#define TIMEOUT_MS (1000)

    /* COMPLIANT - Simple inline functions (better than complex macros) */
    static inline float
    voltage_mv_to_v(uint16_t mv) {
  return (float)mv / 1000.0f;
}

/* AVOID - Complex token-pasting macros */
// #define CREATE_HANDLER(name) handler_##name##_init()  /* Too complex */
```

5.183.10 Rule 9 : Restrict Pointer Use

Rule : Maximum one dereferencing level(`*ptr` OK, `**ptr` forbidden); no function pointers.

Rationale : Multiple indirection and function pointers make static data - flow analysis impossible.

STAR Compliance: △ !INTENTIONAL DEVIATION

Intentional Architectural Deviation: STAR uses function pointers for **Dependency Inversion Principle(DIP)** – an industry-standard pattern for testability and modularity. This is a conscious trade-off for better architecture.

DIP Pattern(Intentional Function Pointer Use) :

```
    /* INTENTIONAL DEVIATION - DIP interface with function pointers */
    typedef struct star_error_interface {
  esp_err_t (*record_error)(void *ctx, esp_err_t error, const char
     *message,
                            const char *file, int line, const char *func);
  bool (*can_retry)(void *ctx);
  esp_err_t (*reset_state)(void *ctx);
  void *ctx; /* Implementation context */
} star_error_interface_t;

/* Why we keep this:
 * 1. Enables mock implementations for unit testing
 * 2. Allows swapping error handlers without recompilation
 * 3. Follows SOLID principles (Dependency Inversion)
 * 4. Industry-standard pattern (used in Linux kernel, RTOS, etc.)
 */
```

5.183.11 Rule 10 : Compile with Maximum Warnings

Rule : Enable all compiler warnings at highest pedantry; achieve zero - warning builds with static analyzers .

Rationale : Modern compilers catch bugs before runtime.Zero warnings, no excuses.

STAR Compliance: ✓ COMPLIANT

```
#platformio.ini - Compiler flags
    build_flags = -Wall #Enable all warnings - Wextra #Extra warnings -
                  Werror #Treat warnings as errors - Wno - unused -
                  parameter #Allow unused params in interface
                     implementations -
                  Wno - missing - field - initializers
```

CI / CD Enforcement : Build fails on any compiler warning.

5.183.12 Summary: STAR Compliance Matrix

Table 30: NASA Power of 10 Compliance Status

Rule	Description	STAR Status
1	No `goto` / recursion	✓ COMPLIANT
2	Fixed loop bounds	✓ COMPLIANT
3	No dynamic memory	✓ COMPLIANT
4	Short functions(<60 lines)	✓ COMPLIANT
5	Use assertions	✓ COMPLIANT
6	Minimal variable scope	✓ COMPLIANT
7	Check all returns	✓ COMPLIANT
8	Limit preprocessor	✓ COMPLIANT
9	Restrict pointers	△ !INTENTIONAL(DIP)
10	Maximum warnings	✓ COMPLIANT

5.183.13 Defensive Coding Measures

Beyond the Power of 10 rules, STAR implements additional defensive coding measures for electrical noise resilience in the RX72N motor controller firmware.

Independent Watchdog Timer(IWDT) :

- 120 kHz dedicated oscillator (independent of main clock)
- 1 second timeout, fed every 4ms from 250Hz control loop
- Detects infinite loops, deadlocks, and system hangs
- Logs reset cause on startup for diagnostics

Register Guard(ESD / EMI Protection) :

- Captures “golden” register values after initialization
- Periodically refreshes critical registers (PORT PDR, MSTPCR)
- Recovers from transient bit-flips caused by electrical noise
- Tracks correction count for field diagnostics

IRQ Digital Filtering:

- Hardware noise rejection on interrupt pins
- Configurable filter clock divisor(PCLK / 1 to PCLK / 64)
- 3 - sample debounce prevents spurious interrupts

```
    /* Defensive coding in motor control loop */
    while (true) {
  /* ... control logic ... */

  rx_iwdt_feed(); /* Feed watchdog every 4ms */

  if (++guard_counter >= 250) { /* Every 1 second */
    rx_register_guard_refresh();
    guard_counter = 0;
 }
}
```

5.183.14 References

- Holzmann, Gerard J. *"The Power of 10: Rules for Developing Safety-Critical Code."* IEEE Computer, 39(6), pp. 95-97, June 2006.
- JPL Institutional Coding Standard for the C Programming Language (JPL DOCID D-60411)
- MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems

== == == == == == == == == == == == = =

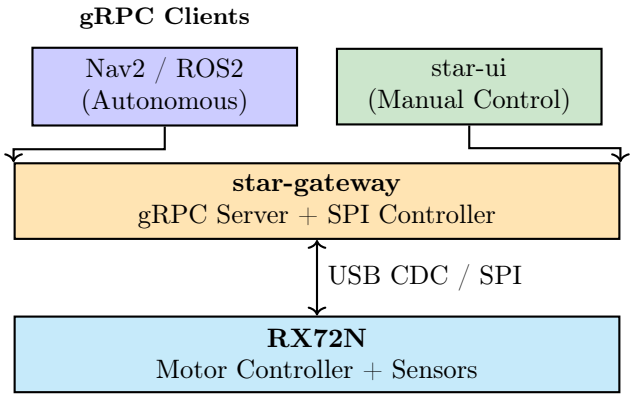
6 Gateway Architecture

This section documents the `star-gateway` Go service that bridges the Raspberry Pi 5 control stack with the RX72N motor controller.

6.1 Overview

The gateway service runs on the Raspberry Pi 5 and serves as the communication bridge between:

- **High-level side:** gRPC clients (Nav2/ROS2 for autonomous navigation, UI for manual control)
- **Low-level side:** RX72N motor controller via USB CDC (primary) or SPI (backup)



6.2 Protocol Stack Implementation

The gateway implements all layers of the communication protocol(see Section **??**) .

Table 31: Gateway Protocol Stack Implementation

Layer	Name	Go Package	Status
5	Application	`internal/service/`	Implemented
4	Serialization	`star-proto/gen/go/`	Implemented
3	HARQ + FEC	`internal/link/, internal/harq/, internal/fec/`	Implemented
2	Framing	`internal/frame/`	Implemented
1	Transport	`internal/transport/`	Implemented

6.3 Directory Structure

```
star - gateway / +--cmd / | +--star - gateway / | |
   +--main.go #Entry point | +--virtual_rx72n / |
   +--main.go #RX72N simulator + --internal / |
   +--transport / #Layer 1 : Physical transport | |
   +--spi.go #SPI transport(periph.io) | | +--cdc.go #USB CDC transport | |
   +--socket.go #Socket transport(simulation) |
   +--frame / #Layer 2 : Frame protocol | | +--frame.go #Constants and types |
   | +--encoder.go #Frame encoder | | +--decoder.go #Stream decoder |
   +--link / #Layer 3 : Reliability(HARQ) | | +--spi.go #SPILink(HARQ) |
   +--harq / #HARQ interface and types | | +--harq.go | +--fec / #FEC codec | |
```

```
+--convolutional.go #Rate 1 / 2,
K = 7 encoder | | +--viterbi.go #Viterbi decoder | |
    +--combiner.go #Chase Combiner | +--manager / #Transport management | |
    +--manager.go #TransportManager + SessionState |
    +--service / #Layer 5 : gRPC services | | +--motor_control.go | |
    +--telemetry.go | | +--battery.go | | +--configuration.go | |
    +--gateway.go #UI bridge service | +--app / #Application wiring | |
    +--gateway.go | +--server / #gRPC server |
    +--server.go + --go.mod + --CLAUDE.md
```

6.4 Frame Package

The `internal / frame` package defines the wire protocol constants and types.

6.4.1 Frame Constants

Table 32: Frame Protocol Constants

Constant	Value	Description
`SyncWord`	`0x55AA`	Frame sync marker(alternating bits : `[0x55, 0xAA]`)
`SyncSize`	2 bytes	Sync word size
`HeaderSize`	6 bytes	SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1)
`CRCSize`	4 bytes	CRC - 32 checksum
`MaxPayloadSize`	1024 bytes	Maximum protobuf payload
`MaxFrameSize`	1036 bytes	Total frame size
`MinFrameSize`	12 bytes	Empty payload frame

6.4.2 Frame Types

- `FrameTypeCommand` –Command from controller to peripheral
- `FrameTypeResponse` –Response from peripheral to controller
- `FrameTypeAck` –Acknowledgment of successful receipt
- `FrameTypeNack` –Negative acknowledgment(CRC error, etc.)

6.5 Transport Package

The `internal/transport` package provides physical transport implementations for communicating with the RX72N.

6.5.1 Available Transports

Table 33: Transport Implementations

Transport	Device	Speed	Usage
SPI	`/dev/spidev0.0`	10 MHz	Backup(priority 5)
USB CDC	`/dev/ttyACM0`	12 Mbps(Full - Speed)	Primary(priority 10)
Socket	Unix socket	N / A	Simulation mode

6.5.2 SPI Configuration

Table 34: SPI Configuration Constants

Parameter	Value	Description
Device	`/dev/spidev0.0`	SPI device path
Speed	10 MHz	SPI clock frequency
Mode	0	CPOL = 0, CPHA = 0
Bits per word	8	Word size
Timeout	100 ms	Default operation timeout

6.5.3 Transport Interface

```
type Transport interface {
 Send(data[] byte)(int, error) Receive(maxLen int)([] byte, error)
     Transfer(txData[] byte)([] byte, error) Close() error
}
```

6.6 Link Layer (HARQ)

The `internal/link` package implements the reliability layer with two link types optimized for their respective transports.

6.6.1 SPILink

`SPILink` provides full Hybrid ARQ(HARQ) with forward error correction for the SPI transport, where the physical channel is more error-prone.

- **HARQ with ACK / NACK** –Retransmission on negative acknowledgment or timeout
- **FEC encoding / decoding** –Convolutional code(K = 7, Rate 1 / 2) with Viterbi decoding
- **Chase Combining** – Soft-combining of retransmitted frames for improved error recovery

6.6.2 CDCLink

`CDCLink` provides a lightweight link layer for USB CDC, since USB handles reliability at the hardware level.

- **No HARQ** –USB protocol provides its own error correction and retransmission
- **Sequence tracking** – Maintains sequence numbers for ordering

6.6.3 Shared Session State

Both link types use the shared `SessionState` from `internal/manager/` to maintain sequence number continuity across transport failovers.

6.6.4 HARQ Parameters

Table 35: HARQ Protocol Parameters

Parameter	Value	Description
Max Retries	3	Maximum transmission attempts
ACK Timeout	10 ms	ACK wait timeout
Sequence Max	0xFFFF	16 - bit wraparound
FEC Codec	Convolutional K = 7, Rate 1 / 2	Forward error correction

6.7 Transport Manager

The `internal / manager` package provides priority - based transport selection and automatic failover.

6.7.1 Transport Priorities

Table 36: Transport Manager Priority Configuration

Transport	Priority	Role
USB CDC	10	Primary transport
SPI	5	Backup transport

6.7.2 Failover Behavior

- **Shared SessionState** –Sequence continuity maintained across transports
- **Automatic failover** –Health monitoring detects transport failures
- **Primary timeout** –200 ms implicit timeout before failover
- **Secondary probe** –1 s PING probe to detect recovery of higher - priority transport

6.8 gRPC Services

The gateway exposes five gRPC services (see Section **??** for detailed specifications):

Table 37: Gateway gRPC Services

Service	Description
MotorControlService	Motor velocity commands, emergency stop, and control loop configuration
TelemetryService	Real-time sensor data streaming (encoders, IMU, battery)
BatteryManagementService	Battery monitoring, state of charge, and safety limits
ConfigurationService	Runtime parameter updates and system configuration
FirmwareUpdateService	Over-the-air firmware update for RX72N controller

6.9 Build and Test

```
#Build the gateway binary
    cd star -
    gateway go build./ cmd / star -
    gateway

#Run all tests
            go test./
        ...

#Run with verbose output
        go test -
    v./
        ...

#Format code
        go fmt./
        ...

#Static analysis
        go vet./
        ...
```

6.10 Dependencies

The gateway depends on :

- `github.com / Locked - Inc / star - proto / gen / go` –Generated protobuf and gRPC code
- `google.golang.org / grpc` –gRPC framework
- `google.golang.org / protobuf` – Protocol Buffers runtime

The `go.work` file at the repository root enables workspace mode for seamless cross-module development:

```
go 1.21

use (
    ./star-proto/gen/go
    ./star-proto/tests/go
    ./star-gateway
)
```

For the complete end - to - end communication path from UI to motor controller, see Section **??**.

7 Yocto Build System

This section documents the Yocto-based build system for the STAR robot's Raspberry Pi 5 controller.

7.1 Overview

The STAR robot uses the Yocto Project (Scarthgap release) with OpenEmbedded to build a custom Linux distribution for the Raspberry Pi 5. This approach provides:

- Full control over the root filesystem
- ROS2 Jazzy integration via meta-ros
- Custom recipes for STAR-specific packages
- Reproducible builds

7.2 Layer Structure

The build uses the following Yocto layers :

Layer	Purpose
poky/meta	Core OpenEmbedded layer
poky/meta-poky	Poky reference distribution
poky/meta-yocto-bsp	Yocto BSP layer
meta-raspberrypi	Raspberry Pi 5 BSP
meta-openembedded/meta-oe	OpenEmbedded utilities
meta-openembedded/meta-python	Python packages
meta-openembedded/meta-networking	Networking packages
meta-openembedded/meta-multimedia	Multimedia packages
meta-ros/meta-ros-common	ROS common components
meta-ros/meta-ros2	ROS2 core
meta-ros/meta-ros2-jazzy	ROS2 Jazzy distribution
meta-star	STAR custom recipes

Table 38: Yocto layers used in the STAR build

7.3 meta - star Layer

The `meta - star` layer contains STAR - specific recipes :

7.3.1 recipes - core

- `network - config` – Static IP configuration for eth0 (192.168.100.2/24)
- `wifi - setup` – WiFi configuration script for easy network setup
- `locale - config` –UTF - 8 locale configuration with automatic generation at boot
- `modules - autoload` –Kernel module auto - loading(i2c - dev)
- `ros2 - env` –ROS2 environment auto - sourcing

7.3.2 recipes - devtools

- `temurin - jdk - bin` – Eclipse Temurin JDK 21 for Java/Kotlin development

7.4 Hardware Configuration

The following hardware interfaces are enabled via `config.txt` :

Interface	Setting	Device
UART	dtoverlay = uart0 - pi5	/ dev / ttyAMA0
I2C	dtparam = i2c_arm = on, i2c1 = on	/ dev / i2c - 1
SPI	dtparam = spi = on	/ dev / spidev0 .0, 0.1
GPIO	pinctrl - rp1	gpiochip0(54 lines)
Camera / Display	camera_auto_detect = 1, display_auto_detect = 1	Auto
USB Power	usb_max_current_enable = 1	5A mode

Table 39: Hardware interface configuration

7.5 Installed Packages

The image includes the following key packages :

7.5.1 ROS2 Jazzy

- ros - core, rclcpp, rclpy
- std - msgs, sensor - msgs, geometry - msgs, nav - msgs

7.5.2 Developer Tools

- vim 9.1(default editor)
- git 2.44.4
- htop 3.3.0
- tmux 3.3a
- Go 1.22.12
- Eclipse Temurin JDK 21.0.5

7.5.3 Hardware Tools

- i2c - tools(i2cdetect, i2cget, i2cset)
- spitools(spi - pipe, spi - config)
- libgpiod - tools(gpiodetect, gpioinfo, gpioget, gpioset)

7.5.4 Computer Vision

- OpenCV
- Python3 with NumPy
- v4l - utils

7.6 Building the Image

7.6.1 Prerequisites

Install required packages on Ubuntu :

```
 sudo apt install gawk wget git diffstat unzip texinfo gcc
            build -
         essential chrpath socat cpio python3 python3 - pip python3 -
         pexpect xz - utils debianutils iputils - ping python3 - git python3 -
         jinja2 python3 - subunit zstd liblz4 -
         tool file locales libacl1 bmaptool
```

7.6.2 Setup and Build

```
 cd / path / to / star - yocto source poky / oe - init -
         build - env build -
         rpi5

#Copy configuration from star - yocto - config
                 cp../
             star -
         yocto - config / conf/* conf/
ln -s ../star-yocto-config/meta-star .

bitbake core-image-minimal
```

7.6.3 Flashing

Use bmaptool for fast, verified flashing:

```
sudo bmaptool copy\\
    tmp/deploy/images/raspberrypi5/core-image-minimal-raspberrypi5.rootfs.wic.gz\\
    /dev/sdX
```

7.7 WiFi Configuration

The image includes a `wifi-setup` script for easy WiFi configuration:

```
# Interactive mode
wifi-setup

# Direct mode
wifi-setup <SSID> <password>

# Check status
wifi-setup --status
```

7.8 Package Management

The image uses opkg for package management. A local package feed can be set up:

```
# On host: serve packages
cd tmp/deploy/ipk
python3 -m http.server 8080

# On Pi: configure feed
echo "src/gz all http://<host-ip>:8080/all" > /etc/opkg/feed.conf
opkg update
opkg install <package>
```

8 USB CDC Protocol

8.1 Overview

The RX72N firmware implements a USB CDC - ACM(Communications Device Class - Abstract Control Model) composite device with three independent virtual serial ports.This enables simultaneous binary protocol communication , ASCII debugging, and log streaming over a single USB connection.

8.1.1 Port Assignment

Port	Purpose	Linux Device	RX(B)	TX(B)	Usage
0	Protocol	/dev/ttyACM0	1024	1024	nanopb frames, motor commands
1	Decoded	/dev/ttyACM1	256	512	ASCII frame dumps, debugging
2	Log	/dev/ttyACM2	256	1024	System logging, diagnostics

Table 40: USB CDC Port Configuration

8.1.2 Hardware Configuration

- **USB Speed** : Full - Speed(12 Mbps)
- **Max Packet Size** : 64 bytes(bulk endpoints)
- **Pipes Used** : 9 pipes total(1 control + 3 CDC x (1 interrupt + 2 bulk))
- **Pins** : USB_DP(PA4), USB_DM(PA5), USB_VBUS(PA6)
- **Clock** : 48 MHz USB clock from PLL

8.2 USB Descriptor Hierarchy

8.2.1 Device Descriptor

```
{
  .bLength = 18, .bDescriptorType = USB_DEVICE_DESCRIPTOR,
  .bcdUSB = 0x0200,             // USB 2.0
      .bDeviceClass = 0xEF,     // Miscellaneous (IAD)
      .bDeviceSubClass = 0x02, // Common Class
      .bDeviceProtocol = 0x01, // Interface Association
      .bMaxPacketSize0 = 64,
  .idVendor = 0x045B,       // Renesas
      .idProduct = 0x0235, // RX72N CDC
      .bcdDevice = 0x0100, // Device v1.0
      .iManufacturer = 1,  // "STAR Project"
      .iProduct = 2,       // "RX72N CDC Composite"
      .iSerialNumber = 3,  // "RX72N-00001"
      .bNumConfigurations = 1
}
```

Listing 1: USB Device Descriptor

8.2.2 Configuration Descriptor

The configuration descriptor contains :

- 3 x Interface Association Descriptors (IAD) - Groups each CDC function
- 3 x CDC Control Interface (Class 0x02, Subclass 0x02)
- 3 x CDC Data Interface (Class 0x0A, Subclass 0x00)
- 3 x Interrupt IN endpoints (control)
- 6 x Bulk endpoints (3 IN + 3 OUT for data)

8.2.3 Interface Association Descriptor(IAD)

Each CDC port uses an IAD to group its control and data interfaces :

```
{
  .bLength = 8,
  .bDescriptorType = 0x0B,        // IAD
      .bFirstInterface = 0,       // Control interface
      .bInterfaceCount = 2,       // Control + Data
      .bFunctionClass = 0x02,     // CDC
      .bFunctionSubClass = 0x02, // ACM
      .bFunctionProtocol = 0x01, // AT Commands
      .iFunction = 4              // "Protocol Port"
}
```

Listing 2: IAD for Port 0(Protocol Port)

Port	Type	EP	Dir	Size	Interval
0	Interrupt	EP1	IN	16	10ms
0	Bulk OUT	EP2	OUT	64	-
0	Bulk IN	EP3	IN	64	-
1	Interrupt	EP4	IN	16	10ms
1	Bulk OUT	EP5	OUT	64	-
1	Bulk IN	EP6	IN	64	-
2	Interrupt	EP7	IN	16	10ms
2	Bulk OUT	EP8	OUT	64	-
2	Bulk IN	EP9	IN	64	-

Table 41: USB Endpoint Assignments

8.2.4 Endpoint Configuration

8.3 USB State Machine

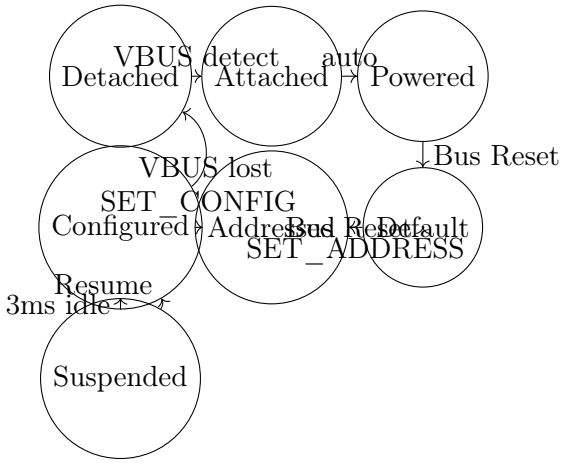


Figure 6: USB Device State Machine

Critical State : Only in `Configured` state can `rx_usb_write / read` transfer data.

8.4 CDC - ACM Class Requests

The USB CDC driver handles these class - specific requests :

Request	Code	Description
SET_LINE_CODING	0x20	Host configures baud rate, parity, stop bits
GET_LINE_CODING	0x21	Host queries current line coding
SET_CONTROL_LINE_STATE	0x22	DTR / RTS control signals(ignored)

Table 42: CDC - ACM Class Requests

Note : Line coding settings(baud rate, parity, etc.) are informational only.USB CDC operates at USB Full - Speed(12 Mbps), not the configured baud rate.

8.5 Bulk Transfer Protocol

8.5.1 Data Flow

Bulk OUT(Host -> Device)

1. Host writes data to / dev / ttyACM *
2. USB driver sends bulk OUT packet(max 64 bytes)
3. RX72N receives packet -> BRDY interrupt fires
4. ISR reads FIFO -> copies to RX ring buffer
5. Application calls `rx_usb_read()` -> retrieves data

Bulk IN(Device -> Host)

1. Application calls `rx_usb_write()` -> copies to TX ring buffer
2. ISR writes FIFO -> sends bulk IN packet(max 64 bytes)
3. BEMP interrupt fires when packet sent
4. Host reads data from / dev / ttyACM *

8.5.2 FIFO Access Requirements(RX72N - Specific)

CRITICAL : RX72N USB FIFO registers **must** be accessed as 16 - bit words, NOT byte - by - byte.

```
        // CORRECT: Read CFIFO as 16-bit words
        for (uint32_t i = 0; i < len; i += 2) {
  const uint16_t word = usb0()->cfifo; // 16-bit read
  data[i] = (uint8_t)(word & 0xFF);    // Low byte
  if ((i + 1) < len) {
    data[i + 1] = (uint8_t)((word >> 8) & 0xFF); // High byte
 }
}
```

Listing 3: Correct FIFO Read(16 - bit access)

```
    // WRONG: Byte-by-byte access causes corruption
    for (uint32_t i = 0; i < len; i++) {
  data[i] = (uint8_t)(usb0()->cfifo & 0xFF); // BAD!
}
```

Listing 4: Incorrect FIFO Read(causes data corruption)

Impact : Byte - by - byte access was the root cause of all bulk transfer failures before Phase 2 fixes.

8.5.3 FIFO Clear Sequence

Before writing to FIFO, the BCLR(Buffer Clear) sequence must be executed :

```
    // Set BCLR bit (hardware clears FIFO)
    usb0() -> cfifoctr
    |= k_usb_fifoctr_bclr;

// Wait for BCLR to complete (hardware clears bit)
uint32_t timeout = 10000;
while ((usb0()->cfifoctr & k_usb_fifoctr_bclr) && timeout--) {
  __asm__ volatile("nop");
}

// Now safe to write data
for (uint32_t i = 0; i < len; i += 2) {
  // ... 16-bit write ...
}
```

Listing 5: FIFO Clear Before Write

Impact : Missing BCLR sequence caused first packet corruption with stale data.

8.6 Frame Protocol

All frames use the same wire format regardless of transport(USB CDC or SPI) :

SYNC (LE)	**SEQ** (LE)	**LEN** (LE)	**TYPE**	**FLAGS**	**PAYLOAD**	**CRC - 32** (LE)
2B	2B	2B	1B	1B	0 - 1024B	4B

Table 43: Frame Wire Format

Header Fields(little - endian) :

- **SYNC** : Magic number `0x55AA` for frame synchronization
- **SEQ** : Sequence number(0 - 65535, wraps around)
- **LEN** : Payload length in bytes
- **TYPE** : Frame type(see Table **??**)
- **FLAGS** : Control flags(RequiresAck, HasFEC, etc.)
- **PAYLOAD** : Variable - length data(Protocol Buffer message)

Checksum(little - endian, IEEE 802.3 LSB - first) : CRC - 32 computed over SYNC + header + payload, stored in little - endian byte order.

Value	Name	Description
0x00	PING	Heartbeat request – sent when link idle for >1s
0x01	PONG	Heartbeat response – echoes PING counter
0x10	COMMAND	Command frame carrying protobuf messages
0x11	RESPONSE	Response frame carrying protobuf messages
0x12	ACK	Acknowledgment (SPI only, HARQ protocol)
0x13	NACK	Negative acknowledgment (SPI only, HARQ protocol)
0xFE	RESET_ACK	Acknowledgment of session reset
0xFF	RESET	Request to reset session (synchronize sequences)

Table 44: Frame Type Definitions

8.6.1 Frame Types

8.6.2 Control Frame Details

PING Frame: TYPE `0x00`, payload is a 4-byte counter (little-endian uint32). Sent as an explicit idle-link probe when no frames have been exchanged for >1s. Both firmware and gateway auto-respond with PONG on PING receipt.

PONG Frame: TYPE `0x01`, payload echoes the PING counter. Confirms link health and validates round-trip latency.

RESET Frame: TYPE `0xFF`, no payload. Requests session reset to synchronize sequence counters.

RESET_ACK Frame: TYPE `0xFE`, no payload. Acknowledges session reset and confirms synchronization.

8.6.3 Cross-Compatibility Test Vectors

Both Go (gateway) and C (firmware) encode/decode frames identically. This is verified by 8 byte-exact test vectors with hardcoded expected wire bytes including CRC-32:

Test locations:

- Go : `star - gateway / internal / frame / frame_test.go`
- C : `e2 - studio - star - rx72n - firmware / tests / test_rx_frame.c`

8.7 USB CDC Lightweight Protocol

The USB CDC protocol is designed for minimal overhead, relying on USB hardware for reliability. Unlike the SPI path which uses HARQ and retransmission, the CDC path omits these as redundant with USB's built-in CRC and retry mechanisms.

Vector	TYPE	SEQ	Payload	CRC - 32
PING	0x00	0	(empty)	0x9B3FAEEB
PONG	0x01	0	counter = 42	0x287737DF
COMMAND	0x10	1	“TEST” + ACK flag	0xDEF35E60
RESPONSE	0x11	1	“OK”	0x6FC08EF4
ACK	0x12	1	(empty)	0xDEABF788
NACK	0x13	1	(empty)	0xC7B0C6C9
RESET	0xFF	0	(empty)	0x081B5399
RESET_ACK	0xFE	0	(empty)	0x110062D8

Table 45: Cross - Compatibility Test Vectors

8.7.1 Send Path

1. Acquire send mutex (serialize concurrent sends)
2. Get next TX sequence from shared SessionState
3. Create frame with sequence, CRC32
4. Encode to wire format
5. Write to `/dev/ttyACM0` with 50ms deadline
6. Release mutex (no wait for ACK)

8.7.2 Receive Path

1. Read from `/ dev / ttyACM0`
2. Decode frame, validate CRC32
3. Check sequence via SessionState :
   - Accept exact match(expected sequence)
   - Accept small gaps($<$ 10 frames, log warning)
   - Reject duplicates or large gaps
4. Return payload to caller

8.7.3 Key Design Decisions

- ✓ Sequence tracking via shared SessionState
- ✓ CRC32 validation
- × No application - level retries(USB hardware handles retries)
- × No FEC encoding(redundant with USB reliability)
- × No ACK / NACK frames(USB provides implicit ACK)

Feature	USB CDC	SPI	Rationale
Physical Layer	Half - duplex, async	Full - duplex, sync	Hardware characteristic
Error Detection	CRC32 only	CRC32 + FEC	USB has built - in CRC
Retransmission	Hardware - level only	Application - level(HARQ)	USB bulk handles retries
FEC	Disabled	Viterbi + Chase Combining	Redundant with USB
ACK / NACK	Not sent	Required	USB provides implicit ACK
Sequence Tracking	Shared SessionState	Shared SessionState	Continuity across transports
Priority	10(preferred)	5(backup)	Prefer simpler USB
Typical Latency	0.5 - 1ms	1 - 2ms	USB has lower overhead
Reliability	Higher(hardware CRC)	High(with retry mechanisms)	USB generally more reliable due to hardware CRC

Table 46: USB CDC vs SPI Transport Comparison

8.8 Transport Comparison(USB CDC vs SPI)

8.9 Heartbeat Mechanism

The heartbeat system uses two complementary detection mechanisms to monitor link health:

8.9.1 Implicit Timeout (Primary) – 200ms

Update `LastSeen` timestamp when ANY valid frame arrives (COMMAND, RESPONSE, PING, etc.). If no frames for 200ms, declare link dead and trigger failover. Zero overhead when link is active — data frames implicitly serve as heartbeats.

8.9.2 Explicit PING (Secondary) – 1s Idle-Link Probe

Send PING with 4-byte counter if link idle for >1s. Wait for PONG echo; validate counter matches.1 consecutive miss triggers failover.Rarely fires under normal traffic; only needed when link is genuinely idle.

Interaction between Implicit Timeout and Explicit PING : The 200ms implicit timeout is the primary dead - link detection mechanism.On an active link, data frames arrive faster than 200ms and continuously reset `LastSeen`, so the implicit timeout never fires.The 1s PING probe is a secondary mechanism : on a link that has occasional traffic(keeping the implicit timer alive) but where the application wants to verify bidirectional health, the PING / PONG exchange confirms round - trip connectivity.Note that on a truly idle link with zero traffic from either side, the 200ms implicit timeout fires before the 1s PING interval elapses– - this is by design, as an idle link with no frames in 200ms is considered dead and triggers failover.

8.9.3 Timing Budget

Parameter	Value	Description
Implicit timeout	200ms	Primary detection, any frame resets
Check interval	50ms	failureTimeout / 4, worst - case detection ~ 250ms
PING interval	1000ms	Secondary, idle - link probe
WDT timeout	1000ms	RX72N hardware watchdog
Safety margin	4x	250ms worst - case $\ll$ 1000ms WDT

Table 47: Heartbeat Timing Budget

8.9.4 Control Frame Dispatching

Both firmware(C) and gateway(Go) auto - respond PONG to PING and RESET_ACK to RESET .Control frames(PING, PONG, ACK, NACK, RESET_ACK) are consumed internally.Only data frames(COMMAND, RESPONSE) are returned to callers.

8.10 Debug Logging Over USB CDC Port 2

8.10.1 Architecture

The logging system supports dual output to both UART and USB CDC Port 2 :

- **UART(SCI12)** : Always works, independent of USB state
- **USB CDC Port 2** : Enabled when `USB_LOG_MIRROR = 1`
- **Boot buffering**: 512B ring buffer for logs during USB enumeration (0-200ms)
- **Thread safety**: ThreadX mutex for multi-task logging
- **Non-blocking**: Drops logs on TX full, tracks statistics

8.10.2 Boot Log Sequence

8.10.3 Implementation

Boot Buffer Structure

```
typedef struct {
  char data[512]; // Ring buffer storage
  uint16_t head;  // Write index
  uint16_t count; // Buffered bytes
  bool overflow;  // Overflow flag
} usb_log_boot_buffer_t;

static usb_log_boot_buffer_t s_boot_buffer = {0};
static volatile bool s_usb_ready = false;
```

Listing 6: Boot Log Ring Buffer

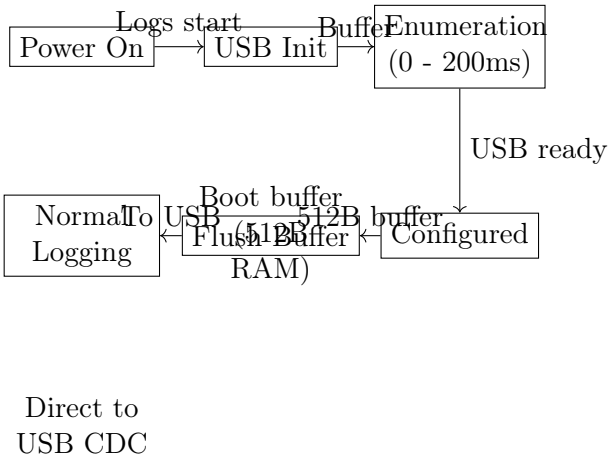


Figure 7: Boot Log Buffering Sequence

Log Write Flow

1. Application calls `rx_log_info()`, `rx_log_warn()`, etc.
2. Macro expands to `uart_debug_puts()` + `rx_log_usb_puts()`
3. `rx_log_usb_puts()` acquires ThreadX mutex
4. Check USB state :
   - **Not configured** : Buffer to 512B ring buffer
   - **Configured** : Write directly to USB CDC Port 2 via `rx_usb_write()`
5. If USB TX buffer full(`k_rx_err_busy`) : Drop log, increment `dropped_bytes`
6. Release mutex

Automatic Flush

```
static void internal_check_usb_ready(void) {
  if (s_usb_ready)
    return; // Already flushed

  if (!rx_usb_is_configured(k_usb_port_log)) {
    return; // Not ready yet, keep buffering
 }

  // USB ready! Flush boot buffer in 64-byte chunks
  while (s_boot_buffer.count > 0) {
    uint16_t chunk_len = min(s_boot_buffer.count, 64);
    rx_err_t err =
        rx_usb_write(k_usb_port_log, &s_boot_buffer.data[tail],
            chunk_len);
    if (err == k_rx_err_busy)
      return; // Retry later

    // Update indices
    tail = (tail + chunk_len) % 512;
```

```
    s_boot_buffer.count -= chunk_len;
 }

  s_usb_ready = true; // Mark as flushed

  // Log overflow warning if buffer overflowed
  if (s_boot_buffer.overflow) {
    rx_usb_puts(k_usb_port_log,
                "[WARN] Boot buffer overflow, some logs dropped\r\n");
 }
}
```

Listing 7: Automatic Boot Buffer Flush

8.10.4 Thread Safety

ThreadX mutex protects all logging operations :

```
void rx_log_usb_puts(const char *str) {
  // Lazy mutex initialization
  if (!s_mutex_initialized) {
    tx_mutex_create(&s_log_mutex, "usb_log_mutex", TX_NO_INHERIT);
    s_mutex_initialized = true;
 }

  // Acquire mutex (blocks until available)
  tx_mutex_get(&s_log_mutex, TX_WAIT_FOREVER);

  // Critical section: write to USB
  internal_write_usb(str, strlen(str));

  // Release mutex
  tx_mutex_put(&s_log_mutex);
}
```

Listing 8: Thread - Safe Logging

8.10.5 Statistics Tracking

```
typedef struct {
  uint32_t total_bytes;   // Total bytes sent/buffered
  uint32_t dropped_bytes; // Bytes dropped (TX full/overflow)
  uint32_t boot_buffered; // Bytes buffered during boot
  uint32_t usb_errors;    // USB error count
} usb_log_stats_t;

// Query statistics
usb_log_stats_t stats;
rx_log_usb_get_stats(&stats);
rx_log_info("USB_LOG", "Dropped: %u bytes (%.1f%%)", stats.dropped_bytes,
            100.0f * stats.dropped_bytes / stats.total_bytes);
```

Listing 9: USB CDC Logging Statistics

8.11 Performance Characteristics

8.11.1 Throughput

Metric	Value	Notes
USB Speed	12 Mbps	Full - Speed(USB 2.0)
Practical Max	10 Mbps	Protocol overhead reduces from 12 Mbps theoretical
Typical Throughput	5 - 8 Mbps	Under normal application load
Packet Size	64 bytes	Full - Speed bulk endpoint limit
Latency(send 1KB)	0.5 - 1 ms	rx_usb_write() to host receive
Latency(receive 1KB)	0.5 - 1 ms	Host write to rx_usb_read()

Table 48: USB CDC Performance Metrics

8.11.2 Memory Usage

Component	Size(bytes)	Description
Port 0 RX Buffer	1024	Protocol port receive ring buffer
Port 0 TX Buffer	1024	Protocol port transmit ring buffer
Port 1 RX Buffer	256	Decoded port receive ring buffer
Port 1 TX Buffer	512	Decoded port transmit ring buffer
Port 2 RX Buffer	256	Log port receive ring buffer
Port 2 TX Buffer	1024	Log port transmit ring buffer
Boot Log Buffer	512	Boot log buffering(USB CDC logging)
USB Descriptors	512	Device, config, interface, endpoint
State Structures	128	3 ports x 42 bytes per port
Total	**5.3 KB**	Static allocation(no malloc / free)

Table 49: USB CDC Memory Usage

8.12 Error Handling and Recovery

8.12.1 USB Not Configured

Symptom : `rx_usb_write / read` returns `k_rx_err_invalid_state`

Recovery :

- Wait for USB enumeration: `while (!rx_usb_is_configured(port))`
- For logging : Buffer to boot buffer, flush after USB ready
- For protocol / data : Retry after delay, or use alternate channel(UART)

8.12.2 TX Buffer Full

Symptom : `rx_usb_write` returns `k_rx_err_busy`
Recovery :

- **Logging** : Drop message, increment statistics(non - blocking policy)
- **Critical data** : Retry with timeout, or signal error to application
- Poll `rx_usb_tx_available()` before write to avoid full condition

8.12.3 Cable Disconnect

Event : `k_usb_event_detached` delivered to callback
Recovery :

- Abort pending transfers
- Clear TX / RX ring buffers
- Reset state to `Detached`
- Wait for cable reconnect (`k_usb_event_attached`)

8.13 NASA Power of 10 Compliance

Rule	Status	Implementation
1. Simple control flow	✓	No goto/setjmp/recursion
2. Fixed loop bounds	✓	All loops bounded by constants
3. No dynamic memory	✓	Zero malloc/free, all buffers static
4. Short functions	✓	All functions <60 lines
5. Assertions	✓	Min 2 checks per function
6. Small scope	✓	Variables near use, static for module data
7. Check returns	✓	All rx_usb_* return values checked
8. Limited preprocessor	✓	C23 typed enums, minimal macros
9. Restrict pointers	Deviation	Function pointers for callbacks (DIP)
10. Compiler warnings	✓	-Wall -Wextra -Werror, zero warnings

Table 50: NASA Power of 10 Compliance Matrix

Intentional Deviation: Rule 9 (function pointers) violated for Dependency Inversion Principle - enables testability via mock implementations.

8.14 References

- **RX72N User's Manual**: Chapter 40 - USB 2.0 Full-Speed Module
- **USB 2.0 Specification**: Chapter 9 - Device Framework
- **USB CDC-ACM Specification**: Class Definitions for Communications Devices v1.2
- **Transport Architecture**: `star - gateway / TRANSPORT_ARCHITECTURE.md`

- **Implementation** : `libs / rx_usb / inc / rx_usb.h`, `libs / rx_core / src / rx_log_usb.c`
- **Cross-Compatibility Tests**: `star-gateway/internal/frame/frame_test.go`, `e2 - studio - star - rx72n - firmware / tests / test_rx_frame.c`
- **Testing Guide** : `e2 - studio - star - rx72n - firmware / USB_CDC_PHASE2_TESTING_GUIDE.md`

== == == == == == == == == == == == = = =

9 ROS2 Integration and Sensor Fusion Architecture

9.1 Overview

The STAR platform utilizes ROS2 Jazzy Jalisco as the primary high-level orchestration layer. The integration strategy focuses on providing a modular, reliable, and deterministic environment for SLAM, navigation, and mission control. This section details the architecture for Visual-LiDAR sensor fusion, coordinate frame definitions, and custom node interfaces.

9.2 Visual-LiDAR Sensor Fusion with RTAB-Map

The platform implements a heterogeneous sensor fusion strategy combining geometric data from LiDAR and appearance-based data from an RGB-D camera using **RTAB-Map** (Real-Time Appearance-Based Mapping).

9.2.1 Fusion Architecture

The fusion process is divided into two main layers:

1. **Local State Estimation(EKF) :** The `robot_localization` package runs an Extended Kalman Filter(EKF) fusing :
   - Wheel Odometry(from RX72N via `star_spi_bridge`)
   - IMU Linear Acceleration and Angular Velocity(from OAK - D or dedicated IMU)
2. **Global SLAM and Loop Closure(RTAB - Map) :** The `rtabmap_ros` node consumes :
   - Laser Scan data from RPLiDAR C1.
   - RGB - D images and PointClouds from the OAK - D camera.
   - Filtered Odometry from the EKF.

9.2.2 Mapping Strategy

RTAB-Map performs loop closure detection using a bag-of-words approach. When a loop is detected, it optimizes the pose graph to minimize drift. The resulting map is published as a 2D occupancy grid for Nav2 and a 3D octomap for obstacle avoidance.

9.3 Coordinate Frames (REP-105)

The STAR platform adheres to the **REP-105** standard for coordinate frames. The transform tree (`tf2`) is structured as follows:

- `map`: The global coordinate frame. Fixed to the world.
- `odom`: The local coordinate frame. Drift-prone, but continuous. The `map` → `odom` transform is published by RTAB-Map (Loop Closure).
- `base_link`: The center of the robot's drive axis. The `odom` → `base_link` transform is published by the EKF.
- `laser_frame`: The reference frame for the RPLiDAR C1.
- `camera_link`: The reference frame for the OAK-D Depth Camera.

9.4 Custom ROS2 Nodes and Interfaces

Three primary custom nodes facilitate the interaction between the STAR hardware and the ROS2 ecosystem.

9.4.1 `star_spi_bridge`

Bridges the low - level SPI protocol(RX72N) to ROS2 topics.

- **Subscribed Topics:** `/ cmd_vel`(geometry_msgs / Twist)
- **Published Topics:** `/ odom / unfiltered`(nav_msgs / Odometry), `/ joint_states`(sensor_msgs / JointState)
- **Parameters:** `spi_device`, `spi_speed_hz`, `gear_ratio`

9.4.2 `star_gateway_bridge`

Provides a bridge between the gRPC / WebSocket gateway and the ROS2 computational graph.

- **Subscribed Topics:** `/ robot_status`, `/ battery_state`
- **Published Topics:** `/teleop/cmd_vel` (overrides for manual control)
- **Services:** `/ set_pid_gains`, `/ trigger_calibration`

9.4.3 `star_safety_monitor`

Ensures platform integrity and manages emergency states.

- **Logic :** Monitors battery voltage, motor current, and heartbeat from the RPi5 and RX72N.
- **Actions :** Publishes `/ emergency_stop` or resets the motor controller if a timeout occurs.

9.5 Multi-Machine Communication

To ensure isolation and prevent interference in multi-robot environments, the `ROS_DOMAIN_ID` environment variable must be set uniquely for each STAR unit (e.g., `export ROS_DOMAIN_ID=42`).

9.6 Hardware Persistence (udev rules)

Consistent device naming is critical for reliable startup. The following `udev` rules are required in `/etc/udev/rules.d/99-star.rules`:

```
#RPLiDAR C1
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", SYMLINK+="ttyLidar"
#RX72N Motor Controller(via USB - Serial if applicable, or SPI identification)
SUBSYSTEM=="spidev", KERNEL=="spidev0.0", GROUP=="spi", MODE=="0660"
```

Note : The `spi` group must exist on the system before applying the udev rules .The STAR Docker container automatically creates this group via `groupadd - f spi` during build.On bare - metal installations, create the group manually :

```
sudo groupadd spi sudo usermod - aG spi $USER
```

10 ROS2 C++ Style Guide

10.1 Overview

The STAR project uses different coding conventions for ROS2 C++ code compared to RX72N firmware. This section documents the ROS2 C++ style guide, which supplements the C firmware style guide (Section **??**).

10.1.1 When to Use This Guide

- **ROS2 packages** (`star - ros2 / src /*/`): Use ROS2 C++ style
- **RX72N firmware** (`star-rx72n-firmware/`): Use C firmware style
- **Gateway** (`star-gateway/`): Follow Go conventions

10.1.2 Key Differences

Feature	C Firmware (RX72N)	ROS2 C++
Classes	N/A	`CamelCase`
Methods	`snake_case`	`snake_case` (same)
Variables	`snake_case`	`snake_case` (same)
Member vars	`s_` prefix	trailing `_`
Headers	`.h`	`.hpp`
Line limit	100 characters	120 characters
Namespaces	Not used	`star::package_name::`
Error handling	Return codes	Exceptions
Logging	`uart_puts()`	`RCLCPP_INFO/WARN/ERROR`
Include guards	`STAR_RX72N_FILE_H`	`PACKAGE_FILE_HPP_`
Doxygen	`/**` and `/**<`	`/**` and `/**<` (same)

Table 51: C vs C++ Style Comparison

10.2 Naming Conventions

10.2.1 Classes and Types

Use `CamelCase` for classes following ROS2 conventions:

```
// CamelCase for classes (ROS2 convention)
class StarGatewayBridgeNode : public rclcpp::Node {
// ...
};

// CamelCase for structs used as types
struct TelemetryData {
double battery_voltage_;
int32_t current_ma_;
};

// Type aliases use CamelCase
using TelemetryPtr = std::shared_ptr<TelemetryData>;
```

Listing 10: Class naming examples

10.2.2 Methods and Functions

Use `snake_case` for method names (same as C firmware):

```
// snake_case for methods (same as C firmware)
void publish_telemetry(const TelemetryData & data);
bool is_connected() const;
void on_battery_state_received(const
   sensor_msgs::msg::BatteryState::SharedPtr
msg);

// Use verb-based names that clarify actions
void check_for_errors();      // Good: Clear intent
void error_check();           // Bad: Noun-first is confusing
```

Listing 11: Method naming examples

10.2.3 Variables

Use `under_scored` for variables with trailing underscores for member variables:

```
// under_scored for variables
int loop_counter = 0;
std::string node_name = "gateway_bridge";

// Member variables with trailing underscore
class MyNode : public rclcpp::Node {
private:
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
std::shared_ptr<grpc::Channel> grpc_channel_;
bool is_connected_;
};

```

```cpp
// Constants: ALL_CAPITALS
const int MAX_RETRIES = 3;
const double DEFAULT_TIMEOUT_S = 5.0;
```

Listing 12: Variable naming examples

10.2.4 Namespaces

Package-based namespaces use `under_scored` naming:

```cpp
namespace star { namespace spi_bridge {

class SpiDriverNode : public rclcpp::Node {
// ...
};

}  // namespace spi_bridge
}  // namespace star
```

Listing 13: Namespace examples

10.3 File Naming and Organization

10.3.1 Header Files

C++ headers use the `.hpp` extension (not `.h`):

- `star_gateway_bridge_node.hpp`
- `message_converter.hpp`
- `grpc_client.hpp`

Include guards follow the pattern: `PACKAGE_FILE_NAME_HPP_`

```cpp
#ifndef STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_
#define STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_

// ... code ...

#endif  // STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_
```

Listing 14: Include guard example

10.3.2 Source Files

Source files use the `.cpp` extension with `under_scored` naming:

- `star_gateway_bridge_node.cpp`
- `message_converter.cpp`
- `grpc_client.cpp`

10.4 Header Organization

Headers must be organized in a standard order (enforced by `.clang-format`):

1. License and copyright
2. Include guard
3. ROS2 core includes (`<rclcpp/...>`)
4. ROS2 message includes (`<geometry_msgs/...>`, etc.)
5. Project includes (`"star/..."`)
6. System C++ includes (`<memory>`, `<string>`, etc.)
7. System C includes (`<cstdint>`, etc.)
8. Namespace declaration
9. Class/function declarations

See CLAUDE.md for complete example.

10.5 Code Formatting

10.5.1 Line Length

- **ROS2 C++**: 120 characters (`star-ros2/.clang-format`)
- **C firmware**: 100 characters (`star-rx72n-firmware/.clang-format`)

10.5.2 Indentation

All code uses 2-space indentation (same as C firmware):

```
class MyNode : public rclcpp::Node { public: MyNode() :
Node("my_node") { timer_ = this->create_wall_timer(
std::chrono::milliseconds(100),
std::bind(&MyNode::timerCallback, this));
}

private:
void timerCallback() {
// ...
}

rclcpp::TimerBase::SharedPtr timer_;
};
```

Listing 15: Indentation example

10.5.3 Braces

- **Functions**: Braces on new line (same as C firmware)
- **Control statements**: Cuddled braces (`if (x) {`)
- **Namespaces**: No indentation inside namespace

```
// Functions: Braces on new line
void myFunction()
{
// ...
}

// Control statements: Cuddled braces
if (condition) {
// ...
} else {
// ...
}

// Namespaces: No indentation inside
namespace star {
namespace spi_bridge {

// Content at zero indent
class MyClass {
};

}  // namespace spi_bridge
}  // namespace star
```

Listing 16: Brace style examples

10.6 ROS2-Specific Patterns

10.6.1 Node Inheritance

Inherit from `rclcpp::Node` for basic nodes or `rclcpp_lifecycle::LifecycleNode` for safety-critical nodes:

```
// Basic node
class StarGatewayBridgeNode : public rclcpp::Node {
public:
StarGatewayBridgeNode();

private:
void telemetryCallback();

rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
rclcpp::TimerBase::SharedPtr telemetry_timer_;
};

// Safety-critical lifecycle node
```

```
class StarSpiDriverNode : public rclcpp_lifecycle::LifecycleNode {
public:
using CallbackReturn =
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;

StarSpiDriverNode();

// Lifecycle transitions
CallbackReturn on_configure(const rclcpp_lifecycle::State &) override;
CallbackReturn on_activate(const rclcpp_lifecycle::State &) override;
CallbackReturn on_deactivate(const rclcpp_lifecycle::State &) override;
CallbackReturn on_cleanup(const rclcpp_lifecycle::State &) override;
};
```

Listing 17: Node inheritance

10.6.2 Publishers and Subscribers

```
class MyNode : public rclcpp::Node { public: MyNode() :
Node("my_node") {
// Publisher: use SharedPtr
telemetry_pub_ = this->create_publisher<std_msgs::msg::String>(
"/telemetry",
10  // QoS depth
);

// Subscriber: use std::bind
cmd_sub_ = this->create_subscription<geometry_msgs::msg::Twist>(
"/cmd_vel",
10,
std::bind(&MyNode::cmdVelCallback, this, std::placeholders::_1)
);
}

private:
void cmdVelCallback(const geometry_msgs::msg::Twist::SharedPtr msg) {
// Handle message
}

rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_sub_;
};
```

Listing 18: Publishers and subscribers

10.6.3 Timers

Use `create_wall_timer` for periodic operations:

```
timer_ = this->create_wall_timer(
std::chrono::milliseconds(100),  // 10 Hz
std::bind(&MyNode::timerCallback, this)
);
```

Listing 19: Timer usage

10.7 Error Handling

ROS2 uses exceptions (different from C firmware return codes):

```
// Throw exceptions for errors
void publishTelemetry()
{
if (!grpc_channel_) {
throw std::runtime_error("gRPC channel not initialized");
}
// ...
}

// Catch exceptions in callbacks (avoid crashing node)
void timerCallback()
{
try {
publishTelemetry();
} catch (const std::exception & e) {
RCLCPP_ERROR(this->get_logger(), "Failed to publish: %s", e.what());
}
}
```

Listing 20: Exception handling

10.8 Logging

Use `RCLCPP_*` macros for logging, not `printf` or `cout`:

```
// ROS2 logging macros (preferred)
RCLCPP_INFO(this->get_logger(), "Node started");
RCLCPP_WARN(this->get_logger(), "Connection lost, retrying...");
RCLCPP_ERROR(this->get_logger(), "Failed to initialize: %s",
    error_msg.c_str());
RCLCPP_DEBUG(this->get_logger(), "Processing message %d", count);

// Throttled logging (max once per 5 seconds)
RCLCPP_WARN_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
"Command stale (%ldms > %dms)", cmd_age_ms, timeout_ms_);
```

Listing 21: ROS2 logging

Important: Never use `printf()` or `std::cout` in ROS2 nodes.

10.9 Documentation

All public APIs must have Doxygen comments using `/**` and `/**<` (same as C firmware):

```
/**
* @brief Brief description of class
*
```

```
* Detailed description can span multiple lines. Explain purpose,
* usage, and any important details.
*/
             class StarGatewayBridgeNode : public rclcpp::Node {
  public:
    /**
     * @brief Constructor for gateway bridge node
     *
     * @param options Node options for configuration
     */
    explicit StarGatewayBridgeNode(
        const rclcpp::NodeOptions &options = rclcpp::NodeOptions());

  private:
    /**
     * @brief Callback for telemetry publishing timer
     *
     * Forwards latest telemetry to Gateway via gRPC. Called at 10 Hz.
     */
    void telemetry_timer_callback();

    rclcpp::TimerBase::SharedPtr telemetry_timer_; /**< Timer for
       telemetry */
};
```

Listing 22: Doxygen documentation

10.10 Constants

Prefer enums and `const`(same philosophy as C firmware) :

```
        // Enum for related constants
        enum class MotorState {
          kIdle = 0,
          kRunning = 1,
          kError = 2
      };

  // const for single values
  const int DEFAULT_QOS_DEPTH = 10;
  const double MAX_LINEAR_VELOCITY_MPS = 1.0;

  // static const for class-specific constants
  class MyNode : public rclcpp::Node {
  private:
    static constexpr int kMaxRetries = 3;
    static constexpr double kTimeoutS = 5.0;
};
```

Listing 23: Constants

10.11 Formatting Enforcement

All ROS2 C++ code uses `clang - format` with configuration in `star - ros2 /.clang - format` :

- 2-space indentation (same as C firmware)
- 120-character line limit (ROS2 standard)
- Cuddled braces for control statements
- Function braces on new line
- No indentation inside namespaces
- ROS2-specific include order (core, messages, system, project)

10.11.1 Manual Formatting

Format code before committing :

```
cd star -
     ros2 find src - name '*.cpp' - o - name '*.hpp' |
     xargs clang - format - i
```

10.11.2 CI Enforcement

The `.github / workflows / ros2.yml` workflow runs `clang - format` checks on all PRs :

```
find src -
     name '*.cpp' - o - name '*.hpp' |
     xargs clang - format-- dry -
         run-- Werror
```

If formatting check fails, the build will fail with instructions to fix.

10.12 References

- **ROS2 C++ Style Guide** : https://docs.ros.org/en/rolling/The-ROS2-Project/Contributing/Code-Style-Language-Versions.html
- **ROS C++ Best Practices** : https://wiki.ros.org/CppStyleGuide
- **Google C++ Style Guide** : https://google.github.io/styleguide/cppguide.html
- **STAR C Firmware Style** : CLAUDE.md Section "Code Style"
- **STAR Project Documentation** : `docs / star_documentation.pdf`

10.13 Integration with Existing Tools

The ROS2 C++ style guide integrates seamlessly with existing STAR project tools :

Tool	Purpose	Configuration
`clang - format`	Code formatting	`star - ros2 /.clang - format`
`ament_cppcheck`	Static analysis	ROS2 workflow
`ament_cpplint`	Style checking	ROS2 workflow
`colcon build`	Build system	CMakeLists.txt
`colcon test`	Unit testing	gtest integration
GitHub Actions	CI / CD enforcement	`.github / workflows / ros2.yml`

Table 52: ROS2 Development Tools

10.14 Summary

The STAR project maintains separate but complementary style guides for C firmware and C++ ROS2 code:

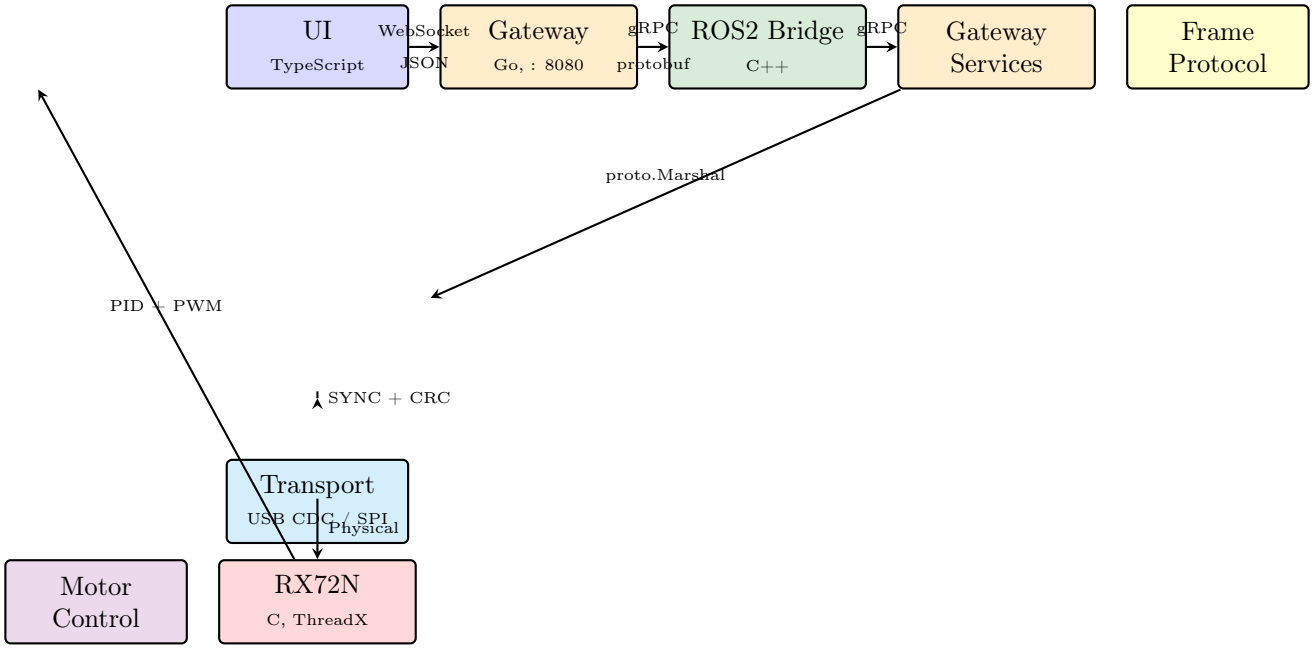
- **C firmware** follows embedded best practices with strict NASA Power of 10 compliance
- **ROS2 C++** follows ROS community conventions while maintaining STAR project philosophy
- Both styles share common principles : enums over macros, error checking, documentation
- Automated tooling enforces consistency across all code

For complete examples and additional details, refer to CLAUDE.md.

11 End - to - End Communication Stack

11.1 Overview

The STAR communication stack connects the user interface through a layered architecture down to the RX72N motor controller.Each layer serves a distinct purpose : the UI provides human interaction, the gateway bridges high - level commands to the robot control framework, ROS2 handles autonomous navigation and sensor fusion, and the frame protocol delivers serialized messages over either USB CDC or SPI to the real - time firmware.



Key Insight : All control paths(autonomous Nav2 and manual UI) converge at the gateway, ensuring a single serialization and transport layer regardless of command source.The RX72N firmware is transport - agnostic– - it processes identical frames whether they arrive over USB CDC or SPI.

11.2 Transport Priority

USB CDC is the **primary** transport and SPI is the **backup**. Priority values are defined in `star-gateway/internal/manager/types.go`:

```
PriorityUSB = 10  // higher = preferred
PrioritySPI = 5
```

Listing 24: Transport Priority Constants

The `TransportManager` selects the highest-priority healthy transport at runtime. USB is preferred for three reasons:

- **Lower latency:** 0.5 –1 ms(USB) vs 1 –2 ms(SPI)
- **Hardware CRC and bulk retries :** Built into the USB protocol , making application - level HARQ redundant on this path
- **Simplified software :** No ACK / NACK handshake or FEC codec overhead required

SPI enables full HARQ(per ADR - 001) because it lacks USB's built-in reliability mechanisms. A shared `SessionState` (TX/RX sequence counters) spans both transports, preventing sequence desynchronization during failover. Both USB and SPI increment the same sequence counter.

11.3 Protocol Stack(5 Layers)

Table 53: Communication Protocol Stack

Layer	Name	Go Package	C Library	Des
5	Application	`internal / service /`	`src / tasks /`	gRP man
4	Serialization	`google.golang.org / protobuf`	`libs / rx_nanopb /`	Prot Go,
3	HARQ + FEC	`internal / link /`	`libs / rx_harq /`, `libs / rx_spi_link /`	HAI (RS / C / F retr
2	Framing	`internal / frame /`	`libs / rx_frame /`	SYN load
1	Transport	`internal / transport /`	RSPI0 / USB peripheral	SPI Full

== ==
== == == == == == == == == == == == == == == = == == == == == == ==
== =

11.4 Protocol Primer

This subsection explains each protocol building block from first principles. Understanding these concepts is essential for working on any part of the communication stack.

11.4.1 CRC-32 (Cyclic Redundancy Check)

A CRC-32 is a 4-byte "fingerprint" computed over a block of data. The sender computes the CRC over the frame contents and appends it; the receiver recomputes the CRC over the same data and compares.If even a single bit flipped during transmission, the CRCs will not match and the receiver knows the frame is corrupt.

How it works:

1. Treat the data bytes as a long binary polynomial .
2. Divide this polynomial by a fixed *generator polynomial*(IEEE 802.3 : `0x04C11DB7`) .
3. The 32 - bit remainder is the CRC.Append it to the frame .
4. The receiver performs the same division.If the remainder is zero, the frame is intact.

Properties:

- **Detection only, not correction :** CRC tells you *something* is wrong, but not *what* or *where*.If a CRC fails, the only option is to discard the frame and request retransmission .
- **Detects all single - bit, double - bit, and odd - bit errors.** Detects all burst errors up to 32 bits.
- **Hardware - accelerated on the RX72N :** The CRC Calculator peripheral computes CRC - 32 in hardware, avoiding CPU overhead.
- **Used on every frame :** Both USB CDC and SPI frames include CRC-32, regardless of whether HARQ is enabled.

Analogy : CRC is like a check digit on a credit card number.It catches typos instantly but cannot fix them– - you have to re - enter the number.

11.4.2 ARQ(Automatic Repeat Request)

ARQ is the simplest reliability mechanism : if a frame arrives corrupted(CRC fails) or does not arrive at all(timeout), the receiver asks the sender to retransmit.This is **error detection + retransmission**, not error correction.

Protocol flow:

1. Sender transmits frame with `REQUIRES_ACK` flag set .
2. Sender starts a timer(10 ms timeout in STAR) .
3. Receiver validates CRC :
   - CRC valid → send **ACK**(positive acknowledgment) .
   - CRC invalid → send **NACK**(negative acknowledgment) .
4. Sender receives response :
   - ACK → success, move to next frame .
   - NACK → retransmit the same frame(set `RETRANSMIT` flag) .
   - Timeout → retransmit(assume frame was lost) .
5. After 3 failed retries, declare the link failed.

Table 54: ARQ Protocol Parameters(STAR Configuration)

Parameter	Value	Description
Max retries	3	Attempts before failure
ACK timeout	10 ms	Wait time per attempt
Worst - case delay	40 ms	Initial send + 3 retransmissions
Sequence range	0 –65535	16 - bit wraparound counter

Limitation: Pure ARQ wastes the corrupted frame entirely. If 99% of the bits arrived correctly and 1% were wrong, ARQ discards all of it and retransmits from scratch. This is where FEC comes in.

11.4.3 FEC (Forward Error Correction)

FEC adds *redundant* data to the transmission so the receiver can **correct** errors without retransmission. The idea is simple: send more bits than strictly necessary, so that even if some are corrupted, the original data can be recovered mathematically.

Everyday analogy: If you spell out a phone number as "five-five-five, one-two-three-four", someone hearing "five-five-STATIC-one-two-three-four" can guess the missing digit because they know phone numbers are 7 digits and the pattern is obvious. FEC formalizes this intuition.

STAR uses convolutional coding, a specific FEC technique:

- **Constraint length $K = 7$:** The encoder has a 6-bit shift register (memory), so each output bit depends on the current input bit and the 6 previous input bits. This creates complex relationships between bits that the decoder can exploit.
- **Rate 1/2:** For every 1 input bit, the encoder produces 2 output bits. This means the encoded data is exactly $2\times$ the size of the original, doubling bandwidth usage in exchange for error correction capability.
- **Generator polynomials:** $G_1 = 171_8$ (`0x79`) and $G_2 = 133_8$ (`0x5B`). These are NASA standard polynomials used in deep-space communication, chosen for their optimal distance properties.

Encoding process:

1. Each input bit shifts into a 6 - stage register.
2. Two output bits are produced by XOR - ing specific register taps(determined by G_1 and G_2).
3. A 100 - byte payload becomes a 200 - byte encoded payload plus tail bits.

11.4.4 Viterbi Decoder (Soft-Decision)

The Viterbi algorithm is the standard method for decoding convolutional codes. It finds the *most likely* original data sequence given a noisy received signal.

Hard bits vs. soft bits:

- **Hard bits :** Binary 0 or 1. No confidence information. "The bit is 1."
- **Soft bits :** A signed value from -127 to $+127$(stored as `int8_t`) .The sign indicates the likely bit value(negative = likely 0, positive = likely 1), and the *magnitude* indicates confidence.For example :
  - $+127$ = "very confident this is a 1"
  - $+10$ = "probably a 1, but not sure"
  - -127 = "very confident this is a 0"
  - 0 = "complete erasure, no idea"

Why soft bits matter : Soft - decision decoding provides approximately 2 dB gain over hard - decision decoding.This means the decoder can correct more errors because it uses *confidence* information, not just guesses.

Viterbi algorithm overview:

1. The encoder can be modeled as a state machine with $2^{K-1} = 64$ states (for $K = 7$).
2. At each time step, the decoder computes the "distance" (error metric) between the received soft bits and what each possible state transition would have produced.

3. It keeps only the best path (lowest cumulative error) into each state — this is the "survivor path."
4. After processing all received bits, the decoder traces back through the survivor paths to find the most likely original bit sequence.

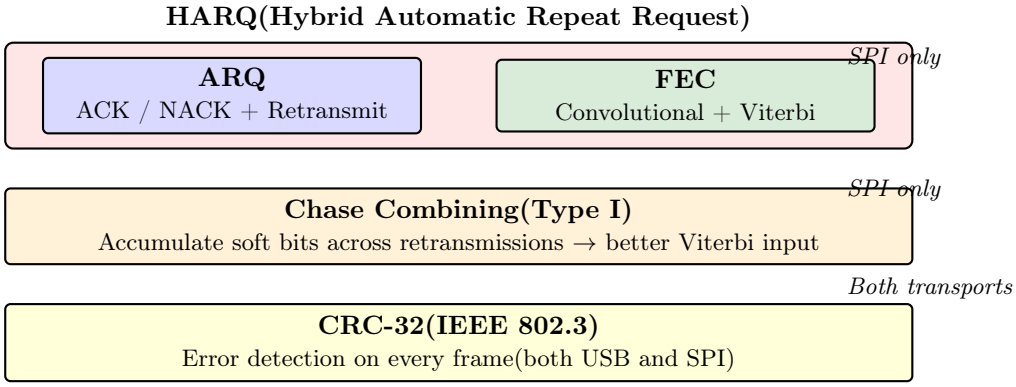
Table 55: Viterbi Decoder Parameters

Parameter	Value	Description
States	64	2^{K-1} for $K = 7$
Traceback depth	35	$\sim 5 \times K$
Decision type	Soft (int8)	-127 to $+127$
Path metric type	uint32	Cumulative error metric
Memory (RX72N)	∼40 KB	State arrays + traceback

11.4.5 HARQ(Hybrid Automatic Repeat Request)

HARQ is **not a separate protocol** — it is the **combination** of ARQ and FEC working together. The key insight is that each mechanism compensates for the other's weakness:

- **ARQ alone:** Discards corrupted frames entirely, wastes bandwidth retransmitting data that was mostly correct.
- **FEC alone:** Can correct limited errors but cannot request retransmission if corruption is too severe.
- **HARQ (both):** FEC corrects minor errors in-place. If corruption exceeds FEC capability, ARQ requests retransmission. On retransmission, Chase Combining accumulates evidence from both attempts for even better decoding.



The formula is : **HARQ = ARQ + FEC + Chase Combining**.CRC - 32 is the foundation used by all paths .

11.4.6 Chase Combining(Type I HARQ)

Chase Combining is the mechanism that makes retransmissions *more valuable* than just sending the same data again.Instead of discarding the failed frame and using only the retransmitted copy, Chase Combining **accumulates soft - bit observations** from every transmission attempt.

How it works:

1. **First transmission:** Receiver gets soft bits (e.g., $[+80, -30, +5, -120, \ldots]$). CRC fails.
2. **Receiver stores soft bits** in an `int16` accumulator array (not discarded!).
3. **Retransmission:** Receiver gets new soft bits for the same data (e.g., $[+60, -90, +40, -100, \ldots]$).
4. **Combine:** Add element-wise: $[+80+60, -30+(-90), +5+40, -120+(-100), \ldots] = [+127, -120, +45, -127, \ldots]$
5. **Clamp** combined values to $[-127, +127]$ (int8 range) and feed to Viterbi decoder.
6. Bits where both transmissions agreed become *more confident* (higher magnitude). Bits where they disagreed partially cancel, correctly reflecting uncertainty.

Gain : Each additional transmission provides approximately 3 dB of combining gain.Two transmissions combined can correct errors that neither could individually.After 3 transmissions with combining, the system achieves roughly 4 –5 dB total coding gain over a single attempt.

Table 56: Chase Combining Example(4 soft bits, 2 transmissions)

	Bit 0	**Bit 1**	**Bit 2**	**Bit 3**
TX 1(soft bits)	+80	−30	+5	−120
TX 2(soft bits)	+60	−90	+40	−100
Combined(sum)	+140	−120	+45	−220
Clamped to int8	+127	−120	+45	−127
Interpretation	Strong 1	Likely 0	Weak 1	Strong 0

Key Insight : Chase Combining is why HARQ is strictly better than pure ARQ.A pure ARQ system discards the failed frame and starts fresh.HARQ with Chase Combining treats every transmission as additional evidence, making each retry exponentially more likely to succeed.

11.4.7 USB CDC(Communications Device Class)

USB CDC is a standard USB device class that creates a **virtual serial port** over USB.When the RX72N is plugged into the Raspberry Pi 5 via USB, the Linux kernel automatically creates a `/dev/ttyACM0` device that applications can read / write like a regular serial port.

Why USB CDC needs no HARQ:

- **Hardware CRC-16:** Every USB packet (max 64 bytes for Full-Speed) includes a 16-bit CRC computed and verified by the USB hardware controller. Corrupted packets are automatically discarded at the hardware level.
- **Automatic bulk retries:** USB bulk transfers automatically retry failed packets up to 3 times at the hardware level, transparent to software. The application never sees the retransmission.
- **Differential signaling:** USB uses a twisted differential pair (D+/D-), which provides inherent common-mode noise rejection. This makes bit errors extremely rare compared to single-ended SPI.
- **Handshake protocol:** Every bulk transfer includes a three-phase handshake (TOKEN → DATA → HANDSHAKE), ensuring reliable delivery.

What STAR's CDCLink does provide:

- Application-level CRC-32 (end-to-end integrity, covering the full frame including header).
- Sequence number tracking via shared `SessionState`.
- Gap tolerance (accepts sequence gaps < 10 frames for rare USB packet loss).
- Send serialization via `sendMutex` (prevents out-of-order transmission).
- Send timeout protection (50 ms timeout prevents blocking on USB disconnect).

11.4.8 SPI(Serial Peripheral Interface)

SPI is a synchronous 4 - wire bus connecting the Raspberry Pi 5(Controller)to the RX72N(Peripheral) .Unlike USB, SPI is a raw electrical interface with **zero built - in error detection or correction** .

SPI wires:

- **SCLK :** Clock signal(10 MHz), driven by the RPi5 Controller.
- **COPI :** Controller Out, Peripheral In(data from RPi5 to RX72N).
- **CIPO :** Controller In, Peripheral Out(data from RX72N to RPi5) .
- **CS :** Chip Select(active low, selects the RX72N) .

Why SPI needs full HARQ:

- **No hardware CRC :** SPI shifts raw bits.A single bit flip from EMI or crosstalk passes through silently unless the application checks.
- **No retransmission :** SPI is fire - and-forget.If a byte is corrupted, there is no hardware mechanism to request it again.
- **Single - ended signaling :** SPI lines are not differential, making them more susceptible to electromagnetic interference(EMI) from nearby motor drivers, switched power supplies, and PWM signals.
- **Motor environment :** The STAR robot runs 4 brushed DC motors with H - bridge drivers(DRV8243S).PWM switching at 20 kHz generates significant EMI that can couple into SPI traces.

STAR's SPI protection stack:

1. **CRC - 32 :** Detects corruption(every frame) .
2. **FEC encoding :** Adds redundancy (rate 1/2 convolutional code).
3. **ACK / NACK :** Requests retransmission if FEC cannot correct.
4. **Chase Combining :** Accumulates soft bits across retransmissions.
5. **Viterbi decoding :** Finds most likely original data from combined soft bits.

11.4.9 Protocol Composition Hierarchy

The following table summarizes which protocol mechanisms are active on each transport :

Table 57: Protocol Mechanisms by Transport

Mechanism	USB CDC	SPI	Rationale
CRC - 32(application)	✓	✓	End - to - end integrity on both paths
Sequence tracking	✓	✓	Duplicate / ordering detection
ARQ(ACK / NACK)		✓	SPI has no built - in retry
FEC(convolutional)		✓	SPI has no built - in error correction
Viterbi decoding		✓	Decodes FEC - encoded data
Chase Combining		✓	Improves retransmission success
Hardware CRC - 16	✓		USB hardware provides this
Hardware bulk retry	✓		USB hardware provides this
`sendMutex`	✓	✓	Atomic sequence + send
Send timeout	✓		Prevents blocking on USB disconnect

11.4.10 Module Map

The following table maps each protocol concept to its implementation in both the Go gateway and the C firmware :

Table 58: Protocol Implementation Module Map

Concept	Go(Gateway)	C(RX72N)
CRC - 32	`internal / frame /`	`libs / rx_crc /`(HW - accelerated)
Frame encode / decode	`internal / frame /`	`libs / rx_frame /`
FEC encoder	`internal / fec / convolutional.go`	`libs / rx_fec /`
Viterbi decoder	`internal / fec / viterbi.go`	`libs / rx_fec /`
Chase Combiner	`internal / fec / combiner.go`	`libs / rx_harq /`
HARQ state machine	`internal / link / spi.go`	`libs / rx_spi_link /`
CDC link(lightweight)	`internal / link / cdc.go`	`libs / rx_usb_comm /`
SPI raw transport	`internal / transport / spi.go`	`libs / rx_spi_comm /`
USB raw transport	`internal / transport / cdc.go`	`libs / rx_usb_comm /`
Session state	`internal / manager / session.go`	`libs / rx_session /`
Transport manager	`internal / manager / manager.go`	`libs / rx_comm_manager /`

11.5 Command Flow(Downward : UI to Motor)

The following steps trace a velocity command from the user interface down to the motor driver :

1. UI sends JSON over WebSocket to Gateway(: `8080`) .
2. Gateway `GatewayService` caches the `VelocityCommand`(mutex - protected) .
3. ROS2 bridge polls `GetTeleopCommand()` at 50 Hz via gRPC .
4. Gateway `MotorControlService.SetVelocity()` serializes with `proto.Marshal()` .
5. `SPILink.Send()`(or `CDCLink.Send()`) wraps in frame : SYNC + header + CRC - 32.
6. Transmit over active transport(USB CDC or SPI).
7. RX72N : `rx_comm_manager` validates CRC, dispatches to callback.

8. `comm_task`(ThreadX priority 5, 100 Hz)cascade - decodes : tries `rx_nanopb_decode_velocity_request()`, then `rx_nanopb_decode_estop_request()`.
9. On success : builds `motor_command_t`, writes to `shared_data`(mutex + event flag).
10. `motor_control_task`(priority 8, 250 Hz)wakes on event, runs PID, writes PWM to DRV8243S.

Key Insight : The inner motor control loop(250 Hz) runs at 2.5 × the communication rate(100 Hz), ensuring smooth PID control between incoming commands.ThreadX event flags provide zero - copy, zero - latency signaling between the communication and motor tasks.

11.6 Telemetry Flow(Upward : Sensor to UI)

1. RX72N telemetry task(10 Hz) gathers encoder, IMU, and battery data.
2. `rx_nanopb_encode_telemetry()` serializes the data; frame + CRC-32 applied.
3. Transmit over active transport (best-effort, no `REQUIRES_ACK` for telemetry).
4. Gateway dispatcher (100 Hz loop) decodes frame, validates CRC, routes to `TelemetryService`.
5. `GatewayService` caches data; ROS2 bridge publishes to `/ telemetry/*` topics.
6. WebSocket streams to UI clients.

11.7 Frame Protocol Summary

The wire format is shared across both transports. Full specification is in Section **??**.

Table 59: Frame Wire Format

SYNC	SEQ	LEN	TYPE	FLAGS	PAYLOAD	CRC-32
2 B	2 B	2 B	1 B	1 B	0–1024 B	4 B

Table 60: Frame Type Definitions

Value	Name	Description
0x00	PING	Heartbeat request
0x01	PONG	Heartbeat response
0x10	COMMAND	Command frame carrying protobuf messages
0x11	RESPONSE	Response frame carrying protobuf messages
0x12	ACK	Acknowledgment (SPI HARQ protocol)
0x13	NACK	Negative acknowledgment (SPI HARQ protocol)
0xFE	RESET_ACK	Acknowledgment of session reset
0xFF	RESET	Request to reset session (synchronize sequences)

11.7.1 Frame Flags

Frame flags are bitfield values in the FLAGS byte of the header. Multiple flags can be combined:

Table 61: Frame Flag Definitions

Bit	Name	Description
0	`REQUIRES_ACK`	Sender expects ACK/NACK response (SPI HARQ only)
1	`FEC_ENABLED`	Payload is FEC-encoded (convolutional rate 1/2)
2	`RETRANSMIT`	This frame is a retransmission (not first attempt)
3	`SOFT_NACK`	NACK was triggered by soft-decision failure, not hard CRC

See Section **??** for the full frame specification including byte ordering, CRC polynomial, and cross-compatibility test vectors.

11.8 Reliability: SPI HARQ Send/Receive Paths

11.8.1 SPI Send Path (HARQ)

When the gateway sends a command over SPI, the following pipeline executes:

1. `SPILink.Send()` acquires the send mutex (atomic sequence assignment).
2. Payload is FEC-encoded: convolutional encoder produces 2× the input size.
3. Frame is built: SYNC + header (with `REQUIRES_ACK` and `FEC_ENABLED` flags) + encoded payload + CRC-32.
4. Frame is transmitted over SPI via `transport.Send()`.
5. `SPILink` starts a 10 ms ACK timer and waits.
6. If ACK received: success, return to caller.
7. If NACK received or timeout:
   (a) Increment retry counter.
   (b) Set `RETRANSMIT` flag in header.
   (c) Retransmit the same encoded frame.
   (d) Wait for ACK again (up to 3 total attempts).
8. After 3 failures: return error, trigger health monitor.

11.8.2 SPI Receive Path (Chase Combining + Viterbi)

When the RX72N receives a frame on SPI:

1. `rx_spi_link_receive()` reads raw bytes from `rx_spi_comm`.

2. If `FEC_ENABLED` flag is set:
   (a) Convert raw bytes to soft bits: each byte produces 8 soft bits using MSB-first extraction, mapping hard $0 \rightarrow -127$ and hard $1 \rightarrow +127$.
   (b) Feed soft bits to `rx_harq_decode()`.
   (c) If `RETRANSMIT` flag is set, the HARQ module performs Chase Combining: adds new soft bits to the accumulator from the previous attempt, then clamps to $[-127, +127]$.
   (d) Run Viterbi decoder on the combined soft bits.
   (e) Check decoded CRC:
   - CRC valid → send ACK, deliver decoded payload to application.
   - CRC invalid → send NACK, keep accumulator for next retry.
3. If FEC disabled: standard CRC-32 validation only.

11.8.3 USB CDC Path: No HARQ

USB CDC does not require application-level HARQ. The USB hardware provides CRC-16 on every packet and automatic bulk retry on error, making ARQ and FEC redundant. The `CDCLink` implementation (`internal/link/cdc.go` and `libs/rx_usb_comm/`) contains **zero references** to HARQ, FEC, Viterbi, or Chase Combining code.

11.9 Safety Features

- **Communication watchdog (500 ms):** The RX72N triggers an emergency stop if no valid frame is received within 500 ms, ensuring the robot halts on link failure.
- **Duplicate detection:** Shared `SessionState` sequence numbers detect and discard duplicate frames across both transports.
- **CRC-32 (IEEE 802.3):** Every frame is protected by CRC-32, hardware-accelerated on the RX72N CRC Calculator peripheral.
- **Failover continuity:** Shared `SessionState` prevents sequence desynchronization during USB↔SPI failover.
- **Atomic sequence assignment:** A `sendMutex` ensures that sequence number assignment and frame transmission are atomic, preventing out-of-order frames from concurrent goroutines.

Important: The 500 ms communication watchdog is the last line of defense. If all transports fail and no valid frame arrives within this window, the RX72N immediately disables all motor outputs and enters the emergency stop state. Recovery requires an explicit reset command after communication is restored.

11.10 Latency Budget

Table 62: Latency Budget: Transport to Motor Control

Stage	Time
SPI ISR frame reception	∼5 µs
Frame reassembly + CRC validation	∼50 µs
Comm manager dispatch	∼5 µs
nanopb protobuf decode[1]	∼15 µs
Protobuf validation + processing	∼100 µs
`shared_data` mutex update	∼7 µs
ThreadX scheduling overhead	∼18 µs
ThreadX context switch (priority 5→8)	∼50 µs
Motor control task wakeup	∼50 µs
PID control update	∼20 µs
Total (transport to motor)	**∼320 µs**
End-to-end (WebSocket to motor)	**∼10–20 ms**

The firmware-side processing (∼320 µs) is dominated by CRC validation and the ThreadX context switch. The end-to-end latency is governed by network round-trip time and gRPC serialization on the RPi5 side.

11.11 Cross-References

- **Section ??** (01_nanopb_protocol.tex): Full frame specification, HARQ protocol details, CRC-32 implementation, and memory budget
- **Section 6** (07_gateway_architecture.tex): Gateway service architecture, gRPC service definitions, and directory structure
- **Section ??** (09_usb_cdc_protocol.tex): USB CDC transport details, descriptor hierarchy, bulk transfer protocol, and heartbeat mechanism
- **Section ??** (13_transport_failover.tex): Auto fallback, hot switching, health monitoring, and failover timing
- **Section ??** (14_dual_channel_arbitration.tex): How the RX72N handles simultaneous data on both channels
- **ADR-001**: HARQ architectural decision record — rationale for convolutional coding, Chase Combining strategy, and phased implementation plan

== == == == == == == == == == == == = =

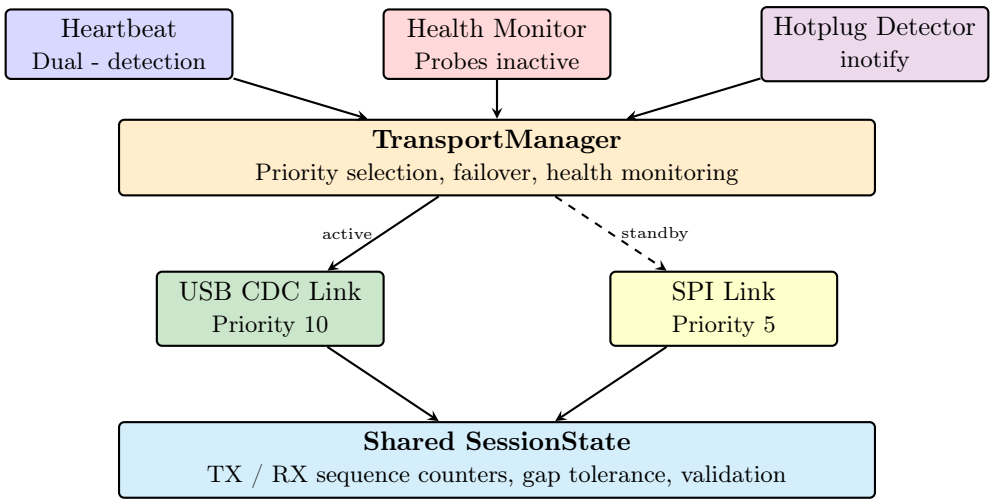
[1] This timing represents the pure nanopb decode operation (`pb_decode()`). The total `comm_task` processing time documented in `comm_task.c` (∼200 µs for `SetVelocityRequest`) includes additional overhead from cascade decoding (trying multiple message types), protobuf field validation, and motor command structure population. The breakdown shown in this table isolates individual component timings for latency analysis. Measurements obtained at 240 MHz with -O2 optimization using RX72N cycle counter.

12 Transport Failover and Hot Switching

This section documents how the STAR gateway automatically detects transport failures, switches between USB CDC and SPI, recovers when a failed transport comes back online, and maintains communication continuity throughout.

12.1 Architecture Overview

The gateway's `TransportManager` (`internal/manager/manager.go`) orchestrates all failover logic. It maintains exactly **one active transport** at any time and proxies all `Send()`/`Receive()` operations through it. The RX72N firmware is a **passive listener** — it does not participate in transport selection decisions.



Key Insight : The gateway sends on exactly one transport at a time .It never sends the same command on both USB and SPI simultaneously .The RX72N firmware listens on both channels but the gateway's single-active-transport design means only one will have data at any given moment (except during brief failover transitions).

12.2 Priority System

Each transport is registered with a numeric priority.Higher values indicate preferred transports :

Table 63: Transport Priority Configuration

Transport	Priority	Role	Rationale
USB CDC	10	Primary	Lower latency, hardware CRC, no HARQ overhead
SPI	5	Backup	Always available, full HARQ protection

Priority rules:

1. At startup, the transport with the highest priority that is healthy becomes active.
2. If the active transport fails, switch to the highest - priority healthy alternative.
3. If a higher - priority transport recovers, switch back to it(after damping period).
4. Priority values are constants defined at compile time, not configurable at runtime.

12.3 Dual - Detection Heartbeat

The heartbeat system uses two independent mechanisms to detect transport failures.This provides both fast failure detection(implicit) and explicit liveness checking(PING / PONG) .

12.3.1 Implicit Heartbeat(200 ms Timeout)

Any valid frame received on a transport resets that transport's implicit heartbeat timer. If no frame arrives within 200 ms, the transport is considered unhealthy.

- **Timer reset :** Every valid frame(COMMAND, RESPONSE, TELEMETRY, PONG, ACK) resets the timer.
- **Timeout:** 200 ms with no frames → mark transport as `degraded` .
- **No extra traffic :** Piggybacked on existing data flow.In normal operation at 100 Hz(10 ms between frames), the implicit timer never fires.
- **Fast detection :** If the USB cable is unplugged mid - stream , detection occurs within 200 ms.

12.3.2 Explicit Heartbeat (1 s PING/PONG)

When the link is idle (no data frames for 1 second), the gateway sends a PING frame and expects a PONG response within 200 ms.

- **PING interval:** Sent after 1 second of idle (no other frames sent or received).
- **PONG timeout:** 200 ms.If no PONG arrives, the transport is marked unhealthy .
- **Purpose :** Detects silent failures where the physical connection exists but the remote device is unresponsive(e.g., firmware crash, watchdog reset) .
- **Frame types :** PING = `0x00`, PONG = `0x01`(see frame type table in Section refsec : communication - stack) .

Why two mechanisms ?

- **Implicit** catches failures during active communication(fast : 200 ms) .
- **Explicit** catches failures during idle periods(slower : 1 s + 200 ms) .
- Neither generates unnecessary traffic alone.Combined, they cover all scenarios.

12.4 Health Monitoring

The `HealthMonitor`(`internal / manager / health_monitor.go`) runs a background goroutine that periodically probes **inactive** transports to detect when they recover .It does not interfere with the active transport.

12.4.1 Health Thresholds

A transport is considered unhealthy if **any** of the following conditions are met :

Table 64: Health Monitoring Thresholds

Metric	Threshold	Detection Method
Consecutive failures	≥ 3	Send or receive returns error 3 times in a row
Average latency	> 200 ms	Exponential moving average of round - trip times
Packet loss rate	$> 10\%$	Sliding window over recent operations

12.4.2 Probing Inactive Transports

The health monitor checks inactive transports every 5 seconds (configurable via `HealthCheckInterval`):

1. Select all transports that are **not** the currently active transport.
2. For each inactive transport, send a PING and wait for PONG.
3. If PONG received within timeout: mark transport as healthy, record recovery timestamp.
4. If probe fails: keep transport marked as unhealthy.
5. If a recovered transport has **higher priority** than the current active transport, trigger failover (subject to damping window).

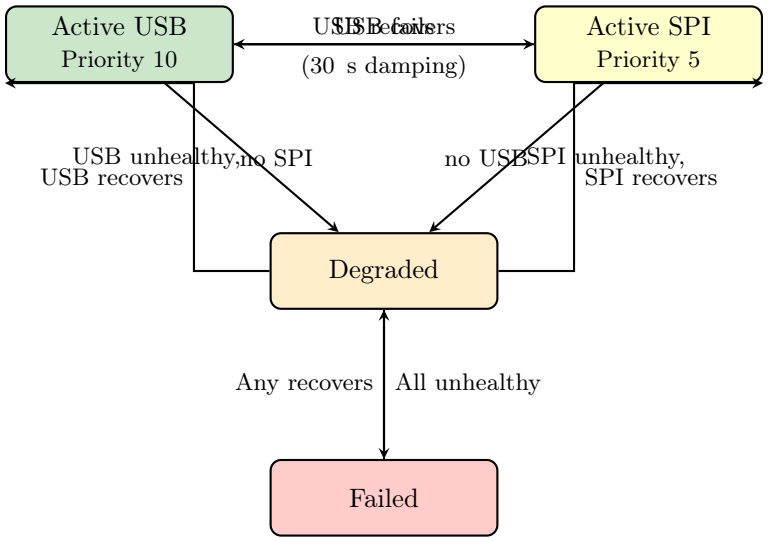
12.5 Failover State Machine

The `TransportManager` operates as a state machine with the following states :

Table 65: Transport Manager States

State	Constant	Description
Active USB	`StateActiveUSB`	USB CDC is the active transport, SPI is standby
Active SPI	`StateActiveSPI`	SPI is the active transport, USB is unavailable or unhealthy
Degraded	`StateDegraded`	Active transport is unhealthy but no better alternative exists
Failed	`StateFailed`	No healthy transports available

12.5.1 State Transitions



12.6 Smart Drain

When switching away from a transport, the gateway uses "smart drain" to handle in-flight frames gracefully:

- **Hard failure (IO error, disconnect):** Skip drain entirely. Switch immediately. In-flight frames are lost but speed is critical — the motor control loop cannot afford a 500 ms pause.
- **Graceful degradation (high latency, packet loss):** Wait up to 500 ms for in-flight frames to complete before switching. This avoids losing frames that are nearly delivered.

  Failover timing with smart drain:

Table 66: Failover Timing by Failure Type

Failure Type	Detection	Total Failover Time
USB cable unplugged	∼200 ms(implicit timeout)	< 300 ms(no drain)
USB device hang	∼1 .2 s(PING timeout)	< 1.5 s(no drain)
High latency	∼200 ms(threshold)	< 700 ms(500 ms drain)
RX72N firmware crash	∼1 .2 s(PING timeout)	< 1.5 s(no drain)

Important : The RX72N's 500 ms communication watchdog monitors *total* communication silence across all transports, not individual transport health. During failover, the gateway completes the transport switch (pause → drain → activate standby) and resumes sending on the new transport. Worst-case timing:

- **Hard failure**(cable pull, IO error) : < 300 ms(no drain, immediate switch)
- **Graceful / timeout** : < 700 ms(500 ms drain + 200 ms switch overhead)
- **Silent failure**(PING timeout) : Up to 1.5 s detection + switch time

For silent failures exceeding 500 ms, the RX72N watchdog **will trigger**, initiating safe motor shutdown.The gateway detects this via health monitoring and resumes communication on the new transport.This is the intended fail - safe behavior : motor safety takes precedence over seamless failover.

12.7 Shared SessionState(Preventing the Handoff Problem)

The "Handoff Problem" occurs when two transports maintain separate sequence counters :

1. Gateway sends frames 1 –100 over USB(USB sequence = 100, SPI sequence = 0) .
2. USB fails.Gateway switches to SPI.
3. If SPI has its own counter starting at 0, the RX72N sees sequence "jump" from 100 to 0.
4. The RX72N rejects frame 0 as a duplicate(already seen sequence 0 from USB earlier) .
5. **Result :** Total communication failure after failover.

Solution : Both transports share a single `SessionState` object that owns the TX and RX sequence counters :

```
        SessionState struct {
  mu sync.Mutex txSequence uint16 // Shared TX counter (both USB and SPI)
      rxSequence uint16           // Shared RX counter (both USB and SPI)
          gapMax uint16 // Maximum acceptable sequence gap (default: 10)
}

func(s *SessionState) NextTxSequence() uint16 {
  s.mu.Lock() defer s.mu.Unlock() seq : = s.txSequence s.txSequence++
     return seq
}
```

Listing 25: Shared SessionState(manager / session.go)

After failover : Gateway sends frames 1 –100 over USB, then frame 101 over SPI.The RX72N sees a continuous sequence regardless of which physical transport delivered it .

12.7.1 Gap Tolerance

The shared `SessionState` accepts sequence gaps of up to 10 frames(`gapMax = 10`) .This handles :

- **Rare USB packet loss :** A frame lost during transmission creates a gap of 1. The next frame(sequence $N + 1$) is accepted despite the missing N.
- **Failover transitions :** During the switch, 1 –3 frames may be in - flight on the old transport and never delivered .The new transport picks up at the next sequence number .

- **Duplicate rejection :** Frames with sequences $\leq$ the last accepted sequence are rejected as duplicates(prevents replay) .

12.8 USB Hot - Plug Detection

The gateway uses Linux `inotify` to detect USB device attach / detach events in real time, without polling :

- **Watch path:** `/ sys / bus / usb / devices`(Linux sysfs)
- **Events:** `IN_CREATE`(device attached), `IN_DELETE`(device removed)
- **Latency :** Near - instant(< 50 ms from physical plug / unplug to software notification)
- **Filter :** Only reacts to changes matching the RX72N's USB VID/PID (`045B:0261`, Renesas RX)

Hot - plug flow:

1. USB cable plugged in $\rightarrow$ inotify fires `IN_CREATE`.
2. Hotplug detector opens `/ dev / ttyACM0`, creates USB transport and CDC link .
3. Registers new transport with `TransportManager`(priority 10) .
4. If USB has higher priority than current transport $\rightarrow$ switch to USB .
5. USB cable unplugged $\rightarrow$ inotify fires `IN_DELETE` .
6. Hotplug detector notifies `TransportManager` $\rightarrow$ immediate failover to SPI.

12.9 Failback Damping(30 - Second Cooldown)

When a previously failed transport recovers, the gateway does **not** switch back immediately. A 30-second damping window prevents rapid oscillation ("flapping") between transports:

- **Scenario:** USB fails, gateway switches to SPI. USB recovers 2 seconds later.
- **Without damping:** Gateway immediately switches back to USB. If USB is intermittent, it fails again 1 second later. Gateway switches to SPI again. This "flapping" disrupts communication.
- **With damping:** Gateway records the USB recovery timestamp. Waits 30 seconds. Only if USB has been continuously healthy for 30 seconds does the gateway switch back.

Damping parameters:

Table 67: Failback Damping Parameters

Parameter	Value	Description
Damping window	30 s	Minimum healthy duration before failback
Health check rate	5 s	How often inactive transports are probed
Recovery condition	6 consecutive healthy probes	6×5 s $= 30$ s of health

12.10 Reset Handshake(Gateway Restart Recovery)

When the gateway restarts(process crash, reboot, update), the RX72N may still be running with sequence counters at, say, 5000. The restarted gateway's counters start at 0. Without synchronization, the RX72N would reject all frames as out-of-sequence.

Three - way reset handshake:

1. **Gateway → RX72N :** Send `RESET` frame(type `0xFF`) .
2. **RX72N → Gateway :** Respond with `RESET_ACK` frame(type `0xFE`) .RX72N resets its TX / RX counters to 0.
3. **Gateway :** Receives `RESET_ACK`, resets its own SessionState counters to 0.
4. Both sides are now synchronized at sequence 0. Normal communication resumes.

Timeout : If no `RESET_ACK` is received within 500 ms, the gateway retries the `RESET` frame(up to 3 times) .If all retries fail, the gateway assumes the RX72N is unreachable and enters the `StateFailed` state.

Important : The reset handshake prevents a "restart deadlock" where the gateway cannot communicate because sequences are mismatched, and the RX72N cannot be told to reset because it rejects the out - of - sequence reset frame.The RESET frame type is handled specially by the RX72N– - it is always accepted regardless of sequence number.

12.11 Firmware Side : Passive Listener

The RX72N firmware does not participate in transport selection.It is a **passive listener** that :

1. Listens on both USB CDC and SPI simultaneously .
2. Accepts valid frames from either channel(validated by CRC - 32 and sequence number) .
3. Does not know or care which transport the gateway considers "active ."
4. Routes all received commands through the same callback(`internal_frame_callback`) .
5. Sends telemetry on whichever channel last received a command(implicit response routing) .

The `rx_comm_manager_poll()` function(textttrx_comm_manager.c : 1317) polls both channels sequentially every 10 ms(100 Hz) :

```
    rx_comm_manager_poll(rx_comm_manager_t * mgr) {
  /* Poll USB first */
  rx_err_t usb_err = internal_poll_usb(mgr);
  /* Poll SPI second */
  rx_err_t spi_err = internal_poll_spi(mgr);
  /* Process event queue */
  internal_process_event_queue(mgr);
  /* Check heartbeat timers */
  internal_check_heartbeat(mgr);
  return received ? k_rx_ok : k_rx_err_timeout;
}
```

Listing 26: RX72N Dual - Channel Polling

12.12 Timing Summary

Table 68: Transport Failover Timing Summary

Event	Timing
Implicit heartbeat timeout	200 ms
Explicit PING interval	1 s idle
PING / PONG timeout	200 ms
Health check interval	5 s
Smart drain(graceful)	up to 500 ms
Smart drain(hard failure)	0 ms(immediate)
Failback damping window	30 s
Reset handshake timeout	500 ms per attempt, 3 retries
RX72N comm watchdog	500 ms
USB hot - plug detection	< 50 ms
Worst - case failover(cable pull)	< 300 ms
Worst - case failover(silent)	< 1.5 s

12.13 Cross - References

- **Section ??** : Protocol stack, HARQ details, frame format
- **Section ??**: How the RX72N handles simultaneous data on both channels
- **Section 6**: Gateway service architecture and directory structure
- **Section ??**: USB CDC transport details
- `star-gateway/internal/manager/manager.go`: TransportManager implementation
- `star-gateway/internal/manager/heartbeat.go`: Dual-detection heartbeat
- `star-gateway/internal/manager/health_monitor.go`: Health probing for inactive transports
- `star-gateway/internal/manager/hotplug_detector.go`: USB inotify-based hot-plug detection
- `star-gateway/internal/manager/session.go`: Shared SessionState

13 Dual - Channel Arbitration on the RX72N

This section documents how the RX72N firmware handles the case where data arrives on both USB CDC and SPI simultaneously, what ordering guarantees exist, and why the architecture prevents conflicts .

13.1 The Question

The RX72N listens on both USB CDC and SPI at all times .What happens if frames arrive on both channels at the same time, potentially carrying different data ?

13.2 Gateway Guarantee: Single Active Transport

Under normal operation, this scenario **cannot occur** because the gateway's `TransportManager` maintains exactly one active transport:

```
                                Send(ctx context.Context, data[] byte, ...)
                                    error {
  tm.mu.RLock() active : = tm.activeTransport // Only ONE transport
                                    activeName
      : = tm.activeTransportName tm.mu.RUnlock()

              if active ==
          nil{return errors.New("no active transport available")}

          err : = active.Send(ctx, data, p...) // Send on THAT one only
                                               // ...
}
```

Listing 27: Gateway sends on ONE transport only(manager.go)

The gateway **never** sends the same command on both USB and SPI simultaneously.It selects one(USB preferred, priority 10) and sends exclusively on that.

13.3 When Overlap Can Occur

Despite the single - active - transport guarantee, brief overlap is possible during failover transitions :

1. **In - flight frame during failover :** Frame N was transmitted on USB and is still being processed by the USB hardware when the gateway detects a failure and switches to SPI.Frame $N + 1$ is sent on SPI.The RX72N receives both frames in the same 10 ms poll cycle.
2. **Telemetry crossing :** The RX72N sends telemetry on USB(because the last command came from USB), while the gateway has already switched to SPI and sends a new command on SPI .The firmware receives the SPI command while USB telemetry is outgoing.

13.4 Sequential Polling : USB First, Then SPI

The `rx_comm_manager_poll()` function processes channels **sequentially**, not in parallel.There is no race condition because the entire function runs in a single thread(`comm_task`, ThreadX Priority 5, 100 Hz) :

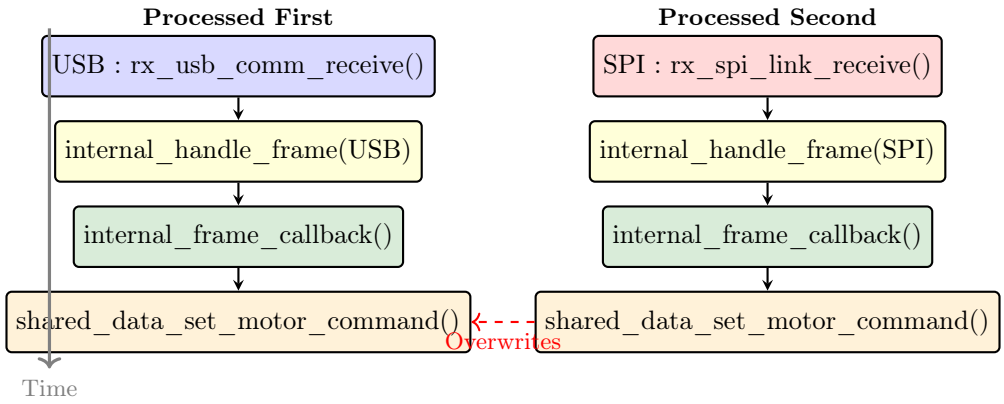
```
          rx_comm_manager_poll(rx_comm_manager_t * mgr) {
  bool received = false;

  /* Step 1: Poll USB channel */
  rx_err_t usb_err = internal_poll_usb(mgr);
  if (usb_err == k_rx_ok) {
    received = true; // USB frame processed
 }

  /* Step 2: Poll SPI channel */
  rx_err_t spi_err = internal_poll_spi(mgr);
  if (spi_err == k_rx_ok) {
    received = true; // SPI frame processed
 }

  /* Step 3: Process event queue */
  internal_process_event_queue(mgr);

  /* Step 4: Check heartbeat timers */
  internal_check_heartbeat(mgr);

  return received ? k_rx_ok : k_rx_err_timeout;
}
```

Listing 28: Sequential channel polling(rx_comm_manager_poll)

Ordering guarantee : Within any single poll cycle, USB is always processed before SPI.This is deterministic and repeatable.

13.5 Frame Processing Pipeline

Both `internal_poll_usb()` and `internal_poll_spi()` follow the same pipeline :



13.6 Last Writer Wins

Both frames call `shared_data_set_motor_command(& cmd)`, which is mutex - protected.Since the two calls execute sequentially(not concurrently), the second call **overwrites** the first :

1. USB frame arrives → `shared_data_set_motor_command()` writes motor velocities from USB frame.
2. SPI frame arrives → `shared_data_set_motor_command()` **overwrites** with motor velocities from SPI frame.
3. Motor task wakes up → reads the **SPI frame's data** (the last one written).

The SPI command wins because it is polled second. The USB command is effectively superseded.

13.7 Why This Is Correct During Failover

During a failover, the overlap works **correctly** because the gateway uses shared `SessionState` sequence numbers:

1. The gateway increments the sequence number for each `Send()` call (shared counter).
2. Frame N goes out on USB (the old transport).
3. Frame $N + 1$ goes out on SPI (the new transport).
4. The RX72N processes USB (N) first, then SPI ($N + 1$) second.
5. The motor task gets frame $N + 1$, which is the **newer** command.
6. This is the correct behavior — the motor should execute the most recent command.

Key Insight : The "last writer wins" semantics combined with sequential polling(USB before SPI) and monotonically increasing sequence numbers guarantee that the motor task always receives the most recent command, even during transport failover.

13.8 Sequence Validation

The shared `rx_session` module on the RX72N validates every received sequence number :

- **Duplicate detection :** If the same sequence number is received twice (e.g., frame retransmitted on both channels), the second copy is rejected by `rx_session_validate_rx()`.
- **Gap tolerance :** Sequence gaps of up to 10 frames are accepted (see Section 12.7). This handles frames lost during failover.
- **Wraparound:** 16 - bit sequence numbers wrap from 65535 to 0. The validation logic handles this correctly.

13.9 Channel Parameter(Currently Unused)

The `internal_handle_command_frame()` function receives a `channel` parameter indicating whether the frame arrived on USB or SPI :

```
    internal_handle_command_frame(rx_comm_channel_t channel,
                                  const rx_frame_t *frame) {
  (void)channel; // Currently unused
                 // ... decode protobuf, update shared_data ...
}
```

Listing 29: Channel parameter in command handler(internal_handle_velocity_request)

The channel parameter is currently cast to `void` (unused). A future enhancement could use it to:

- Route response/telemetry back on the same channel that sent the command.
- Log which channel is actively receiving commands for diagnostics.
- Implement channel-specific command filtering (e.g., only accept emergency stop from USB).

13.10 Communication Watchdog Interaction

The 500 ms communication watchdog(`shared_data_update_last_comm_tick()`) is updated on **every valid frame from either channel** :

```
            internal_frame_callback(rx_comm_channel_t channel,
                                    const rx_frame_t *frame, void *ctx) {
  (void)ctx;
  if (frame == nullptr) {
    return;
 }

  /* Update watchdog on ANY valid frame from ANY channel */
  shared_data_update_last_comm_tick();

  switch (frame->header.type) {
  case k_frame_type_command:
    internal_handle_command_frame(channel, frame);
    break;
    // ...
 }
}
```

Listing 30: Watchdog update on any frame(internal_update_watchdog)

This means :

- During failover, if USB stops sending but SPI starts, the watchdog is still satisfied.
- The 500 ms timer only triggers emergency stop if **both** channels are completely silent .
- A single valid frame on either channel resets the watchdog and prevents emergency stop.

13.11 Summary: Dual - Channel Behavior Matrix

Table 69: RX72N Behavior for Dual-Channel Scenarios

Scenario	What Happens	Result
Normal operation	Gateway sends on ONE channel only	Single frame per cycle
Failover overlap	Both channels have data	USB processed first, SPI overwrites (newer command = correct)
Duplicate frame	Same sequence on both channels	Sequence validation rejects second
Different senders	Cannot happen (single gateway)	Architecture prevents this
All channels silent	No frames for 500 ms	Emergency stop triggered
One channel fails	Other channel still delivers frames	Watchdog still satisfied

13.12 Cross - References

- **Section ??** : Protocol stack, frame format, CRC validation
- **Section 12** : Transport failover, health monitoring, shared SessionState
- `e2 - studio - star - rx72n - firmware / libs / rx_comm_manager / src / rx_comm_manager.c` : Dual - channel polling implementation
- `e2 - studio - star - rx72n - firmware / src / tasks / comm_task.c` : Frame callback, command dispatch, watchdog update
- `star - gateway / internal / manager / manager.go` : Single - active - transport guarantee